Universitatea de Vest din Timişoara
Facultatea de Matematică şi Informatică

METODE NUMERICE

PROBLEME DE SEMINAR
ŞI LUCRĂRI DE LABORATOR

Simina Mariş Liliana Brăescu

Timişoara

2007

Introducere

Procesul de restructurare al Învăţământului Superior din România şi trecerea acestuia pe trei cicluri, a determinat la nivelul întregii ţări elaborarea de noi planuri de învăţământ şi de programe analitice adecvate.

Metode numerice - Probleme de seminar şi lucrări de laborator este un material adiţional la cursul de *Metode numerice* elaborat în acord cu noile cerinţe, pe baza programei analitice concepută la nivelul Departamentului de Informatică şi aprobată în Consiliul Profesoral al Facultăţii de Matematică şi Informatică de la Universitatea de Vest din Timişoara.

Problemele şi lucrările de laborator prezentate în această carte se adresează în primul rând studenţilor de la Facultatea de Matematică şi Informatică, fiind abordate toate temele din programa analitică, la nivelul studenţilor Secţiei de Informatică aflaţi în semestrul al cincilea de studiu, oferind exemple şi detalii referitoare la metodele numerice prezentate în curs.

Lucrarea este structurată pe şapte capitole, primul dintre acestea fiind rezervat pentru prezentarea unui set de cunoştinţe minimale de programare în Maple. Capitolele 2-7 corespund capitolelor din cursul de *Metode numerice* şi sunt organizate după cum urmează:

- breviar teoretic
- problemă rezolvată
- probleme propuse
- implementare

Prin această lucrare, autorii pun la dispoziţia cititorilor toate cunoştinţele necesare în vederea construirii de algoritmi şi proceduri capabile să ia ca argument un obiect matematic şi să returneze un rezultat final.

Autorii

Lista proiectelor

1. Metoda lui Gauss cu pivot total. Rezolvarea unui sistem
2. Inversa unei matrice. Rezolvarea unui sistem
3. Factorizarea LU Doolittle. Rezolvarea unui sistem
4. Factorizarea Cholesky. Rezolvarea unui sistem
5. Factorizarea Householder. Rezolvarea unui sistem
6. Metoda Gauss-Seidel. Comparaţie cu metoda lui Jacobi şi cu soluţia exactă
7. Metoda relaxării succesive. Comparaţie cu metoda Gauss-Seidel şi cu soluţia exactă
8. Metoda lui Newton simplificată în dimensiunea n. Comparaţie cu metoda lui Newton clasică în dimensiunea n
9. Metoda lui Newton simplificată în dimensiunea 1. Comparaţie cu metoda lui Newton clasică. Reprezentare intuitiva.
10. Polinomul lui Newton cu diferenţe finite la dreapta. Comparaţie pentru o funcţie cunoscută
11. Polinomul lui Newton cu diferenţe finite la stânga. Comparaţie pentru o funcţie cunoscută
12. Functia spline liniară. Comparaţie pentru o funcţie cunoscută
13. Polinoame Bernstein. Comparaţi cu polinomul Lagrange pentru o funcţie cunoscută.
14. Aproximarea derivatei prin diferenţe finite. Comparaţie cu valoarea exactă şi între diferite valori ale pasului h.
15. Formule de tip Gauss de ordinul 3, 4. Comparaţie cu rezultatul exact.
16. Metoda lui Taylor de ordinul 3. Comparaţie cu rezultatul exact.
17. Metoda Runge-Kutta de ordinul 3. Comparaţie cu rezultatul exact, pentru diverse valori ale parametrilor.
18. Metoda Runge-Kutta de ordinul 4. Comparaţie cu rezultatul exact, pentru diverse valori ale parametrilor.
19. Metoda Adams-Bashforth de ordinul 3. Comparaţie cu rezultatul exact, pentru diverse valori ale parametrilor.
20. Metoda Adams-Bashforth de ordinul 4. Comparatie cu rezultatul exact, pentru diverse valori ale parametrilor.
21. Metoda Adams-Bashforth de ordinul 5. Comparatie cu rezultatul exact, pentru diverse valori ale parametrilor.

Cuprins

1 `MapleV4` - scurtă introducere **4**
- 1.1 Reguli generale de introducere a comenzilor . . . 4
- 1.2 Pachete de programe . . . 6
- 1.3 Constante, operatori şi funcţii des utilizate . . . 7
- 1.4 Structuri de date . . . 8
- 1.5 Calcule cu matrice şi vectori. Pachetul `linalg` . . . 10
- 1.6 Grafice . . . 13
- 1.7 Elemente de programare . . . 15

2 Rezolvarea sistemelor liniare **21**
- 2.1 Metoda lui Gauss . . . 21
- 2.2 Factorizarea LU . . . 33
- 2.3 Sisteme tridiagonale . . . 40
- 2.4 Factorizarea Cholesky . . . 47
- 2.5 Factorizarea Householder . . . 49
- 2.6 Metoda Jacobi . . . 54
- 2.7 Metoda Gauss-Seidel . . . 61
- 2.8 Metoda relaxării succesive . . . 64

3 Ecuaţii şi sisteme de ecuaţii neliniare **68**
- 3.1 Metoda punctului fix . . . 68
- 3.2 Metoda lui Newton . . . 73

4 Interpolare polinomială. Funcţii spline **83**
- 4.1 Polinomul lui Newton cu diferenţe divizate . . . 83
- 4.2 Polinomul de interpolare Lagrange . . . 92
- 4.3 Interpolare spline . . . 95
- 4.4 Polinoame Bernstein . . . 104

5 Derivare numerică **107**
- 5.1 Aproximarea derivatei prin diferenţe finite . . . 107
- 5.2 Aproximarea derivatei . . . 109

6 Integrare numerică **112**
- 6.1 Formule de tip Newton-Cotes . . . 112
- 6.2 Formule de tip Gauss . . . 115

7 Ecuaţii diferenţiale **118**
7.1 Metoda diferenţelor finite 118
7.2 Metoda lui Taylor 124
7.3 Metoda Runge-Kutta 128
7.4 Metoda Adams-Bashforth 132
7.5 Metoda Adams-Moulton 136
7.6 Metoda predictor-corector 137
7.7 Probleme la limită liniare 141
7.8 Metoda colocaţiei şi metoda celor mai mici pătrate 147

Capitolul 1

MapleV4 - scurtă introducere

Maple este un sistem de calcul algebric (CAS) dezvoltat de firma Maplesoft (`http://www.maplesoft.com`), care poate fi utilizat în:

1. calcule simbolice;
2. calcule numerice;
3. programarea unor metode numerice;
4. reprezentări grafice.

În cele ce urmează, vom prezenta principalele elemente necesare în programarea unor metode numerice, corespunzătoare softului `MapleV4`.

1.1 Reguli generale de introducere a comenzilor

Un document `MapleV4` poate avea patru tipuri de câmpuri:

1. comenzi Maple (introduse de către utilizator);
2. rezultate Maple (răspunsuri ale CAS-ului la comenzile introduse);
3. grafice (răspunsuri ale CAS-ului);
4. texte (introduse de către utilizator).

În continuare, vom prezenta câteva reguli de introducere a comenzilor.

1. Orice comandă se termină cu ; (dacă dorim să afişeze rezultatul) sau : (dacă nu dorim ca rezultatul să fie afişat). De exemplu:

```
> sin(Pi);
```

$$0$$

```
> 1+3:
```

2. Asignarea se face cu := , iar dezasignarea se face prin asignarea numelui variabilei. De exemplu, putem avea secvenţa:

```
> x:= 7;
```

$$x := 7$$

```
> x:=x+1:
> x;
```

$$8$$

```
> x:='x';
```

$$x := 'x'$$

```
> x;
```

$$x$$

3. Comentariile sunt precedate de caracterul #. De exemplu:

```
> x:=3; # se atribuie lui x valoarea 3
```

$$x := 3$$

4. Maple face diferenţa între litere mici şi litere mari:

```
> x:=3; X:=5; a:=X-x;
```

$$x := 3$$

$$X := 5$$

$$a := 2$$

5. Secvenţele sunt scrise între paranteze rotunde, ( ), listele între paranteze pătrate, [ ], iar mulţimile între acolade, {}. De exemplu:

```
> secv:=(1,2,3); lista:=[2,1,2,3]; multime:={2,1,2,3};
```

$$secv := 1, 2, 3$$

$$lista := [2, 1, 2, 3]$$

$$multime := \{1, 2, 3\}$$

6. Argumentele unei funcţii se pun între paranteze rotunde, () , iar indicii între paranteze pătrate, [] .

```
> a:=cos(Pi); b:=lista[2];
```

$$a := -1$$

$$b := 1$$

7. Procentul, % , face referire la ultima comandă executată anterior. De exemplu:

```
> a:=2;
```

$$a := 2$$

```
> b:=%+1;
```

$$b := 3$$

8. Dacă rezultatul furnizat de Maple este identic cu comanda introdusă (Maple răspunde prin ecou la comandă), atunci aceasta arată că Maple nu poate interpreta comanda introdusă. Pentru a remedia situaţia, verificaţi dacă aţi introdus corect comanda sau dacă nu cumva funcţia utilizată face parte dintr-un pachet care trebuie încărcat în prealabil.

```
> arctg(1); # o incercare de a calcula arctangenta de 1
```

$$\mathrm{arctg}(1)$$

```
> arctan(1); # apelarea corecta a functiei arctangenta
```

$$\frac{\pi}{4}$$

9. Pentru a nu reţine eventuale atribuiri anterioare, este util ca pentru rezolvarea unei probleme noi să începem cu instrucţiunea

```
> restart;
```

1.2 Pachete de programe

Pachetele sunt colecţii de funcţii care permit efectuarea de calcule specifice. Apelarea lor se face cu ajutorul comenzii `with(nume_pachet)`. Pentru a apela o anumită funcţie dintr-un pachet, se foloseşte sintaxa:
`pachet['functie'](argumente)`

Printre cele mai utilizate pachete sunt:
`plots` - pentru reprezentări grafice;
`DEtools` - pentru rezolvarea ecuaţiilor diferenţiale;
`linalg` - pentru rezolvarea unor probleme de algebră liniară;
`student` - pentru analiză matematică.

De exemplu, la apelarea pachetului grafic, se obţine lista tuturor funcţiilor apelabile:

```
> with(plots);

 Warning, the name changecoords has been redefined

  [animate, animate3d, changecoords, complexplot, complexplot3d,
        conformal, contourplot, contourplot3d, coordplot,
        coordplot3d, cylinderplot, densityplot, display,
        display3d, fieldplot, fieldplot3d, gradplot, gradplot3d,
        implicitplot, implicitplot3d, inequal, listcontplot,
        listcontplot3d, listdensityplot, listplot, listplot3d,
        loglogplot, logplot, matrixplot, odeplot, pareto,
        pointplot, pointplot3d, polarplot, polygonplot,
        polygonplot3d, polyhedraplot, replot, rootlocus,
        semilogplot, setoptions, setoptions3d, spacecurve,
        sparsematrixplot, sphereplot, surfdata, textplot,
        textplot3d, tubeplot]
```

1.3 Constante, operatori şi funcţii des utilizate

Constantele folosite de Maple sunt:

Constantă	Semnificaţie
`false`	"fals"
`true`	"adevărat"
`gamma`	constanta lui Euler
`infinity`	$+\infty$
`Catalan`	constanta lui Catalan
`Fail`	valoare de adevăr necunoscută
`Pi`	π
`I`	unitatea imaginară
`NULL`	secvenţa vidă

Operatorii folosiţi frecvent sunt:

Operator	Sintaxă	Semnificaţie
`+`, `-`	`a+b`, `a-b`	suma $a+b$ (diferenţa $a-b$)
`*`	`a*b`, `2*a`	produsul $a \cdot b$, sau $2a$
`/`	`a/b`	câtul $\frac{a}{b}$
`^`, `**`	`a^b`, `a**b`	puterea a^b
`!`	`n!`	factorialul $1 \cdot 2 \cdot ... \cdot n$
`max`, `min`	`max(a,b,c)`, `min(a,b,c)`	maximul (minimul) dintre a, b, c
`<`,`<=`,`>`,`>=`,`=`,`<>`		operatori booleeni
`:=`	`f:=expr`	operatorul de asignare $f = expr$
`=`	`a=b`	ecuaţia $a = b$
`..`	`x=a..b`	$a \leq x \leq b$
`and`, `or`, `xor`, `implies`, `not`		operatori logici

Funcţii folosite frecvent în Maple:

Funcţie	Sintaxă	Semnificaţie
`sin`, `cos`, `tan`, `cot`	`sin(x)` , ...	funcţii trigonometrice
`arcsin`, `arctan`, `arccos`	`arctan(x)`, ...	
`ln`, `log10`	`ln(x)`, `log10(x)`	logaritmi
`exp`	`exp(x)`, exp(1)	funcţia exponenţială
`sqrt`	`sqrt(x)`	radical
`abs`	`abs(x)`	modul

1.4 Structuri de date: secvenţe, liste, mulţimi, şiruri de caractere

1.4.1 Secvenţe

O secvenţă este o înşiruire de expresii, separate prin virgule. Există mai multe moduri de a defini o secvenţă:

a. direct:

```
> s:=1,2,3,4; t:=(a,b,c);
```

$s := 1, 2, 3, 4$

$t := a, b, c$

b. cu ajutorul funcţiei `seq`:

```
> seq(3*x, x=2..7);
```

$6, 9, 12, 15, 18, 21$

c. cu ajutorul unui ciclu `for` (vezi secţiunea 1.7.1)

Cu ajutorul funcţiilor *min* şi *max* se poate calcula minimul, respectiv maximul unei secvenţe.

```
> min(s); max(s,2,15);
```

3

15

1.4.2 Liste

O listă este o secvenţă de expresii, scrisă între paranteze pătrate, [ ]. De exemplu, putem avea lista:

```
> ll:=[1,2,5,2,4,2,7,2,a,2,c];
```

$ll := [1, 2, 5, 2, 4, 2, 7, 2, a, 2, c]$

Putem afla numărul de operanzi din listă cu ajutorul funcţiei `nops`:

```
> nops(ll);
```

11

Al n-lea element din listă poate fi afişat cu una din comenzile `op(n,ll)` sau `ll[n]`:

```
> aa:=ll[3]; bb:=op(9,ll);
```

$aa := 5$

$bb := a$

Funcţia `member(elem, ll)` returnează *true* dacă elementul respectiv se află în listă `ll`, şi *false* în caz contrar:

```
> member(d, ll);
```

$false$

Lista vidă este desemnată prin `[]`:

```
> lista:=[];
```

$lista := [\ \ ]$

Se poate adăuga un element nou la lista `ll` astfel: `[op(ll),elem]`. De exemplu:

```
> lista:=[op(lista), d,e,f];
```

$lista := [d, e, f]$

Se poate şterge al n-lea element din listă `ll` astfel: `subsop(n=NULL,ll)`. De exemplu:

```
> lista:=subsop(2=NULL, lista);
```

$lista := [d, f]$

1.4.3 Mulţimi

O mulţime este o secvenţă de expresii, scrisă între acolade, {}, în care fiecare element figurează o singură dată. De exemplu:

```
> ll:={1,2,5,2,4,2,7,2,a,2,c};
```

$ll := \{1, 2, 4, 5, 7, a, c\}$

Adăugarea unui nou element la mulţime, sau ştergerea elementului de pe poziţia n se face la fel ca la liste.

Mulţimea vidă este desemnată prin `{}`.

Reuniunea a două mulţimi se face utilizând operatorul `union`:

```
> s:={1,2,3} : t:={2,3,4} :
> s union t;
```

$\{1, 2, 3, 4\}$

Intersecţia a două mulţimi se realizează cu ajutorul operatorului `intersect`:

```
> s intersect t;
```

$\{2, 3\}$

Diferenţa a două mulţimi se realizează utilizând operatorul `minus`:

```
> s minus t;
```

$\{1\}$

```
> s minus s;
```

$\{\}$

1.4.4 Şiruri de caractere

Şirurile de caractere sunt delimitate de apostrof invers,‘, după cum urmează:

```
> ‘acesta este un sir‘;
```

acesta este un sir

Şirurile de caractere se pot concatena cu ajutorul comenzii `cat`. De exemplu, putem avea:

```
> i:=4;
```

$i := 4$

```
> cat( ‘Valoarea lui i este ‘, i);
```

Valoarea lui i este 4

Atenţie! La concatenarea unui şir de cifre, se obtine un şir de caractere, nu un număr:

```
> a:=cat(5,7,9); b:=52;
```

$a :=$*579*

$b := 52$

```
> whattype(a); # afla tipul expresiei a
```

symbol

```
> whattype(b); # afla tipul expresiei b
```

integer

```
> a+b;
```

$a :=$*579*$+52$

```
> whattype(a+b); # afla tipul expresiei a+b
```

symbol

1.5 Calcule cu matrice şi vectori. Pachetul `linalg`

Cu ajutorul cuvântului-cheie `array` se pot defini vectori şi matrice.

Un vector se defineşte în următorul mod:

```
> v:=array(1..dim_vect);
```

Elementele unui vector se pot defini unul câte unul, sau printr-un ciclu `for` (vezi secţiunea 1.7.1):

```
> v:=array(1..4);
```

$$v := \text{array}(1..4, [\,])$$

```
> v[1]:=a; v[2]:=b; v[3]:={a,b}; v[4]:=3;
```

$$v_1 := a$$
$$v_2 := b$$
$$v_3 := \{a, b\}$$
$$v_4 := 3$$

```
> evalm(v); # evalueaza valoarea lui v
```

$$[a, b, \{a, b\}, 3]$$

O matrice se defineşte astfel:

```
> M:=array(1..nr_rand, 1..nr_col);
```

Elementele unei matrice se pot defini unul câte unul, sau printr-un ciclu `for` (vezi secţiunea 1.7.1):

```
> M:=array(1..2,1..2);
```

$$M := \text{array}(1..2, 1..2, [\,])$$

```
> M[1,1]:=1: M[1,2]:=a: M[2,1]:=3: M[2,2]:={}:
> evalm(M); # evalueaza valoarea lui M
```

$$\begin{bmatrix} 1 & a \\ 3 & \{\,\} \end{bmatrix}$$

Un alt mod de a defini matrice şi vectori, precum şi de a efectua operaţii specifice cu aceste obiecte, este folosirea pachetului `linalg`. Pachetul `linalg` se încarcă astfel:

```
>with(linalg);
 Warning, the protected names norm and trace have been
    redefined and unprotected

  [BlockDiagonal, GramSchmidt, JordanBlock, LUdecomp, QRdecomp,
        Wronskian, addcol, addrow, adj, adjoint, angle, augment,
        backsub, band, basis, bezout, blockmatrix, charmat,
        charpoly,cholesky, col, coldim, colspace, colspan,
        companion, concat, cond, copyinto, crossprod, curl,
        definite, delcols, delrows, det, diag, diverge, dotprod,
        eigenvals, eigenvalues, eigenvectors, eigenvects,
        entermatrix, equal, exponential, extend, ffgausselim,
        fibonacci, forwardsub, frobenius, gausselim, gaussjord,
        geneqns, genmatrix, grad, hadamard, hermite, hessian,
        hilbert, htranspose, ihermite, indexfunc, innerprod,
        intbasis, inverse, ismith, issimilar, iszero, jacobian,
```

```
jordan, kernel, laplacian, leastsqrs, linsolve, matadd,
matrix, minor, minpoly, mulcol, mulrow, multiply, norm,
normalize, nullspace, orthog, permanent, pivot,
potential, randmatrix, randvector, rank, ratform, row,
rowdim, rowspace, rowspan, rref, scalarmul, singularvals,
smith, stack, submatrix, subvector, sumbasis, swapcol,
swaprow, sylvester, toeplitz, trace, transpose,
vandermonde, vecpotent, vectdim, vector, wronskian]
```

O matrice se defineşte cu comanda `matrix`:

`matrix(nr_randuri, nr_coloane, lista_elem sau fc_generatoare)`

Astfel, putem avea:

```
> a:=matrix(3,2,[1,2,3,4,5,6]);
```

$$a := \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

dar şi

```
> f:=(i,j)->i+j; # functia generatoare a elem matricei
```

$$f := (i,j) \to i+j$$

```
> b:=matrix(2,3,f); # adica b[i,j]=f(i,j)
```

$$b := \begin{bmatrix} 2 & 3 & 4 \\ 3 & 4 & 5 \end{bmatrix}$$

Elementul a_{ij} se scrie `a[i,j]`. Astfel, pentru matricea a din exemplul anterior, putem avea:

```
> a[3,1];
```

$$5$$

Un vector se defineşte cu ajutorul comenzii `vector`:

`vector(nr_elem, lista_elem sau fc_generatoare)`

Astfel, putem avea:

```
v:=vector([2,4,8,2]);
```

$$v := [2,4,8,2]$$

sau

```
f:=x-> 2*x+1;
```

$$f := x \to 2x+1$$

```
w:=vector(5,f); # adica w[i]=f(i)
```

$$w := [3,5,7,9,11]$$

Elementul i al vectorului v, v_i, se scrie `v[i]`. Astfel, pentru vectorul v din exemplul anterior, putem avea:

```
> v[3];
```

$$8$$

Redăm mai jos cele mai utilizate funcţii din pachetul `linalg`, împreună cu descrierea lor. Pentru mai multe detalii referitoare la aceste funcţii, precum şi la celelalte funcţii din pachetul `linalg`, se poate consulta pagina de help referitoare la pachetul `linalg`.

Funcţie	Descriere
`addcol(A,c1,c2,m)`	înlocuieşte coloana `c2` a matricei `A` cu `m*c1+c2`
`addrow(A,r1,r2,m)`	înlocuieşte linia `r2` a matricei `A` cu `m*r1+r2`
`adj(A)`, `adjoint(A)`	calculează matricea adjunctă a matricei `A`
`angle(u,v)`	calculează unghiul vectorilor `u` şi `v`
`augment(A,B)`	concatenează (alătură) matricile `A` şi `B` pe orizontală
`backsub(U,b)`	rezolvă sistemul `Ux=b`, prin substituţie inversa, unde `U` este o matrice superior triunghiulară
`band(b,n)`	construieşte o matrice `n x n` care are pe diagonala principală elementele vectorului `b`, iar celelalte elemente sunt nule
`cholesky(A)`	efectuează descompunerea Cholesky a matricei `A`
`col(A,i)`, `col(A,i..k)`	extrage coloana `i`, respectiv coloanele `i` până la `k`, din matricea `A`
`coldim(A)`	returnează numărul de coloane ale matricei `A`
`crossprod(u,v)`	returnează produsul vectorial al vectorilor `u` şi `v`
`delcols(A,r..s)`	şterge coloanele de la `r` la `s` din matricea `A`
`delrows(A,r..s)`	şterge liniile de la `r` la `s` din matricea `A`
`det(A)`	calculează determinantul matricei `A`
`diverge(f)`	calculează divergenţa vectorului `f`
`dotprod(u,v)`	calculează produsul scalar al vectorilor `u` şi `v`
`exponential(A)`	calculează e^A
`extend(A,m,n,x)`	adaugă `m` linii şi `n` coloane matricei `A`, iniţializate cu `x`
`forwardsub(L,b)`	rezolvă sistemul `Lx=b` prin substituţie înainte, unde `L` este o matrice inferior triunghiulară
`gausselim(A)`	efectuează eliminarea gaussiană cu semipivot asupra matricei `A`

continuare pe pagina următoare

Pachetul `linalg` *- continuare*

Funcţie	Descriere
`geneqns(A,x)`	generează un sistem de ecuaţii pornind de la matricea `A` şi vectorul necunoscutelor `x`
`genmatrix(sist, var)`	generează matricea coeficienţilor sistemului `sist`, in raport cu multimea variabilelor `var`
`grad(expr, vect)`	calculează gradientul expresiei `expr`, in funcţie de variabilele `vect`
`inverse(A)`	calculează inversa matricei `A`
`matadd(A,B,c1,c2)`	calculează `c1*A+c2*B`
`minor(r,c)`	calculează minorul de ordin `(r,c)` (elimină linia `r` şi coloana `c`) din matricea `A`
`mulcol(A,c,expr)`	multiplică coloana `c` a matricei `A` cu expresia `expr`
`mulrow(A,r,expr)`	multiplică linia `r` a matricei `A` cu expresia `expr`
`multiply(A,B)`	efectuează înmulţirea matricelor `A` şi `B`
`norm(A)`	calculează norma matricei `A`
`normalize(v)`	calculează versorul vectorului `v`
`rank(A)`	calculează rangul matricei `A`
`row(A,i)`, `row(A,i..j)`	extrage linia `i`, respectiv liniile de la `i` la `j`, ale matricei `A`
`rowdim(A)`	returnează numărul de linii din matricea `A`
`scalarmult(A,s)`	înmulţeşte toate elementele matricei `A` cu scalarul `s`
`stack(A,B)`	concatenează matricele `A` şi `B` pe verticală
`submatrix(A,r1..r2,c1..c2)`	extrage o submatrice a matricei `A`, între liniile `r1`, `r2`, şi coloanele `c1`, `c2`
`subvector(A,r1..r2)`	extrage un subvector al vectorului `A`, de la rangul `r1` la rangul `r2`
`swapcol(A,c1,c2)`	interschimbă coloanele `c1` şi `c2` ale matricei `A`
`swaprow(A,r1,r2)`	interschimbă liniile `r1` şi `r2` ale matricei `A`
`trace(A)`	calculează urma matricei `A`
`vectdim(v)`	returnează dimensiunea vectorului `v`

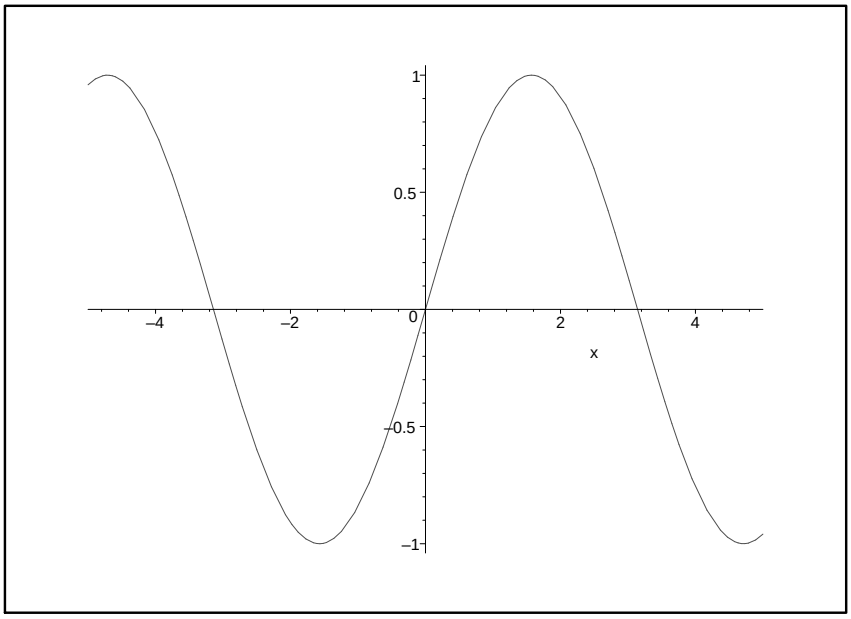
1.6 Grafice

Graficul unei funcţii se realizează folosind comanda `plot`, a cărei sintaxă este `plot(functie, x=x_min..x_max, y_min..y_max)`

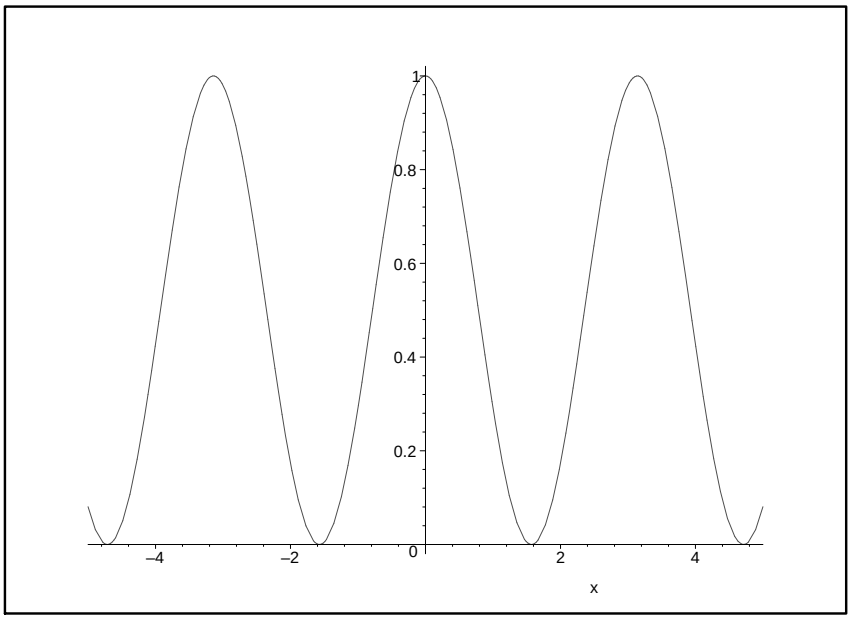
unde argumentul `y_min..y_max` este opţional.

De exemplu, putem avea:

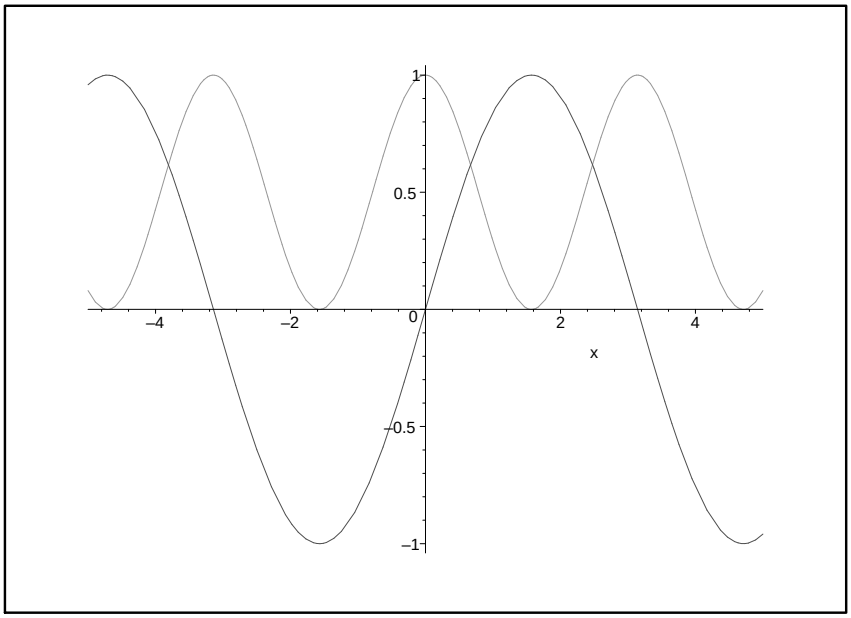
```
> plot(sin(x), x=-5..5);
```




```
> plot(cos(x)^2, x=-5..5);
```




```
> plot([sin(x),cos(x)^2], x=-5..5);
```



Mai multe detalii despre grafice se pot găsi accesând pagina de help referitoare la instrucţiunea `plot`, sau la pachetul `plots`.

1.7 Elemente de programare

1.7.1 Condiţionarea şi ciclarea

A. Condiţionarea
Sintaxa unei instrucţiuni condiţionale este

```
if CONDITIE then EXPRESIE
    [ elif CONDITIE then EXPRESIE ]
    [ else EXPRESIE ]
fi
```

Instrucţiunile puse între paranteze pătrate, [ ], sunt opţionale.

De exemplu, putem avea secvenţa:
`> if a<0 then -a else a fi; # pentru calculul modulului`
pentru a returna modulul numărului a.

Un alt exemplu este dat de secvenţa:
`> if x<0 then -1 elif x=0 then 0 else 1 fi; # functia signum`
pentru a returna semnul unui număr (funcţia sgn).

B. Ciclarea
O instrucţiune repetitivă poate avea una din următoarele două sintaxe:

```
[ for CONTOR ] [ from EXPR ] [ by EXPR ] [ to EXPR ]
[ while EXPR ]
          do INSTRUCTIUNI od;
```

sau

```
[ for CONTOR ] [ in EXPR ] [ while EXPR ]
          do INSTRUCTIUNI od;
```

unde:

- `from` indică punctul de plecare în iteraţie (dacă este omis, valoarea sa implicită este 1);
- `by` indică pasul contorului (dacă este omis, se consideră implicit că are valoarea 1);
- `to` indică punctul de oprire a iteraţiei (dacă este omis, se consideră implicit că are valoarea $+\infty$ şi se obţine o buclă infinită);
- `while` indică o expresie booleană, care trebuie să poată fi evaluată ca adevărată sau falsă;
- `in` indică elementele succesive ale expresiei `EXPR`.

De exemplu, pentru a scrie toate numerele pare de la 6 la 100 putem folosi:

```
> for i from 6 by 2 to 100 do  print(i)  od;
```

Cu ajutorul buclei `for` se pot defini secvenţe, liste, mulţimi, vectori sau matrice.

```
> s:=NULL;
  for i from 1 to 3 do s:=s,2*i+1 od; # definirea unei secvente
```

$s :=$

$s := 3$

$s := 3, 5$

$s := 3, 5, 7$

```
> l:=[];
  for i from 1 to 4 do l:=[op(l),i^2] od; # definirea unei liste
```

$l := [\,]$

$l := [1]$

$l := [1, 4]$

$l := [1, 4, 9]$

$l := [1, 4, 9, 16]$

```
> v:=vector(3); # definirea vectorului
  for i from 1 to 3 do v[i]:=i^3-i^2+1 od; # definirea elem vect
  evalm(v); # vizualizarea vectorului
```

$v := \mathrm{array}(1..3, [\,])$

$v_1 := 1$

$v_2 := 5$

$v_3 := 19$

$[1, 5, 19]$

```
M:=array(1..3,1..4); # definirea matricei
```

$M := \mathrm{array}(1..3, 1..4, [\,])$

```
> for i from 1 to 3 do # definirea elem matricei
    for j from 1 to 4 do
      M[i,j]:=i^j
    od;
  od;
> evalm(M);
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 4 & 8 & 16 \\ 3 & 9 & 27 & 81 \end{bmatrix}$$

Putem afişa elemetele unei liste (secvenţe, mulţimi, matrice, vector) astfel:

```
> lista:=[3,2,4,5,1]:
> for i in lista do print(i) od;
```

Mai multe detalii despre instrucţiunile de condiţionare şi de ciclare se pot gasi accesând pagina de help referitoare la acestea.

1.7.2 Funcţii şi proceduri

O funcţie poate fi definită cu ajutorul operatorului `->`. Putem defini funcţii de o variabilă sau funcţii de mai multe variabile.

```
> f:=x->x^2+1;
```

$$f := x \to x^2 + 1$$

```
> g:=(x,y)->x^2+y;
```

$$g := (x, y) \to x^2 + y$$

```
> f(3);
```

10

```
> g(3,4);
```

13

```
> g(4,3);
```

19

O procedură este un grup de instrucţiuni, variabile şi constante. Sintaxa este:

```
proc (ARGUMENTE)
    local VARIABILE_LOCALE;
    global VARIABILE_GLOBALE;
    options OPTIUNI;
    description SIR_DE_CARACTERE;
    INSTRUCTIUNI;
end;
```

O procedură returnează ultimul rezultat obţinut. Pentru a forţa returnarea unui alt rezultat, se foloseşte `RETURN`. De asemenea, pentru a returna un mesaj de eroare, se foloseşte `ERROR`.
De exemplu, putem defini procedura:

```
> modul:=proc(a) if a<0 then -a else a fi; end;
```

$$modul := \mathbf{proc}(a)\ \mathbf{if}\ a < 0\ \mathbf{then}\ -a\ \mathbf{else}\ a\ \mathbf{end\ if\ end\ proc}$$

pe care o putem apela astfel:

```
> modul(-3);
```

3

Un alt exemplu de procedură este următorul:

```
> ec2:=proc(a,b,c)
    local delta,x1,x2;
    description ‘Rezolvarea ecuatiei de gradul 2‘;
    delta:=b^2-4*a*c;
    if delta>0 then
      x1:=(-b+sqrt(delta))/(2*a);
      x2:=(-b-sqrt(delta))/(2*a);
      RETURN(x1,x2);
    elif delta=0 then RETURN(-b/(2*a));
    else
      RETURN(‘ecuatia nu ere solutii reale‘);
    fi;
  end:
```

care produce următoarele rezultate:

```
> ec2(1,6,9); # ecuatia x^2+6*x+9=0
```

-3

```
> ec2(1,2,9); # ecuatia x^2+2*x+9=0
```

ecuatia nu are solutii reale

```
> ec2(1,2,-3); # ecuatia x^2+2*x-3=0
```

$1, -3$

Pentru a defini tipul unui argument, se foloseşte sintaxa `argument::tip`. De exemplu, să luăm următoarea procedură şi situaţiile care pot apărea:

```
> # procedura care returneaza determinantul unei matrice
> determinant:=proc(A) RETURN(det(A)) end:
> determinant(2);
Error, (in linalg:-det) expecting a matrix
```

Procedura `determinant` se poate ”imbunătăţi” astfel:

```
> determinant1:=proc(A)
    if not type(A, matrix)
      then ERROR(‘argumentul trebuie sa fie matrice!!!‘)
    fi;
    RETURN(det(A))
    end:
```

care produce următorul rezultat:

```
> determinant1(2);
Error, (in determinant1) argumentul trebuie sa fie matrice!!!
```

Se mai poate defini argumentul `A` ca fiind de tipul matrice:

```
> determinant3:=proc(A::matrix) RETURN(det(A)) end:
```

şi se obţine următorul rezultat:

```
> determinant3(2);
Error, invalid input: determinant3 expects its 1st argument, A,
 to be of type matrix, but received 2
```

Mai multe detalii despre tipurile existente se pot găsi accesând pagina de help (cuvântul cheie este `type`).

Un alt exemplu este procedura `rdc`, procedură pentru calculul lui $\frac{1}{\sqrt{x}}$:

```
> rdc:=proc(x)
    if x<0 then ERROR(‘numar negativ!‘)
    elif x=0 then RETURN(infinity)
    else simplify(x^(-1/2));
    fi;
  end;
```

$$rdc := \textbf{proc}(x)\ \textbf{if}\ x < 0\ \textbf{then}\ \text{ERROR}(\text{‘}numar\ negativ!\text{‘})$$
$$\textbf{elif}\ x = 0\ \textbf{then}\ \text{RETURN}(\infty)$$
$$\textbf{else}\ \text{simplify}(1/(\text{x}\hat{}\ (1/2)))\ \textbf{end if end proc}$$

```
> rdc(-1);
Error, (in rdc) numar negativ!
> rdc(0);
```
$$\infty$$
```
> rdc(4);
```
$$\frac{1}{2}$$

Pentru a putea urmări execuţia unei proceduri, se foloseşte `debug`, iar pentru a stopa urmărirea, se foloseşte `undebug`. De exemplu, putem avea:

```
> f:=proc(a,b)
      local y,z;
      y:=a+b/2;
      z:=1/y;
      RETURN(y+z)
    end;
```

$$f := \textbf{proc}(a, b)$$
$$\textbf{local}\ y, z;$$
$$y := a + 1/2 * b;\ z := 1/y$$
$$\text{RETURN}(y + z)\ \textbf{end proc}$$

```
> debug(f);
```
$$f$$
```
> f(2,4);
```

```
{--> enter f, args = 2, 4
```
$$y := 4$$
$$z := \frac{1}{4}$$
```
<-- exit f (now at top level) = 17/4}
```
$$\frac{17}{4}$$
```
> f(0,1);
{--> enter f, args = 0, 1
```
$$y := \frac{1}{2}$$
$$z := 2$$
```
<-- exit f (now at top level) = 5/2}
```
$$\frac{5}{2}$$
```
undebug(f)
```
$$f$$
```
> f(10,20);
```
$$\frac{401}{20}$$

Alte detalii despre funcţii şi proceduri, precum şi despre opţiunile `debug` şi `undebug`, puteţi găsi pe paginile de help referitoare la acestea.

Capitolul 2

Rezolvarea sistemelor liniare

În acest capitol vom prezenta metode de rezolvare a sistemelor liniare de tip Cramer (numărul de ecuaţii este egal cu numărul de necunoscute, şi determinantul matricei sistemului este nenul):

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2 \\ \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\ a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n \end{cases} \tag{2.1}$$

în care a_{ij} şi b_i sunt numere reale date, $i = 1 \ldots n, j = 1 \ldots n$, iar $x_1, x_2, \ldots, x_n$ sunt numere reale necunoscute.

Sistemul (2.1) se poate scrie matriceal sub forma:

$$Ax = b$$

unde: $A = (a_{ij})_{i,j=\overline{1,n}}$, $b = (b_1, b_2, \ldots, b_n)^T$, $x = (x_1, x_2, \ldots, x_n)^T$.

Dacă matricea A este nesingulară, sistemul $Ax = b$ are soluţie unică:

$$x = A^{-1}b.$$

Deoarece în cele mai multe cazuri matricea A are număr mare de linii şi coloane, iar calculul matricei A^{-1} este dificil şi acumulează erori, se impun metode directe şi metode iterative pentru rezolvarea acestor sisteme.

2.1 Metoda lui Gauss

2.1.1 Breviar teoretic

Metoda lui Gauss presupune transformarea sistemului $Ax = b$ într-un sistem superior triunghiular, şi apoi rezolvarea acestuia prin substituţie inversă.

Construcţia sistemului superior triunghiular se face astfel: la pasul k se elimină x_k din ecuaţiile $k + 1$, ..., n, prin înmulţirea ecuaţiei k cu

$$m_{ik} = -\frac{a_{ik}}{a_{kk}}$$

(elementul a_{kk} se numeşte pivot) şi adunarea acestora la ecuaţia i $(i > k)$.

În funcţie de alegerea pivotului, există următoarele **variante ale metodei lui Gauss**:

1. **metoda lui Gauss clasică** - în care la fiecare pas, pivotul este elementul a_{kk}, $k = \overline{1,n}$;

2. **metoda lui Gauss cu semipivot** - în care la fiecare pas, se alege ca pivot elementul a_{ik} maxim în valoare absolută pe coloană, pentru $i > k$, permutându-se linia k cu linia i;

3. **metoda lui Gauss cu pivot total** - în care la fiecare pas, se alege ca pivot elementul maxim atât pe linie, cât şi pe coloană, pentru $i > k$, $j > k$, permutându-se linia k cu linia i şi coloana k cu coloana j;

În acest fel, sistemul (2.1) se reduce la forma superior triunghiulară

$$\begin{pmatrix} \tilde{a}_{11} & \tilde{a}_{12} & ... & \tilde{a}_{1,n-2} & \tilde{a}_{1,n-1} & \tilde{a}_{1,n} \\ 0 & \tilde{a}_{22} & ... & \tilde{a}_{2,n-2} & \tilde{a}_{2,n-1} & \tilde{a}_{2,n} \\ ... & ... & ... & ... & ... & ... \\ 0 & 0 & ... & 0 & \tilde{a}_{n-1,n-1} & \tilde{a}_{n-1,n} \\ 0 & 0 & ... & 0 & 0 & \tilde{a}_{nn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ ... \\ x_{n-1} \\ x_n \end{pmatrix} = \begin{pmatrix} \tilde{b}_1 \\ \tilde{b}_2 \\ ... \\ \tilde{b}_{n-1} \\ \tilde{b}_n \end{pmatrix} \tag{2.2}$$

iar rezolvarea sistemului (2.2) se face prin substituţie inversă:

$$\begin{aligned} x_n &= \frac{\tilde{b}_n}{\tilde{a}_{nn}} \\ x_k &= \left(\tilde{b}_k - \sum_{j=k+1}^{n} \tilde{a}_{kj} \cdot x_j \right) \cdot \frac{1}{\tilde{a}_{kk}},\ k = n-1, n-2, \ldots, 1 \end{aligned} \tag{2.3}$$

Observaţia 2.1.1. Cu ajutorul eliminării gaussiene se poate determina şi inversa unei matrice. Redăm în continuare algoritmul de aflare a inversei unei matrice A.

1. generarea matricei B prin concatenarea matricelor A (de dimensiune n) cu matricea I_n

2. pentru $i = \overline{1,n}$

   $m = B_{ii}$
   pentru $j = \overline{1,2n}$
      $B_{ij} = \dfrac{B_{ij}}{m}$
   pentru $j = \overline{1,n}$, $j \neq i$
      $m_1 = B_{ji}$
      pentru $k = \overline{1,2n}$
         $B_{jk} = B_{jk} - m_1 \, B_{ik}$

3. prin ştergerea primelor n coloane ale matricei B astfel transformate, se obţine inversa matricei A

2.1.2 Probleme rezolvate

Exerciţiul 2.1.1. Să se rezolve următorul sistem folosind cele trei variante ale eliminării Gauss:

$$\begin{cases} x+y+z=6 \\ 2x-y+3z=9 \\ x+4y+z=12. \end{cases}$$

Matricea sistemului este

$$A=\begin{pmatrix} 1 & 1 & 1 \\ 2 & -1 & 3 \\ 1 & 4 & 1 \end{pmatrix},$$

iar $\bar{A}$ este matricea sa extinsă:

$$\bar{A}=(A,b)=\begin{pmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{pmatrix}.$$

Deoarece numărul ecuaţiilor este egal cu cel al necunoscutelor şi

$$\det A=-3\neq 0,$$

sistemul este compatibil determinat (de tip Cramer), şi deci metoda eliminării a lui Gauss este aplicabilă.

În continuare, pentru a efectua operaţiile asupra matricei extinse a sistemului vom nota linia i cu L_i, iar coloana j cu C_j.

Rezolvare utilizând metoda lui Gauss clasică

A. Construcţia sistemului superior triunghiular

Pasul 1

- pivot: $a_{11}=1$
- $m_{21}=-\dfrac{2}{1}=-2$
- $m_{31}=-\dfrac{1}{1}=-1$

$$\begin{pmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{pmatrix} \xrightarrow[L_3\to L_3+m_{31}L_1]{L_2\to L_2+m_{21}L_1} \begin{pmatrix} 1 & 1 & 1 & 6 \\ 0 & -3 & 1 & -3 \\ 0 & 3 & 0 & 6 \end{pmatrix}$$

Pasul 2

- pivot: $a_{22}=-3$
- $m_{32}=-\dfrac{3}{-3}=1$

$$\begin{pmatrix} 1 & 1 & 1 & 6 \\ 0 & -3 & 1 & -3 \\ 0 & 3 & 0 & 6 \end{pmatrix} \xrightarrow{L_3\to L_3+m_{32}L_2} \begin{pmatrix} 1 & 1 & 1 & 6 \\ 0 & -3 & 1 & -3 \\ 0 & 0 & 1 & 3 \end{pmatrix}$$

În acest moment am ajuns la un sistem de forma $\tilde{A}\mathrm{x} = \tilde{b}$, echivalent cu sistemul iniţial, în care matricea $\tilde{A}$ este superior triunghiulară, unde:

$$\tilde{A} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & -3 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \qquad \tilde{b} = \begin{pmatrix} 6 \\ -3 \\ 3 \end{pmatrix}.$$

B. Rezolvarea sistemului superior triunghiular
Prin metoda substituţiei inverse, avem:

$$\begin{aligned} z &= \frac{3}{1} \\ y &= \frac{1}{-3}(-3 - 1 \cdot z) \\ x &= \frac{1}{1}(6 - 1 \cdot y - 1 \cdot z), \end{aligned}$$

de unde obţinem soluţia sistemului: $x = 1$, $y = 2$, $z = 3$.

Rezolvare cu metoda lui Gauss cu semipivot

A. Construcţia sistemului superior triunghiular

Pasul 1

• Ca pivot se ia elementul a_{i1} de modul maxim de pe coloana 1. În cazul nostru, pivotul este a_{12}, deci se permută linia 1 cu linia 2, şi se fac zerouri pe coloana 1 pentru $i > 1$:

$$\begin{pmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{pmatrix} \xrightarrow{L_2 \leftrightarrow L_1} \begin{pmatrix} 2 & -1 & 3 & 9 \\ 1 & 1 & 1 & 6 \\ 1 & 4 & 1 & 12 \end{pmatrix}$$

$$\begin{pmatrix} 2 & -1 & 3 & 9 \\ 1 & 1 & 1 & 6 \\ 1 & 4 & 1 & 12 \end{pmatrix} \xrightarrow[L_3 \to L_3 - \frac{1}{2}L_1]{L_2 \to L_2 - \frac{1}{2}L_1} \begin{pmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{3}{2} & -\frac{1}{2} & \frac{3}{2} \\ 0 & \frac{9}{2} & -\frac{1}{2} & \frac{15}{2} \end{pmatrix}$$

Pasul 2

• Ca pivot se ia elementul a_{i2} de modul maxim de pe coloana 2, pentru $i \geq 2$. În cazul nostru, pivotul este a_{32}, deci se permută linia 2 cu linia 3 şi se fac zerouri pe coloana 2, pentru $i > 2$:

$$\begin{pmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{3}{2} & -\frac{1}{2} & \frac{3}{2} \\ 0 & \frac{9}{2} & -\frac{1}{2} & \frac{15}{2} \end{pmatrix} \xrightarrow{L_3 \leftrightarrow L_2} \begin{pmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{9}{2} & -\frac{1}{2} & \frac{15}{2} \\ 0 & \frac{3}{2} & -\frac{1}{2} & \frac{3}{2} \end{pmatrix}$$

$$\begin{pmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{9}{2} & -\frac{1}{2} & \frac{15}{2} \\ 0 & \frac{3}{2} & -\frac{1}{2} & \frac{3}{2} \end{pmatrix} \xrightarrow{L_3 \to L_3 - \frac{1}{3}L_2} \begin{pmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{9}{2} & -\frac{1}{2} & \frac{15}{2} \\ 0 & 0 & -\frac{1}{3} & -1 \end{pmatrix}$$

În acest moment am ajuns la un sistem de forma $\tilde{A}\mathrm{x} = \tilde{b}$, echivalent cu sistemul iniţial, unde matricea $\tilde{A}$ este superior triunghiulară, iar:

$$\tilde{A} = \begin{pmatrix} 2 & -1 & 3 \\ 0 & \frac{9}{2} & -\frac{1}{2} \\ 0 & 0 & -\frac{1}{3} \end{pmatrix}, \qquad \mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \qquad \tilde{b} = \begin{pmatrix} 9 \\ \frac{15}{2} \\ -1 \end{pmatrix}.$$

B. Rezolvarea sistemului superior triunghiular se face ca şi în cazul metodei lui Gauss clasice, şi conduce la soluţia $x = 1$, $y = 2$, $z = 3$.

Rezolvare cu metoda lui Gauss cu pivot total

A. Construcţia sistemului superior triunghiular

Pasul 1

• ca pivot se alege elementul a_{ij} de modul maxim pentru $i, j \geq 1$. În cazul nostru pivotul este a_{32}, deci se permută linia 3 cu linia 1, şi coloana 2 cu coloana 1:

$$\begin{pmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{pmatrix} \xrightarrow{L_3 \leftrightarrow L_1} \begin{pmatrix} 1 & 4 & 1 & 12 \\ 2 & -1 & 3 & 9 \\ 1 & 1 & 1 & 6 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 4 & 1 & 12 \\ 2 & -1 & 3 & 9 \\ 1 & 1 & 1 & 6 \end{pmatrix} \xrightarrow{C_2 \leftrightarrow C_1} \begin{pmatrix} 4 & 1 & 1 & 12 \\ -1 & 2 & 3 & 9 \\ 1 & 1 & 1 & 6 \end{pmatrix}$$

• pentru corectitudinea rezultatului final este necesar ca, ori de câte ori se permută coloanele matricei extinse, să se permute şi elementele corespunzătoare ale vectorului x. Astfel, avem:

$$\mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \xrightarrow{\mathrm{x}_2 \leftrightarrow \mathrm{x}_1} \begin{pmatrix} y \\ x \\ z \end{pmatrix}$$

• în final, obţinem:

$$\begin{pmatrix} 4 & 1 & 1 & 12 \\ -1 & 2 & 3 & 9 \\ 1 & 1 & 1 & 6 \end{pmatrix} \xrightarrow[L_3 \to L_3 - \frac{1}{4} L_1]{L_2 \to L_2 + \frac{1}{4} L_1} \begin{pmatrix} 4 & 1 & 1 & 12 \\ 0 & \frac{9}{4} & \frac{13}{4} & 12 \\ 0 & \frac{3}{4} & \frac{3}{4} & 3 \end{pmatrix}$$

Pasul 2

• ca pivot se alege elementul a_{ij} de modul maxim pentru $i, j \geq 2$. Deoarece pivotul este a_{23}, se permută coloana 3 cu coloana 2:

$$\begin{pmatrix} 4 & 1 & 1 & 12 \\ 0 & \frac{9}{4} & \frac{13}{4} & 12 \\ 0 & \frac{3}{4} & \frac{3}{4} & 3 \end{pmatrix} \xrightarrow{C_3 \leftrightarrow C_2} \begin{pmatrix} 4 & 1 & 1 & 12 \\ 0 & \frac{13}{4} & \frac{9}{4} & 12 \\ 0 & \frac{3}{4} & \frac{3}{4} & 3 \end{pmatrix}$$

$$\mathrm{x} = \begin{pmatrix} y \\ x \\ z \end{pmatrix} \xrightarrow{\mathrm{x}_3 \leftrightarrow \mathrm{x}_2} \begin{pmatrix} y \\ z \\ x \end{pmatrix}$$

$$\begin{pmatrix} 4 & 1 & 1 & 12 \\ 0 & \frac{13}{4} & \frac{9}{4} & 12 \\ 0 & \frac{3}{4} & \frac{3}{4} & 3 \end{pmatrix} \xrightarrow{L_3 \to L_3 - \frac{3}{13} L_2} \begin{pmatrix} 4 & 1 & 1 & 12 \\ 0 & \frac{13}{4} & \frac{9}{4} & 12 \\ 0 & 0 & \frac{3}{13} & \frac{3}{13} \end{pmatrix}$$

În acest moment am ajuns la un sistem de forma $\tilde{A}\mathrm{x} = \tilde{b}$, echivalent cu sistemul iniţial, unde matricea $\tilde{A}$ este superior triunghiulară, iar:

$$\tilde{A} = \begin{pmatrix} 4 & 1 & 1 \\ 0 & \frac{13}{4} & \frac{9}{4} \\ 0 & 0 & \frac{3}{13} \end{pmatrix}, \qquad \mathrm{x} = \begin{pmatrix} y \\ z \\ x \end{pmatrix}, \qquad \tilde{b} = \begin{pmatrix} 12 \\ 12 \\ \frac{3}{13} \end{pmatrix}.$$

B. Rezolvarea sistemului superior triunghiular se face ca şi în cazul metodei lui Gauss clasice, şi conduce la soluţia $x = 1$, $z = 3$, $y = 2$.

Exerciţiul 2.1.2. Să se găsească inversa matricei

$$A = \begin{pmatrix} 2 & 2 & 3 \\ 2 & 1 & 1 \\ 3 & 1 & 2 \end{pmatrix}.$$

Rezolvare

Considerăm matricea B obţinută prin concatenarea matricei A cu matricea unitate I_3:

$$B = \left(\begin{array}{ccc|ccc} 2 & 2 & 3 & 1 & 0 & 0 \\ 2 & 1 & 1 & 0 & 1 & 0 \\ 3 & 1 & 2 & 0 & 0 & 1 \end{array}\right).$$

Folosind metoda eliminării a lui Gauss, transformăm matricea B după cum urmează:

$$\left(\begin{array}{ccc|ccc} 2 & 2 & 3 & 1 & 0 & 0 \\ 2 & 1 & 1 & 0 & 1 & 0 \\ 3 & 1 & 2 & 0 & 0 & 1 \end{array}\right) \xrightarrow{L1 \to \frac{1}{B_{11}} L1} \left(\begin{array}{ccc|ccc} 1 & 1 & \frac{3}{2} & \frac{1}{2} & 0 & 0 \\ 2 & 1 & 1 & 0 & 1 & 0 \\ 3 & 1 & 2 & 0 & 0 & 1 \end{array}\right)$$

$$\left(\begin{array}{ccc|ccc} 1 & 1 & \frac{3}{2} & \frac{1}{2} & 0 & 0 \\ 2 & 1 & 1 & 0 & 1 & 0 \\ 3 & 1 & 2 & 0 & 0 & 1 \end{array}\right) \xrightarrow[L3 \to L3 - B_{31} L1]{L2 \to L2 - B_{21} L1} \left(\begin{array}{ccc|ccc} 1 & 1 & \frac{3}{2} & \frac{1}{2} & 0 & 0 \\ 0 & -1 & -2 & -1 & 1 & 0 \\ 0 & -2 & -\frac{5}{2} & -\frac{3}{2} & 0 & 1 \end{array}\right)$$

$$\left(\begin{array}{ccc|ccc} 1 & 1 & \frac{3}{2} & \frac{1}{2} & 0 & 0 \\ 0 & -1 & -2 & -1 & 1 & 0 \\ 0 & -2 & -\frac{5}{2} & -\frac{3}{2} & 0 & 1 \end{array}\right) \xrightarrow{L2 \to \frac{1}{B_{22}} L2} \left(\begin{array}{ccc|ccc} 1 & 1 & \frac{3}{2} & \frac{1}{2} & 0 & 0 \\ 0 & 1 & 2 & 1 & -1 & 0 \\ 0 & -2 & -\frac{5}{2} & -\frac{3}{2} & 0 & 1 \end{array}\right)$$

$$\left(\begin{array}{ccc|ccc} 1 & 1 & \frac{3}{2} & \frac{1}{2} & 0 & 0 \\ 0 & 1 & 2 & 1 & -1 & 0 \\ 0 & -2 & -\frac{5}{2} & -\frac{3}{2} & 0 & 1 \end{array}\right) \xrightarrow[L3 \to L3 - B_{32} L2]{L1 \to L1 - B_{12} L2} \left(\begin{array}{ccc|ccc} 1 & 0 & -\frac{1}{2} & -\frac{1}{2} & 1 & 0 \\ 0 & 1 & 2 & 1 & -1 & 0 \\ 0 & 0 & \frac{3}{2} & \frac{1}{2} & -2 & 1 \end{array}\right)$$

$$\left(\begin{array}{ccc|ccc} 1 & 0 & -\frac{1}{2} & -\frac{1}{2} & 1 & 0 \\ 0 & 1 & 2 & 1 & -1 & 0 \\ 0 & 0 & \frac{3}{2} & \frac{1}{2} & -2 & 1 \end{array}\right) \xrightarrow{L3\to\frac{1}{B_{33}}L3} \left(\begin{array}{ccc|ccc} 1 & 0 & -\frac{1}{2} & -\frac{1}{2} & 1 & 0 \\ 0 & 1 & 2 & 1 & -1 & 0 \\ 0 & 0 & 1 & \frac{1}{3} & -\frac{4}{3} & \frac{2}{3} \end{array}\right)$$

$$\left(\begin{array}{ccc|ccc} 1 & 0 & -\frac{1}{2} & -\frac{1}{2} & 1 & 0 \\ 0 & 1 & 2 & 1 & -1 & 0 \\ 0 & 0 & 1 & \frac{1}{3} & -\frac{4}{3} & \frac{2}{3} \end{array}\right) \xrightarrow[L2\to L2-B_{23}L3]{L1\to L1-B_{13}L3} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & -\frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 1 & 0 & \frac{1}{3} & \frac{5}{3} & -\frac{4}{3} \\ 0 & 0 & 1 & \frac{1}{3} & -\frac{4}{3} & \frac{2}{3} \end{array}\right)$$

Inversa matricei A va fi matricea C, obţinută prin ştergerea primelor 3 coloane ale matricei B:

$$C = \begin{pmatrix} -\frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{5}{3} & -\frac{4}{3} \\ \frac{1}{3} & -\frac{4}{3} & \frac{2}{3} \end{pmatrix}$$

Într-adevăr, se verifică uşor că

$$A \cdot C = C \cdot A = I_3.$$

2.1.3 Probleme propuse

Exerciţiul 2.1.3. Să se rezolve următoarele sisteme, folosind cele trei variante ale metodei lui Gauss:

a) $\begin{cases} x + 2y + z = 1 \\ 3x - y + 5z = 14 \\ x + y - z = -2 \end{cases}$

b) $\begin{cases} x + y + z + t = 0 \\ 3x - 2y - z + t = -8 \\ x - 2y - z + 4t = 1 \\ x + y - z + 2t = 1 \end{cases}$

Exerciţiul 2.1.4. Să se găsească soluţia sistemelor anterioare, calculând inversa matricei A a sistemului, şi efectuând înmulţirea $A^{-1}b$.

2.1.4 Implementare

A. Algoritm

Algoritmii pentru cele 3 metode sunt asemănători, diferenţa dintre ei apărând (aşa cum se poate vedea şi din exemplul rezolvat) în modul de rezolvare a eliminării Gauss.

Date de intrare: un sistem de ecuaţii (scris ca mulţime de ecuaţii)

Date de ieşire: soluţia sistemului

Algoritmul constă din următoarele etape:

1. generarea matricei extinse a sistemului, $A = (a_{ij})_{i=\overline{1,n}, j=\overline{1,n+1}}$
   - n= numărul de ecuaţii (numărul de linii ale matricei A);
2. a) eliminarea Gauss pentru metoda lui Gauss clasică
   - pentru $k = \overline{1, n-1}$

- dacă $a_{kk} = 0$, atunci se caută r pentru care $a_{kr} \neq 0$, $r = \overline{k+1, n}$ şi se schimbă linia k cu linia r;
- dacă toţi $a_{kr} = 0$, $r = \overline{k+1, n}$ atunci se returnează eroare;
- pentru $i = \overline{k+1, n}$

$$m = -\frac{a_{ik}}{a_{kk}}, \text{ unde } a_{kk} \neq 0;$$

- pentru $j = \overline{k, n}$

$$a_{ij} = a_{ij} + m \cdot a_{kj};$$

b) eliminarea Gauss pentru metoda lui Gauss cu semipivot

- pentru $k = \overline{1, n-1}$ se caută elementul de modul maxim pe linie, i.e. dacă $|a_{kr}| > |a_{kk}|$, $r = \overline{k+1, n}$, se schimbă linia k cu linia r
- pentru $i = \overline{k+1, n}$

$$m = -\frac{a_{ik}}{a_{kk}}, \text{ unde } a_{kk} \neq 0;$$

- pentru $j = \overline{k, n}$

$$a_{ij} = a_{ij} + m \cdot a_{kj};$$

c) eliminarea Gauss pentru metoda lui Gauss cu pivot total

- pentru $k = \overline{1, n-1}$ se caută elementul de modul maxim pe linie şi coloană, i.e. dacă $|a_{pr}| > |a_{kk}|$, $p, r = \overline{k+1, n}$, se schimbă coloana p cu coloana k şi linia r cu linia k
- pentru $i = \overline{k+1, n}$

$$m = -\frac{a_{ik}}{a_{kk}}, \text{ unde } a_{kk} \neq 0;$$

- pentru $j = \overline{k, n}$

$$a_{ij} = a_{ij} + m \cdot a_{kj};$$

3. rezolvarea sistemului superior triunghiular prin substituţie inversă

$$x_n = \frac{a_{n,n+1}}{a_{nn}},$$

- pentru $i = \overline{n-1, 1}$

$$x_i = \frac{1}{a_{ii}} \left(a_{i,n+1} - \sum_{j=i+1}^{n} a_{ij} x_j \right).$$

B. Programe MAPLE şi rezultate

Deoarece diferitele variante ale metodei lui Gauss se deosebesc doar prin modul în care se realizează eliminarea Gauss, în cele ce urmează am implementat separat cele trei variante de eliminare, folosind procedurile `cgauss`, `spgauss`, `tpgauss`. Aceste proceduri vor fi folosite apoi ca opţiuni în procedura finală `gauss`.

```
restart: with(linalg):

cgauss:=proc(A::matrix)
local A1, A2, n, k, r, i, m, j;
n:=rowdim(A);
A1:=A;
A2:=delcols(A1,n+1..n+1);
if(det(A2)=0) then ERROR(‘sistemul nu are solutie unica!‘) fi;
for k from 1 to n-1 do
  if A1[k,k]=0 then
    for r from k+1 to n
    while A1[k,r]=0 do r=r+1 od;
    if r>n then ERROR(‘sistemul nu are solutie unica!‘)
      else A1:=swaprow(A1,k,r);
    fi;
  fi;
    for i from k+1 to n do
    m:=A1[i,k]/A1[k,k];
      for j from k to n+1 do
      A1[i,j]:=A1[i,j]-m*A1[k,j];
    od;
  od;
od;
RETURN(evalm(A1));
end:

spgauss:=proc(A::matrix)
local A1, A2, n, k, r, i, m, j, mx;
n:=rowdim(A);
A1:=A;
A2:=delcols(A1,n+1..n+1);
if(det(A2)=0) then ERROR(‘sistemul nu are solutie unica!‘) fi;
for k from 1 to n-1 do
  mx:=k;
  for r from k to n do
    if (abs(A1[r,k])>abs(A1[k,k])) then mx:=r
    fi;
  od;
  if mx<>k then A1:=swaprow(A1,k,mx); fi;
    for i from k+1 to n do
    m:=A1[i,k]/A1[k,k];
      for j from k to n+1 do
      A1[i,j]:=A1[i,j]-m*A1[k,j];
    od;
  od;
od;
```

```
RETURN(evalm(A1));
end:

gauss:=proc(eqn::set(equation), opt::symbol)
local A,A1,l,n,r,k,i,m,j,s,x,rez;
l:=[op(indets(eqn))];
n:=nops(l);
A:=genmatrix(eqn, l, flag);
if opt=clasic then A1:=cgauss(A);
elif opt=semipivot then A1:=spgauss(A);
elif opt=totalpivot then
  rez:=tpgauss(A);
  A1:=rez[1];
  l:=[seq(l[rez[2][i]],i=1..n)];
else ERROR(‘optiunile sunt: clasic, semipivot sau totalpivot‘);
fi;
x[n]:=A1[n,n+1]/A1[n,n];
for i from n-1 by -1 to 1 do
  s:=0;
  for j from i+1 to n do
    s:=s+A1[i,j]*x[j];
  od;
  x[i]:=1/A1[i,i]*(A1[i,n+1]-s);
od;
RETURN(seq(l[i]=x[i],i=1..n));
end:
```

Observaţia 2.1.2. Instrucţiunea `indets(set_eq)` returnează mulţimea nedeterminatelor sistemului `set_eq`. Deoarece ordinea elementelor acestei mulţimi nu este neapărat aceeaşi cu ordinea nedeterminatelor din prima ecuaţie a sistemului, pot apărea diferenţe între rezultatele furnizate cu ajutorul codului `MAPLE` şi rezultatele calculate pe hârtie. Deşi matricea sistemului generată cu ajutorul instrucţiunii `indets` nu este întotdeauna aceeaşi cu matricea sistemului scrisă pe hârtie, rezultatele furnizate de program vor fi aceleaşi (eventual ordinea soluţiilor va fi schimbată).

Observaţia 2.1.3. Pentru a urmări execuţia unei proceduri, se foloseşte instrucţiunea `debug`. În cazul programelor din exemplele de mai sus, se poate folosi următorul set de instrucţiuni:

```
debug(cgauss):
debug(spgauss):
debug(gauss):
```

Redăm mai jos, cu titlu de exemplu, rezultatul urmăririi procedurilor `gauss`, `cgauss`, `spgauss` şi `tpgauss` pentru acelaşi sistem de ecuaţii.

```
> gauss({x+y+z=6,2*x-y+3*z=9,x+4*y+z=12},clasic);

{--> enter gauss, args = {x+y+z = 6, 2*x-y+3*z = 9, x+4*y+z = 12},
clasic
```

$$l := [x, y, z]$$

$$n := 3$$

$$A := \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{bmatrix}$$

```
{--> enter cgauss, args = A
```

$$n := 3$$

$$A1 := A$$

$$A2 := \begin{bmatrix} 1 & 1 & 1 \\ 2 & -1 & 3 \\ 1 & 4 & 1 \end{bmatrix}$$

$$m := 2$$

$$A1_{2,1} := 0$$

$$A1_{2,2} := -3$$

$$A1_{2,3} := 1$$

$$A1_{2,4} := -3$$

$$m := 1$$

$$A1_{3,1} := 0$$

$$A1_{3,2} := 3$$

$$A1_{3,3} := 0$$

$$A1_{3,4} := 6$$

$$m := -1$$

$$A1_{3,2} := 0$$

$$A1_{3,3} := 1$$

$$A1_{3,4} := 3$$

```
<-- exit cgauss (now in gauss) = array(1 .. 3, 1 .. 4,[(3, 3)=1,(2,
1)=0,(3, 2)=0,(1, 3)=1,(3, 1)=0,(1, 4)=6,(2, 2)=-3,(2, 3)=1,(3,
4)=3,(1, 2)=1,(1, 1)=1,(2, 4)=-3])}
```

$$A1 := \begin{bmatrix} 1 & 1 & 1 & 6 \\ 0 & -3 & 1 & -3 \\ 0 & 0 & 1 & 3 \end{bmatrix}$$

$$x_3 := 3$$

$$s := 0$$

$$s := 3$$

$$x_2 := 2$$

$$s := 0$$

$$s := 2$$

$$s := 5$$

$$x_1 := 1$$

```
<-- exit gauss (now at top level) = x = 1, y = 2, z = 3}
```

$$x = 1,\ y = 2,\ z = 3$$

```
>  gauss({x+y+z=6,2*x-y+3*z=9,x+4*y+z=12},semipivot);
```

```
{--> enter gauss, args = {x+y+z = 6, 2*x-y+3*z = 9, x+4*y+z = 12},
semipivot
```

$$l := [x,\ y,\ z]$$

$$n := 3$$

$$A := \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{bmatrix}$$

```
{--> enter spgauss, args = A
```

$$n := 3$$

$$A1 := A$$

$$A2 := \begin{bmatrix} 1 & 1 & 1 \\ 2 & -1 & 3 \\ 1 & 4 & 1 \end{bmatrix}$$

$$mx := 1$$

$$mx := 2$$

$$A1 := \begin{bmatrix} 2 & -1 & 3 & 9 \\ 1 & 1 & 1 & 6 \\ 1 & 4 & 1 & 12 \end{bmatrix}$$

$$m := \frac{1}{2}$$

$$A1_{2,\,1} := 0$$

$$A1_{2,\,2} := \frac{3}{2}$$

$$A1_{2,\,3} := \frac{-1}{2}$$

$$A1_{2,\,4} := \frac{3}{2}$$

$$m := \frac{1}{2}$$

$$A1_{3,\,1} := 0$$

$$A1_{3,\,2} := \frac{9}{2}$$

$$A1_{3,\,3} := \frac{-1}{2}$$

$$A1_{3,\,4} := \frac{15}{2}$$

$$mx := 2$$

$$mx := 3$$

$$A1 := \begin{bmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{9}{2} & \frac{-1}{2} & \frac{15}{2} \\ 0 & \frac{3}{2} & \frac{-1}{2} & \frac{3}{2} \end{bmatrix}$$

$$m := \frac{1}{3}$$

$$A1_{3,\,2} := 0$$

$$A1_{3,3} := \frac{-1}{3}$$
$$A1_{3,4} := -1$$

```
<-- exit spgauss (now in gauss) = array(1 .. 3, 1 .. 4,[(3,
4)=-1,(2, 3)=-1/2,(1, 4)=9,(1, 1)=2,(3, 1)=0,(2, 1)=0,(1, 3)=3,(2,
4)=15/2,(3, 2)=0,(1, 2)=-1,(3, 3)=-1/3,(2, 2)=9/2])}
```

$$A1 := \begin{bmatrix} 2 & -1 & 3 & 9 \\ 0 & \frac{9}{2} & \frac{-1}{2} & \frac{15}{2} \\ 0 & 0 & \frac{-1}{3} & -1 \end{bmatrix}$$
$$x_3 := 3$$
$$s := 0$$
$$s := \frac{-3}{2}$$
$$x_2 := 2$$
$$s := 0$$
$$s := -2$$
$$s := 7$$
$$x_1 := 1$$

```
<-- exit gauss (now at top level) = x = 1, y = 2, z = 3}
```

$$x = 1,\ y = 2,\ z = 3$$

Observaţia 2.1.4. Pachetul `linalg` furnizează procedurile `gausselim` şi `backsub`. Astfel, procedura `gausselim` efectuează eliminarea gaussiană cu pivot parţial asupra unei matrice $n \times m$. Procedura `backsub` ia ca argument rezultatul procedurii `gausselim` si furnizează soluţia sistemului. Astfel, pentru matricea din exemplul precedent, avem:

```
> A := matrix([[1, 1, 1, 6], [2, -1, 3, 9], [1, 4, 1, 12]]);
```

$$A := \begin{bmatrix} 1 & 1 & 1 & 6 \\ 2 & -1 & 3 & 9 \\ 1 & 4 & 1 & 12 \end{bmatrix}$$

```
> gausselim(A);
```

$$\begin{bmatrix} 1 & 1 & 1 & 6 \\ 0 & -3 & 1 & -3 \\ 0 & 0 & 1 & 3 \end{bmatrix}$$

```
> backsub(%);
```

$$[1,\ 2,\ 3]$$

2.2 Factorizarea LU

2.2.1 Breviar teoretic

Fie sistemul compatibil determinat

$$Ax = b. \tag{2.4}$$

Factorizarea LU presupune descompunerea matricei A într-un produs de matrice $L \cdot U$, unde

$$L = \begin{pmatrix} \lambda_{11} & 0 & \dots & 0 \\ \lambda_{21} & \lambda_{22} & \dots & 0 \\ \dots & \dots & \dots & \dots \\ \lambda_{n1} & \lambda_{n2} & \dots & \lambda_{nn} \end{pmatrix} \qquad U = \begin{pmatrix} \mu_{11} & \mu_{12} & \dots & \mu_{1n} \\ 0 & \mu_{22} & \dots & \mu_{2n} \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & \mu_{nn} \end{pmatrix}. \tag{2.5}$$

Această descompunere este posibilă dacă toţi determinanţii de colţ ai matricei A sunt nenuli.

Pentru a asigura unicitatea descompunerii, trebuie precizate n elemente ale matricei L sau U. În mod tradiţional, se specifică λ_{ii} sau μ_{ii}; dacă $\lambda_{ii} = 1$ atunci factorizarea LU se numeşte **factorizare Doolittle**, iar dacă $\mu_{ii} = 1$ se numeşte **factorizare Crout**.

Astfel, rezolvarea sistemului (2.4) se reduce la rezolvarea sistemelor triunghiulare

$$Ly = b \tag{2.6}$$

cu soluţia

$$\begin{cases} y_1 = \dfrac{b_1}{\lambda_{11}} \\ y_i = \left(b_i - \displaystyle\sum_{j=1}^{i-1} \lambda_{ij} y_j \right) \cdot \dfrac{1}{\lambda_{ii}}, \ i = 2, 3, \dots, n \end{cases} \tag{2.7}$$

şi

$$Ux = y \tag{2.8}$$

cu soluţia

$$\begin{cases} x_n = \dfrac{y_n}{\mu_{nn}} \\ x_i = \left(y_i - \displaystyle\sum_{j=i+1}^{n} \mu_{ij} x_j \right) \cdot \dfrac{1}{\mu_{ii}}, \ i = 2, 3, \dots, n. \end{cases} \tag{2.9}$$

2.2.2 Problemă rezolvată

Exerciţiul 2.2.1. Să se determine soluţia sistemului următor, folosind factorizarea LU:

$$\begin{cases} x + y - z = 2 \\ 2x - y + z = 1 \\ x + 3y - 2z = 5. \end{cases}$$

Sistemul se scrie în forma matriceală:

$$A\mathrm{x} = b,$$

unde

$$A = \begin{pmatrix} 1 & 1 & -1 \\ 2 & -1 & 1 \\ 1 & 3 & -2 \end{pmatrix}, \quad \mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 1 \\ 5 \end{pmatrix}.$$

Deoarece

$$1 \neq 0\,, \ \begin{vmatrix} 1 & 1 \\ 2 & -1 \end{vmatrix} = -3 \neq 0\,, \ \begin{vmatrix} 1 & 1 & -1 \\ 2 & -1 & 1 \\ 1 & 3 & -2 \end{vmatrix} = -3 \neq 0\,,$$

rezultă că matricea A este nesingulară şi are toţi determinanţii de colţ nenuli, deci se poate folosi factorizarea LU pentru rezolvarea acestui sistem.

Rezolvare folosind factorizarea Crout

A. Factorizarea Crout

Presupunem că

$$A = \begin{pmatrix} 1 & 1 & -1 \\ 2 & -1 & 1 \\ 1 & 3 & -2 \end{pmatrix} = \begin{pmatrix} \lambda_{11} & 0 & 0 \\ \lambda_{21} & \lambda_{22} & 0 \\ \lambda_{31} & \lambda_{32} & \lambda_{33} \end{pmatrix} \cdot \begin{pmatrix} 1 & \mu_{12} & \mu_{13} \\ 0 & 1 & \mu_{23} \\ 0 & 0 & 1 \end{pmatrix},$$

şi ne propunem să determinăm coeficienţii $l_{ij},\ u_{jk}$. Pentru aceasta, folosim definiţia înmulţirii matricelor. Astfel, avem:

$$\begin{aligned}
a_{11} &= \lambda_{11} \cdot 1 && \Rightarrow \lambda_{11} = 1 \\
a_{12} &= \lambda_{11} \cdot \mu_{12} && \Rightarrow \mu_{12} = 1 \\
a_{13} &= \lambda_{11} \cdot \mu_{13} && \Rightarrow \mu_{13} = -1 \\
a_{21} &= \lambda_{21} \cdot 1 && \Rightarrow \lambda_{21} = 2 \\
a_{22} &= \lambda_{21} \cdot \mu_{12} + \lambda_{22} \cdot 1 && \Rightarrow \lambda_{22} = -3 \\
a_{23} &= \lambda_{21} \cdot \mu_{13} + \lambda_{22} \cdot \mu_{23} && \Rightarrow \mu_{23} = -1 \\
a_{31} &= \lambda_{31} \cdot 1 && \Rightarrow \lambda_{31} = 1 \\
a_{32} &= \lambda_{31} \cdot \mu_{12} + \lambda_{32} \cdot 1 && \Rightarrow \lambda_{32} = 2 \\
a_{33} &= \lambda_{31} \cdot \mu_{13} + \lambda_{32} \cdot \mu_{23} + \lambda_{33} \cdot 1 && \Rightarrow \lambda_{33} = 1
\end{aligned}$$

sau

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & -3 & 0 \\ 1 & 2 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 1 & 1 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}.$$

B. Rezolvarea sistemelor triunghiulare

Pentru rezolvarea sistemului iniţial, avem de rezolvat două sisteme triungiulare:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & -3 & 0 \\ 1 & 2 & 1 \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 5 \end{pmatrix},$$

a cărui soluţie este

$$\mathrm{y} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix},$$

şi respectiv:

$$\begin{pmatrix} 1 & 1 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix},$$

a cărui soluţie este

$$\mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}.$$

Rezolvare folosind factorizarea Doolittle

A. Factorizarea Doolittle

Presupunem că

$$A = \begin{pmatrix} 1 & 1 & -1 \\ 2 & -1 & 1 \\ 1 & 3 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ \lambda_{21} & 1 & 0 \\ \lambda_{31} & \lambda_{32} & 1 \end{pmatrix} \cdot \begin{pmatrix} \mu_{11} & \mu_{12} & \mu_{13} \\ 0 & \mu_{22} & \mu_{23} \\ 0 & 0 & \mu_{33} \end{pmatrix}$$

şi ne propunem să determinăm coeficienţii l_{ij}, μ_{jk}, la fel ca şi în exemplul precedent. Astfel avem:

$$\begin{aligned}
a_{11} &= 1 \cdot \mu_{11} && \Rightarrow \mu_{11} = 1 \\
a_{12} &= 1 \cdot \mu_{12} && \Rightarrow \mu_{12} = 1 \\
a_{13} &= 1 \cdot \mu_{13} && \Rightarrow \mu_{13} = -1 \\
a_{21} &= \lambda_{21} \cdot \mu_{11} && \Rightarrow \lambda_{21} = 2 \\
a_{22} &= \lambda_{21} \cdot \mu_{12} + 1 \cdot \mu_{22} && \Rightarrow \mu_{22} = -3 \\
a_{23} &= \lambda_{21} \cdot \mu_{13} + 1 \cdot \mu_{23} && \Rightarrow \mu_{23} = 3 \\
a_{31} &= \lambda_{31} \cdot \mu_{11} && \Rightarrow \lambda_{31} = 1 \\
a_{32} &= \lambda_{31} \cdot \mu_{12} + \lambda_{32} \cdot \mu_{22} && \Rightarrow \lambda_{32} = -\frac{2}{3} \\
a_{33} &= \lambda_{31} \cdot \mu_{13} + \lambda_{32} \cdot \mu_{23} + 1 \cdot \mu_{33} && \Rightarrow \mu_{33} = 1
\end{aligned}$$

sau

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & -\frac{2}{3} & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 1 & 1 & -1 \\ 0 & -3 & 3 \\ 0 & 0 & 1 \end{pmatrix}.$$

B. Rezolvarea sistemelor triunghiulare

Pentru rezolvarea sistemului iniţial, avem de rezolvat două sisteme triungiulare:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & -\frac{2}{3} & 1 \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 5 \end{pmatrix},$$

a cărui soluţie este

$$\mathrm{y} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 2 \\ -3 \\ 1 \end{pmatrix},$$

şi respectiv:

$$\begin{pmatrix} 1 & 1 & -1 \\ 0 & -3 & 3 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 2 \\ -3 \\ 1 \end{pmatrix},$$

a cărui soluţie este

$$\mathrm{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}.$$

2.2.3 Probleme propuse

Exerciţiul 2.2.2. Să se găsească soluţiile următoarelor sisteme, folosind cele două variante ale factorizării LU:

a) $\begin{cases} x+2y+z=1 \\ 3x-y+5z=14 \\ x+y-z=-2 \end{cases}$

b) $\begin{cases} 3x+y-2z=1 \\ x+y+z=6 \\ -2x-y+4z=7 \end{cases}$

2.2.4 Implementare

A. Algoritm

Date de intrare: un sistem de ecuaţii
Date de ieşire: soluţia sistemului

Algoritmul constă din următoarele etape:

1. generarea matricei A a sistemului, şi a vectorului coloană b
   - n = numărul de linii ale matricei A (numărul de ecuaţii ale sistemului)
2. a) factorizarea Crout

   pentru $i = \overline{1,n}$

   $\mu_{ii} = 1$

   pentru $i = \overline{1,n}$

   pentru $j = \overline{1,i}$

   $$\lambda_{ij} = a_{ij} - \sum_{k=1}^{j-1} \lambda_{ik}\mu_{kj}$$

   pentru $j = \overline{i+1,n}$

   $$\mu_{ij} = \frac{1}{\lambda_{ii}} \left(a_{ij} - \sum_{k=1}^{i-1} \lambda_{ik}\mu_{kj} \right)$$

   b) factorizarea Doolittle

   pentru $i = \overline{1,n}$

   $\lambda_{ii} = 1$

   pentru $i = \overline{1,n}$

   pentru $j = \overline{1,i-1}$

   $$\lambda_{ij} = \frac{1}{\mu_{jj}} \left(a_{ij} - \sum_{k=1}^{i} \lambda_{ik}\mu_{kj} \right)$$

   pentru $j = \overline{i,n}$

$$\mu_{ij} = a_{ij} - \sum_{k=1}^{i-1} \lambda_{ik}\mu_{kj}$$

3. Rezolvarea celor două sisteme triunghiulare

$y_1 = \dfrac{b_1}{\lambda_{11}}$

pentru $i = \overline{2,n}$

$$y_i = \left(b_i - \sum_{j=1}^{i-1} \lambda_{ij} y_j\right) \cdot \frac{1}{\lambda_{ii}}$$

$x_n = \dfrac{y_n}{\mu_{nn}}$

pentru $i = \overline{2,n}$

$$x_i = \left(y_i - \sum_{j=i+1}^{n} \mu_{ij} x_j\right) \cdot \frac{1}{\mu_{ii}}$$

B. Programe MAPLE şi rezultate

Deoarece cele două variante ale descompunerii LU diferă doar prin modul de factorizare a matricei sistemului, am implementat separat cele două variante de factorizare: `LUcrout` şi `LUdoolittle`, după care le-am folosit ca opţiuni în procedura finală `LUsist`.

```
restart: with(linalg):

LUcrout:=proc(A::matrix)
local a1,n,l,u,i,s,j,k;
n:=rowdim(A);
a1:=A;
if a1[1,1]=0 then
  ERROR(‘factorizarea LU nu este aplicabila!‘);
fi;
for i from n by -1 to 2 do
  if det(a1)<>0 then a1:=delrows(delcols(a1,i..i),i..i);
  else ERROR(‘factorizarea LU nu este aplicabila!‘);
  fi;
od;
l:=matrix(n,n,0); u:=matrix(n,n,0);
for i from 1 to n do
  u[i,i]:=1;
od;
for i from 1 to n do
  for j from 1 to i do
    s:=0; for k from 1 to j-1 do s:=s+l[i,k]*u[k,j]; od;
    l[i,j]:=A[i,j]-s;
  od;
```

```
  for j from i+1 to n do
    s:=0; for k from 1 to i-1 do s:=s+l[i,k]*u[k,j]; od;
    u[i,j]:=1/l[i,i]*(A[i,j]-s);
  od;
od;
RETURN(evalm(l), evalm(u));
end:

LUsist:=proc(l::set(equation), opt::symbol)
local lst, eqm, A, b, n, lu, L, U,i,s,j,aux, rez, rfin;
eqm:=genmatrix(l, [op(indets(l))], flag);
lst:=indets(l);
n:=nops(lst);
A:=delcols(eqm,n+1..n+1);
b:=col(eqm,n+1);
if opt=Crout then
  lu:=LUcrout(A);
elif opt=Doolittle then
  lu:=LUdoolittle(A);
else ERROR(‘optiunile sunt: Crout sau Doolittle‘)
fi;
L:=lu[1];
U:=lu[2];
for i from 1 to n do
  s:=0; for j from 1 to i-1 do s:=s+L[i,j]*aux[j] od;
  aux[i]:=1/L[i,i]*(b[i]-s)
od;
for i from n by -1 to 1 do
  s:=0; for j from i+1 to n do s:=s+U[i,j]*rez[j] od;
  rez[i]:=1/U[i,i]*(aux[i]-s)
od;
RETURN(seq(lst[i]=rez[i], i=1..n));
end:

debug(LUsist);
LUsist({x+y-z=2,2*x-y+z=1,x+3*y-2*z=5}, Crout);
```

```
{--> enter LUsist, args = {x+y-z = 2, 2*x-y+z = 1, x+3*y-2*z = 5},
Crout
```

$$eqm := \begin{bmatrix} -1 & 1 & 1 & 2 \\ 1 & 2 & -1 & 1 \\ -2 & 1 & 3 & 5 \end{bmatrix}$$

$$lst := \{z, x, y\}$$

$$n := 3$$

$$A := \begin{bmatrix} -1 & 1 & 1 \\ 1 & 2 & -1 \\ -2 & 1 & 3 \end{bmatrix}$$

$$b := [2,\ 1,\ 5]$$

$$lu := \begin{bmatrix} -1 & 0 & 0 \\ 1 & 3 & 0 \\ -2 & -1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & -1 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$L := \begin{bmatrix} -1 & 0 & 0 \\ 1 & 3 & 0 \\ -2 & -1 & 1 \end{bmatrix}$$

$$U := \begin{bmatrix} 1 & -1 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$s := 0$$
$$aux_1 := -2$$
$$s := 0$$
$$s := -2$$
$$aux_2 := 1$$
$$s := 0$$
$$s := 4$$
$$s := 3$$
$$aux_3 := 2$$
$$s := 0$$
$$rez_3 := 2$$
$$s := 0$$
$$s := 0$$
$$rez_2 := 1$$
$$s := 0$$
$$s := -1$$
$$s := -3$$
$$rez_1 := 1$$

```
<-- exit LUsist (now at top level) = z = 1, x = 1, y = 2}
```

$$z = 1,\ x = 1,\ y = 2$$

2.3 Sisteme tridiagonale

2.3.1 Breviar teoretic

O clasă specială de sisteme liniare este aceea în care matricea A a sistemului este tridiagonală, adică:

$$A = \begin{pmatrix} b_1 & c_2 & 0 & 0 & \dots & \dots & \dots & 0 \\ a_2 & b_2 & c_3 & 0 & \dots & \dots & \dots & 0 \\ 0 & a_3 & b_3 & c_3 & \dots & \dots & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & \dots & \dots & \dots & a_{n-1} & b_{n-1} & c_n \\ 0 & \dots & \dots & \dots & \dots & 0 & a_n & b_n \end{pmatrix}. \tag{2.10}$$

Pentru aceste sisteme se aplică factorizarea LU. Astfel, matricea A se descompune, folosind un caz particular al factorizării Crout, într-un produs $L \cdot U$ unde:

$$L = \begin{pmatrix} \beta_1 & 0 & 0 & \dots & 0 & 0 \\ a_2 & \beta_2 & 0 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & \beta_{n-1} & 0 \\ 0 & 0 & 0 & \dots & a_n & \beta_n \end{pmatrix} \tag{2.11}$$

şi

$$U = \begin{pmatrix} 1 & \nu_2 & 0 & \dots & 0 & 0 \\ 0 & 1 & \nu_3 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & 1 & \nu_n \\ 0 & 0 & 0 & \dots & \dots & 1 \end{pmatrix}. \tag{2.12}$$

Coeficienţii a_2, ..., a_n sunt cunoscuţi din matricea A, iar coeficienţii β_i, μ_j se obţin din definiţia înmulţirii matricelor:

$$\begin{aligned} &\beta_1 = b_1 \\ &\beta_i \cdot \nu_{i+1} = c_{i+1}\,,\ i = \overline{2, n-1} \\ &a_i \cdot \nu_i + \beta_i = b_i\,,\ i = \overline{2, n} \end{aligned} \tag{2.13}$$

2.3.2 Problemă rezolvată

Exerciţiul 2.3.1. Să se rezolve sistemul tridiagonal:

$$\begin{cases} x & +2y & & & = 3 \\ 2x & -y & +z & & = 2 \\ & 3y & +2z & -t & = 4 \\ & & -2z & +t & = -1. \end{cases}$$

Rezolvare

Matricea sistemului este

$$A = \begin{pmatrix} 1 & 2 & 0 & 0 \\ 2 & -1 & 1 & 0 \\ 0 & 3 & 2 & -1 \\ 0 & 0 & -2 & 1 \end{pmatrix}$$

Descompunem această matrice astfel:

$$\begin{pmatrix} 1 & 2 & 0 & 0 \\ 2 & -1 & 1 & 0 \\ 0 & 3 & 2 & -1 \\ 0 & 0 & -2 & 1 \end{pmatrix} = \begin{pmatrix} \beta_1 & 0 & 0 & 0 \\ 2 & \beta_2 & 0 & 0 \\ 0 & 3 & \beta_3 & 0 \\ 0 & 0 & -2 & \beta_4 \end{pmatrix} \cdot \begin{pmatrix} 1 & \nu_2 & 0 & 0 \\ 0 & 1 & \nu_3 & 0 \\ 0 & 0 & 1 & \nu_4 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Din definiţia produsului a două matrice, obţinem:

$$\begin{aligned}
b_1 &= \beta_1 \cdot 1 && \Rightarrow \beta_1 = 1\\
c_2 &= \beta_1 \cdot \nu_2 && \Rightarrow \nu_2 = 2\\
b_2 &= a_2 \cdot \nu_2 + \beta_2 && \Rightarrow \beta_2 = -5\\
c_3 &= \beta_2 \nu_3 && \Rightarrow \nu_3 = -\frac{1}{5}\\
b_3 &= a_3 \nu_3 + \beta_3 && \Rightarrow \beta_3 = \frac{13}{5}\\
c_4 &= \beta_3 \cdot \nu_4 && \Rightarrow \nu_4 = -\frac{5}{13}\\
b_4 &= a_4 \cdot \nu_4 + \beta_4 && \Rightarrow \beta_4 = \frac{3}{13}.
\end{aligned}$$

B. Rezolvarea sistemelor triunghiulare

Pentru a rezolva sistemul iniţial, avem de rezolvat două sisteme triunghiulare:

$$\begin{pmatrix} 1 & 0 & 0 & 0\\ 2 & -5 & 0 & 0\\ 0 & 3 & \frac{13}{5} & 0\\ 0 & 0 & -2 & \frac{3}{13} \end{pmatrix} \cdot \begin{pmatrix} y_1\\ y_2\\ y_3\\ y_4 \end{pmatrix} = \begin{pmatrix} 3\\ 2\\ 4\\ -1 \end{pmatrix},$$

a cărui soluţie este

$$\begin{pmatrix} y_1\\ y_2\\ y_3\\ y_4 \end{pmatrix} = \begin{pmatrix} 3\\ \frac{4}{5}\\ \frac{8}{13}\\ 1 \end{pmatrix},$$

şi respectiv:

$$\begin{pmatrix} 1 & 2 & 0 & 0\\ 0 & 1 & -\frac{1}{5} & 0\\ 0 & 0 & 1 & -\frac{5}{13}\\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x\\ y\\ z\\ t \end{pmatrix} = \begin{pmatrix} 3\\ \frac{4}{5}\\ \frac{8}{13}\\ 1 \end{pmatrix},$$

a cărui soluţie este

$$\begin{pmatrix} x\\ y\\ z\\ t \end{pmatrix} = \begin{pmatrix} 1\\ 1\\ 1\\ 1 \end{pmatrix}.$$

2.3.3 Probleme propuse

Exerciţiul 2.3.2. Să se rezolve sistemele tridiagonale:

a) $\begin{cases} x + y = 3\\ 2x - y + z = 1\\ 3y - z = 5 \end{cases}$

b) $\begin{cases} 2x + y = 0\\ x - y + 2z = -1\\ 2y - z + t = 5\\ z + 2t = 5. \end{cases}$

2.3.4 Implementare

A. Algoritm

Date de intrare: un sistem de ecuaţii tridiagonal
Date de ieşire: soluţia sistemului

Algoritmul constă în:

1. generarea matricei A a sistemului (matrice tridiagonală) şi a vectorului coloană b
   - $n =$ numărul de linii ale matricei A
2. descompunerea LU aplicată matricei tridiagonale A

   $L = (\lambda_{ij})_{i,j=\overline{1,n}}$, $U = (\mu_{ij})_{i,j=\overline{1,n}}$

   pentru $i = \overline{1,n}$

   $\mu_{ii} = 1$

   pentru $i = \overline{2,n}$

   $\lambda_{i,i-1} = a_{i,i-1}$

   $\lambda_{11} = a_{11}$

   pentru $i = \overline{1,n-1}$

   $\mu_{i,i+1} = \dfrac{a_{i,i+1}}{\lambda_{ii}}$

   $\lambda_{i+1,i+1} = a_{i+1,i+1} - a_{i+1,i} \cdot \mu_{i,i+1}$
3. rezolvarea sistemelor triunghiulare

   $y_1 = \dfrac{b_1}{\lambda_{11}}$

   pentru $i = \overline{2,n}$

   $y_i = \left(b_i - \lambda_{i,i-1} y_{i-1}\right) \cdot \dfrac{1}{\lambda_{ii}}$

   $x_n = y_n$

   pentru $i = \overline{n-1,1}$

   $x_i = \left(y_i - \mu_{i,i+1} x_{i+1}\right)$

B. Programe MAPLE şi rezultate

Observaţia 2.3.1. Spre deosebire de metodele anterioare, unde ordinea necunoscutelor în sistem nu era esenţială, în cazul sistemelor tridiagonale, dacă se schimbă ordinea necunoscutelor, atunci matricea sistemului nu va mai fi tridiagonală. De aceea, este necesară construirea unei proceduri, `nedeterminate`, care să returneze necunoscutele sistemului în ordinea în care apar ele în ecuaţii.

```
restart: with(linalg):

tridiagonal:=proc(A::matrix)
local i,j,n,a1,l,u;
n:=rowdim(A);
for i from 1 to n do
  for j from 1 to i-2 do
    if A[i,j]<>0 then
      ERROR(‘matricea nu este tridiagonala!‘);
    fi;
  od;
  for j from i+2 to n do
    if A[i,j]<>0 then
      ERROR(‘matricea nu este tridiagonala!‘);
    fi;
  od;
od;
a1:=A;
if a1[1,1]=0 then
  ERROR(‘factorizarea LU nu este aplicabila!‘);
fi;
for i from n by -1 to 2 do
  if det(a1)<>0 then a1:=delrows(delcols(a1,i..i),i..i);
  else ERROR(‘factorizarea LU nu este aplicabila!‘);
  fi;
od;
l:=matrix(n,n,0); u:=matrix(n,n,0);
for i from 1 to n do
  u[i,i]:=1;
od;
for i from 2 to n do
  l[i,i-1]:=A[i,i-1];
od;
l[1,1]:=A[1,1];
for i from 1 to n-1 do
  u[i,i+1]:=A[i,i+1]/l[i,i];
  l[i+1,i+1]:=A[i+1,i+1]-A[i+1,i]*u[i,i+1];
od;
RETURN(evalm(l), evalm(u));
end:

# procedura care returneaza necunoscutele sistemului
# in ordinea in care apar in ecuatii

nedeterminate:=proc(l::set(equation))
local n,i,j,ops,opst;
```

```
n:=nops(l);
for i from 1 to n do
ops[i]:=[seq(op(op(l[i])[1])[j] /
         coeff(op(l[i])[1],op(indets(op(op(l[i] )[1])[j]))),
         j=1..nops(op(l[i])[1]))];
od;
opst:=ops[1];
for i from 1 to n do
  for j from 1 to nops(ops[i]) do
    if not(ops[i][j] in opst) then
      opst:=[op(opst),ops[i][j]]
    fi;
  od;
od;
RETURN(opst);
end:

tridiagonalsist:=proc(l::set(equation))
local eqm, opst, A, b, n, lu, L, U, i, s, j, aux, rez;
n:=nops(l);
opst:=nedeterminate(l);
eqm:=genmatrix(l, opst, flag);
A:=delcols(eqm,n+1..n+1);
b:=col(eqm,n+1);
lu:=tridiagonal(A);
L:=lu[1];
U:=lu[2];
aux[1]:=b[1]/L[1,1];
for i from 2 to n do
  aux[i]:=1/L[i,i]*(b[i]-L[i,i-1]*aux[i-1])
od;
rez[n]:=aux[n];
for i from n-1 by -1 to 1 do
  rez[i]:=aux[i]-U[i,i+1]*rez[i+1];
od;
RETURN(seq(opst[i]=rez[i], i=1..n));
end:

debug(tridiagonalsist):

tridiagonalsist({x+2*y=3,2*x-y+z=2, 3*y+2*z-t=4, -2*z+t=-1});
```

```
{--> enter tridiagonalsist, args = {x+2*y = 3, 2*x-y+z = 2, 3*y+2*z-t
= 4, -2*z+t = -1}
```

$$n := 4$$

$$opst := [x,\ y,\ z,\ t]$$

$$eqm := \begin{bmatrix} 1 & 2 & 0 & 0 & 3 \\ 2 & -1 & 1 & 0 & 2 \\ 0 & 3 & 2 & -1 & 4 \\ 0 & 0 & -2 & 1 & -1 \end{bmatrix}$$

$$A := \begin{bmatrix} 1 & 2 & 0 & 0 \\ 2 & -1 & 1 & 0 \\ 0 & 3 & 2 & -1 \\ 0 & 0 & -2 & 1 \end{bmatrix}$$

$$b := [3,\ 2,\ 4,\ -1]$$

$$lu := \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & -5 & 0 & 0 \\ 0 & 3 & \frac{13}{5} & 0 \\ 0 & 0 & -2 & \frac{3}{13} \end{bmatrix}, \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 1 & \frac{-1}{5} & 0 \\ 0 & 0 & 1 & \frac{-5}{13} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$L := \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & -5 & 0 & 0 \\ 0 & 3 & \frac{13}{5} & 0 \\ 0 & 0 & -2 & \frac{3}{13} \end{bmatrix}$$

$$U := \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 1 & \frac{-1}{5} & 0 \\ 0 & 0 & 1 & \frac{-5}{13} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$aux_1 := 3$$

$$aux_2 := \frac{4}{5}$$

$$aux_3 := \frac{8}{13}$$

$$aux_4 := 1$$

$$rez_4 := 1$$

$$rez_3 := 1$$

$$rez_2 := 1$$

$$rez_1 := 1$$

```
<-- exit tridiagonalsist (now at top level) = x = 1, y = 1, z = 1, t =
1}
```

$$x = 1,\ y = 1,\ z = 1,\ t = 1$$

2.4 Factorizarea Cholesky

2.4.1 Breviar teoretic

Un caz particular de sisteme liniare este acela în care matricea A a sistemului este simetrică şi pozitiv definită (adică toţi determinanţii de colţ sunt strict pozitivi). Pentru astfel de sisteme putem folosi un caz particular al factorizării LU: descompunem matricea A a sistemului într-un produs $L \cdot L^T$, unde

$$L = \begin{pmatrix} \lambda_{11} & 0 & \dots & 0 \\ \lambda_{21} & \lambda_{22} & \dots & 0 \\ \dots & \dots & \dots & \dots \\ \lambda_{n1} & \lambda_{n2} & \dots & \lambda_{nn} \end{pmatrix}. \tag{2.14}$$

Coeficienţii λ_{ij} se obţin din definiţia produsului a două matrice.

2.4.2 Problemă rezolvată

Exerciţiul 2.4.1. Să se rezolve sistemul:

$$\begin{cases} x + 2y + z = 5 \\ 2x + 5y + 2z = 11 \\ x + 2y + 3z = 7. \end{cases}$$

Rezolvare

A. Factorizarea Cholesky

Matricea sistemului este

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 3 \end{pmatrix}$$

Se observă că $a_{ij} = a_{ji}$, adică matricea A este simetrică. Deoarece

$$1 > 0\,, \quad \begin{vmatrix} 1 & 2 \\ 2 & 5 \end{vmatrix} = 1 > 0\,, \quad \begin{vmatrix} 1 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 3 \end{vmatrix} = 2 > 0\,,$$

matricea A este pozitiv definită. Aplicând factorizarea Cholesky avem:

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 3 \end{pmatrix} = \begin{pmatrix} \lambda_{11} & 0 & 0 \\ \lambda_{21} & \lambda_{22} & 0 \\ \lambda_{31} & \lambda_{32} & \lambda_{33} \end{pmatrix} \cdot \begin{pmatrix} \lambda_{11} & \lambda_{21} & \lambda_{31} \\ 0 & \lambda_{22} & \lambda_{32} \\ 0 & 0 & \lambda_{33} \end{pmatrix}.$$

Folosind definiţia produsului a două matrice, obţinem:

$$\begin{aligned} a_{11} &= \lambda_{11}^2 && \Rightarrow \lambda_{11} = 1 \\ a_{12} &= \lambda_{11} \cdot \lambda_{21} && \Rightarrow \lambda_{21} = 2 \\ a_{13} &= \lambda_{11} \cdot \lambda_{31} && \Rightarrow \lambda_{31} = 1 \\ a_{22} &= \lambda_{21}^2 + \lambda_{22}^2 && \Rightarrow \lambda_{22} = 1 \\ a_{23} &= \lambda_{21} \cdot \lambda_{31} + \lambda_{22} \cdot \lambda_{32} && \Rightarrow \lambda_{32} = 0 \\ a_{33} &= \lambda_{31}^2 + \lambda_{32}^2 + \lambda_{33}^2 && \Rightarrow \lambda_{33} = \sqrt{2}. \end{aligned}$$

Se observă că pentru găsirea elementelor λ_{ij}, $i = \overline{1,n}$, $j = \overline{i,n}$, este suficient să calculăm dezvoltările corespunzătoare elementelor a_{ij}, $i = \overline{1,n}$, $j = \overline{i,n}$.

B. Rezolvarea sistemelor triunghiulare

Pentru a determina soluţia sistemului iniţial, avem de rezolvat două sisteme triunghiulare:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & \sqrt{2} \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \\ 7 \end{pmatrix},$$

cu soluţia

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \\ \sqrt{2} \end{pmatrix},$$

şi

$$\begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & \sqrt{2} \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \\ \sqrt{2} \end{pmatrix},$$

cu soluţia

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}.$$

2.4.3 Probleme propuse

Exerciţiul 2.4.2. Să se găsească soluţia sistemului următor, folosind factorizarea Cholesky:

$$\begin{cases} 3x + y + 3z = 11 \\ x + y + 2z = 6 \\ 3x + y + 4z = 12. \end{cases}$$

2.4.4 Implementare

A. Algoritm

Date de intrare: un sistem de ecuaţii

Date de ieşire: soluţia sistemului

Algoritm

1. generarea matricei A a sistemului (simetrică şi pozitiv definită) şi a vectorului b
   - n = numărul de ecuaţii ale sistemului (numărul de linii ale matricei A)
2. factorizarea Cholesky

   $\lambda_{11} = a_{11}$

   pentru $i = \overline{2,n}$

   pentru $j = \overline{1,i-1}$

   $$\lambda_{ij} = \frac{1}{\lambda_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \lambda_{ik}\lambda_{jk} \right)$$

$$\lambda_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} \lambda_{ik}^2}$$

3. rezolvarea sistemelor triunghiulare

$y_1 = \dfrac{b_1}{\lambda_{11}}$

pentru $i = \overline{2, n}$

$$y_i = \left(b_i - \sum_{k=1}^{i-1} \lambda_{ik} \cdot y_k \right) \cdot \frac{1}{\lambda_{ii}}$$

$x_n = \dfrac{y_n}{\lambda_{nn}}$

pentru $i = \overline{n-1, 1}$

$$x_i = \left(y_i - \sum_{k=i+1}^{n} \lambda_{ki} \cdot x_k \right) \cdot \frac{1}{\lambda_{ii}}$$

B. Programe MAPLE şi rezultate

2.5 Factorizarea Householder

2.5.1 Breviar teoretic

Factorizarea Householder este o metodă de rezolvare numerică a sistemelor de tip Cramer simetrice, şi constă în determinarea unei matrice simetrice nesingulare U, astfel încât $UAU = T$ să fie o matrice tridiagonală. Atunci soluţia sistemului $Ax = b$ este dată de

$$x = Uy, \tag{2.15}$$

unde y este soluţia sistemului

$$Ty = Ub. \tag{2.16}$$

Factorizarea Householder se bazează pe următoarele rezultate.

Propoziţia 2.5.1. *Oricare ar fi A o matrice pătratică de ordinul n şi simetrică, există un vector $v = (v_1, v_2, \ldots, v_n)^T$ astfel încât vectorul coloană $a^1 = Ae_1, e_1 = (1, 0, \ldots, 0)^T$ (a^1 este prima coloană a matricei A) are proprietatea*

$$a^1 - \frac{2v \cdot < v, a^1 >}{\|v\|^2} = \lambda \cdot e_1. \tag{2.17}$$

Pentru evitarea ambiguităţilor vom considera vectorul v dat de:

$$v = a^1 + sign(a_{11}) \cdot \|a^1\| \cdot e_1. \tag{2.18}$$

Propoziţia 2.5.2. *Oricare ar fi matricea simetrică A, matricea P definită prin:*

$$P = I - \frac{2 \cdot v \cdot v^T}{\|v\|^2} \tag{2.19}$$

este simetrică şi are proprietatea că elementele $2, 3, \ldots, n$ de pe prima coloană a matricei PA sunt nule, unde vectorul v este dat de relaţia (2.18).

Definiţia 2.5.1. *Se numeşte* ***matrice Householder*** *de ordin $n-1$ asociată matricei A şi se notează cu P_{n-1} o matrice de ordin $n-1$ de forma:*

$$P_{n-1} = I_{n-1} - \frac{2 \cdot v \cdot v^T}{\|v\|^2} \tag{2.20}$$

unde: $v = a^1_{n-1} + sign(a_{21}) \cdot \|a^1_{n-1}\| \cdot e_1$ este vectorul format cu componentele vectorului a^1 care este prima coloană a matricei A, $e_1 = (\underbrace{1, 0, \ldots, 0}_{n-1})^T$ şi I_{n-1} este matricea unitate de ordin $n-1$.

Propoziţia 2.5.3. *Matricea Hauseholder P_{n-1} asociată unei matrice simetrice A este simetrică şi are proprietatea că matricea U_1 definită prin:*

$$U_1 = \begin{pmatrix} 1 & 0 & \ldots & 0 \\ 0 & & & \\ & & P_{n-1} & \\ 0 & & & \end{pmatrix} \tag{2.21}$$

este simetrică şi verifică relaţia:

$$A^{(1)} = U_1 \, A \, U_1 = \begin{pmatrix} a_{11} & \alpha_1 & 0 & \ldots & 0 \\ \alpha^1 & a^{(1)}_{22} & a^{(1)}_{23} & \ldots & a^{(1)}_{2n} \\ 0 & a^{(1)}_{32} & a^{(1)}_{33} & \ldots & a^{(1)}_{3n} \\ \ldots & \ldots & \ldots & \ldots & \ldots \\ 0 & a^{(1)}_{n2} & a^{(1)}_{n3} & \ldots & a^{(1)}_{nn} \end{pmatrix} \tag{2.22}$$

Se consideră vectorul coloană a^2_{n-2} cu ultimele $n-2$ elemente ale coloanei matrice $A^{(1)}$. Cu acest vector se construieşte o matrice Householder de ordinul $n-2$, P_{n-2}. Matricea U_2 definită prin:

$$U_2 = \begin{pmatrix} 1 & 0 & 0 & \ldots & 0 \\ 0 & 1 & 0 & \ldots & 0 \\ 0 & 0 & & & \\ \vdots & \vdots & & P_{n-2} & \\ 0 & 0 & & & \end{pmatrix} \tag{2.23}$$

are proprietatea:

$$A^{(2)} = U_2 \, A^{(1)} \, U_2 = \begin{pmatrix} a_{11} & \alpha_1 & 0 & 0 & \ldots & 0 \\ \alpha_1 & a^{(1)}_{22} & \alpha_2 & 0 & \ldots & 0 \\ 0 & \alpha_2 & a^{(2)}_{33} & a^{(2)}_{34} & \ldots & a^{(2)}_{3n} \\ \ldots & \ldots & \ldots & \ldots & \ldots & \\ 0 & 0 & a^{(2)}_{n3} & a^{(2)}_{n4} & \ldots & a^{(2)}_{nn} \end{pmatrix} \tag{2.24}$$

Matricea P_{n-2} a condus la obţinerea unei noi linii şi coloane a matricei tridiagonale la care vrem să reducem matricea A.

Continuând astfel prin $n-1$ transformări, obţinem egalitatea: $UAU = T$ în care T este matrice tridiagonală.

2.5.2 Problemă rezolvată

Exerciţiul 2.5.1. Să se rezolve următorul sistem folosind factorizarea Householder:

$$\begin{cases} 2x + 2y + z = 2 \\ 2x - y + z = \sqrt{5} \\ x + y + 2z = 0. \end{cases}$$

Rezolvare

Sistemul se poate scrie sub forma $A\mathrm{x} = b$, unde:

$$A := \begin{pmatrix} 2 & 2 & 1 \\ 2 & -1 & 1 \\ 1 & 1 & 2 \end{pmatrix} \quad \text{şi} \quad b := \begin{pmatrix} 2 \\ \sqrt{5} \\ 0 \end{pmatrix}$$

Se observă că matricea A este simetrică, deci factorizarea Householder este aplicabilă.

Generarea matricei U

Calculăm elementele vectorului

$$v = a^1 + sign(a^1_{11}) \cdot \|a^1\| \cdot e_1$$

unde $a^1 = (2,1)^T$, $\|a^1\| = \sqrt{5}$ şi $e_1 = (1,0)^T$. De aici rezultă

$$v = (0, 2+\sqrt{5}, 1)^T$$

şi $\|v\| = 10 + 4\sqrt{5}$. Elementele matricei U sunt date de:

$$U_{j,k} = I_3 - \frac{2v_j \cdot v_k}{\|v\|}.$$

După efectuarea calculelor, obţinem:

$$U = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -\dfrac{2\,(2+\sqrt{5})}{5+2\sqrt{5}} & -\dfrac{2+\sqrt{5}}{5+2\sqrt{5}} \\ 0 & -\dfrac{2+\sqrt{5}}{5+2\sqrt{5}} & \dfrac{2\,(2+\sqrt{5})}{5+2\sqrt{5}} \end{pmatrix}$$

şi

$$T = UAU = \begin{pmatrix} 2 & -\dfrac{5\,(2+\sqrt{5})}{5+2\sqrt{5}} & 0 \\ -\dfrac{5\,(2+\sqrt{5})}{5+2\sqrt{5}} & \dfrac{2}{5} & \dfrac{-9}{5} \\ 0 & \dfrac{-9}{5} & \dfrac{3}{5} \end{pmatrix}, \; Ub = \begin{pmatrix} 2 \\ -2 \\ -1 \end{pmatrix}$$

Soluţia sistemului tridiagonal

$$Ty = Ub$$

este

$$y = \begin{pmatrix} 1 \\ \frac{20-10\sqrt{5}}{21} \\ \frac{5-6\sqrt{5}}{15} \end{pmatrix},$$

iar soluţia sistemului iniţial este

$$x = Uy = \begin{pmatrix} \frac{2+\sqrt{5}}{3} \\ \frac{2-\sqrt{5}}{3} \\ -\frac{2}{3} \end{pmatrix},$$

adică $x = \frac{2+\sqrt{5}}{3}$, $y = \frac{2-\sqrt{5}}{3}$ $z = -\frac{2}{3}$.

2.5.3 Probleme propuse

Exerciţiul 2.5.2. Să se găsească soluţiile următoarelor sisteme folosind factorizarea Householder:

a) $\begin{cases} x + 2y + z = 5 \\ 2x - y + 3z = 6 \\ x + 3y - 2z = 3 \end{cases}$

b) $\begin{cases} x + y - z + t = 3 \\ x + 2y + 3z + t = 9 \\ -x + 3y + 2t = 7 \\ x + y + 2z = 5. \end{cases}$

2.5.4 Implementare

A. Algoritm

Date de intrare: un sistem de ecuaţii

Date de ieşire: soluţia sistemului, obţinută folosind factorizarea Householder

Algoritmul constă din următoarele etape:

1. generarea matricei tridiagonale pentru $i = 1 \ldots n-2$

pentru $l = 1 \ldots n$

pentru $m = 1 \ldots n$

dacă $m = l$ atunci $u_{ml} = 1$

dacă $m \neq l$ atunci $u_{ml} = 0$

//Generăm vectorul v

$$\text{norm a} = \sqrt{\sum_{j=i+1}^{n} a_{ij}^2}$$

$e_{i+1} = 1$

pentru $j = i+2 \ldots n$

$\quad e_j = 0$

pentru $j = i+1 \ldots n$

$\quad v_j = a_{ij} + sign(a_{i,i+1}) \cdot \text{norm a} \cdot e_j$

$$\text{norm v} = \sum_{j=i+1}^{n} v_j^2$$

//Generăm matricea U

$j = i+1 \ldots n$

$\quad k = i+1 \ldots n$

$\quad\quad u_{jk} = u_{jk} - 2 \cdot v_j \cdot v_k / \text{ norm v}$

// D=AU

$m = 1 \ldots n$

$\quad l = 1 \ldots n$

$$\quad\quad d_{ml} = \sum_{k=1}^{n} a_{mk} \cdot u_{kl}$$

//A=UD=UAU

$m = 1 \ldots n$

$\quad l = 1 \ldots n$

$$\quad\quad a_{ml} = \sum_{k=1}^{n} u_{mk} \cdot d_{kl}$$

2. rezolvarea sistemului tridiagonal (vezi paragraful 2.3.4):

$$Ty = Ub$$

3. găsirea soluţiei sistemului iniţial:

$$x = Uy$$

B. Programe MAPLE şi rezultate

Observaţia 2.5.1. Deoarece o condiţie neceesară pentru aplicarea factorizării Householder este ca sistemul să fie simetric, folosim procedura `nedeterminate` (prezentată în paragraful 2.4.4). De asemenea, pentru descompunerea matricei tridiagonale, am folosit procedura `tridiagonal` prezentată în paragraful 2.3.4.

2.6 Metoda Jacobi

2.6.1 Breviar teoretic

Metoda Jacobi este o metodă iterativă de rezolvare a sistemelor liniare de forma

$$Ax = b. \tag{2.25}$$

Matricea A se descompune în suma $L + D + U$, unde

$$L = \begin{pmatrix} 0 & 0 & 0 & \dots & 0 \\ a_{21} & 0 & 0 & \dots & 0 \\ a_{31} & a_{32} & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & a_{n3} & \dots & 0 \end{pmatrix} \tag{2.26}$$

$$D = \begin{pmatrix} a_{11} & 0 & 0 & \dots & 0 \\ 0 & a_{22} & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & a_{nn} \end{pmatrix} \tag{2.27}$$

$$U = \begin{pmatrix} 0 & a_{12} & a_{13} & \dots & a_{1n} \\ 0 & 0 & a_{23} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & a_{n-1,n} \\ 0 & 0 & 0 & \dots & 0 \end{pmatrix}. \tag{2.28}$$

Se defineşte traiectoria Jacobi a vectorului $x^{(0)}$ ca fiind vectorul

$$x^{(k+1)} = D^{-1}[b - (L + U)x^{(k)}] \quad k = 0, 1, 2, \dots \tag{2.29}$$

Folosind teorema de convergenţă se studiază dacă traiectoria Jacobi converge la soluţia $x^{(*)}$ a sistemului (2.25).

Traiectoria Jacobi converge la soluţia $x^{(*)}$ a sistemului (2.25), dacă şi numai dacă raza spectrală ρ a matricei

$$M = -D^{-1}(L + U) \tag{2.30}$$

este strict subunitară, adică

$$\max\{|\lambda| \mid \det(M - \lambda I_n) = 0\} < 1. \tag{2.31}$$

În caz de convergenţă, componentele $x_1^{(k+1)}$, ..., $x_n^{(k+1)}$ ale vectorului $x^{(k+1)}$, situat pe traiectoria Jacobi a vectorului $x^{(0)}$, sunt date de relaţiile:

$$x_i^{(k+1)} = \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij} \cdot x_j^{(k)} \right) \cdot \frac{1}{a_{ii}},\ i = 1, 2, \dots, n;\ k = 0, 1, \dots \tag{2.32}$$

2.6.2 Problemă rezolvată

Exerciţiul 2.6.1. Calculaţi primii trei termeni ai traiectoriei Jacobi asociate vectorului $(0, 0, 0)$ pentru sistemul:

$$\begin{cases} 5x - 2y + 3z = -1 \\ -3x + 9y + z = 2 \\ 2x - y - 7z = 3. \end{cases}$$

Rezolvare
Sistemul se mai poate scrie sub forma $A\mathrm{x} = b$, unde:

$$A = \begin{pmatrix} 5 & -2 & 3 \\ -3 & 9 & 1 \\ 2 & -1 & -7 \end{pmatrix}, \; b = \begin{pmatrix} -1 \\ 2 \\ 3 \end{pmatrix}.$$

Matricea A se descompune în suma $L + D + U$ cu

$$L = \begin{pmatrix} 0 & 0 & 0 \\ -3 & 0 & 0 \\ 2 & -1 & 0 \end{pmatrix}, \; D = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & -7 \end{pmatrix}, \; U = \begin{pmatrix} 0 & -2 & 3 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

Verificarea condiţiei de convergenţă a algoritmului presupune calculul valorilor proprii ale matricei

$$M = -D^{-1}(L + U) = \begin{pmatrix} 0 & \frac{2}{5} & \frac{-3}{5} \\ \frac{1}{3} & 0 & \frac{-1}{9} \\ \frac{2}{7} & \frac{-1}{7} & 0 \end{pmatrix}$$

Calculând maximul în modul al valorilor proprii ale matricei M, obţinem

$$\rho(M) = 0.2673998083 < 1,$$

şi deci algoritmul converge.
Aplicăm formulele (2.32), plecând de la $x^{(0)} = 0$, $y^{(0)} = 0$, $z^{(0)} = 0$, obţinem succesiv:

$$\begin{aligned} x^{(1)} &= -0.2000000000 \\ y^{(1)} &= 0.2222222222 \\ z^{(1)} &= -0.4285714286 \\ x^{(2)} &= 0.1460317460 \\ y^{(2)} &= 0.2031746032 \\ z^{(2)} &= -0.5174603174 \\ x^{(3)} &= 0.1917460316 \\ y^{(3)} &= 0.3283950617 \\ z^{(3)} &= -0.4158730159. \end{aligned}$$

Pentru comparaţie, am rezolvat acest sistem folosind procedura `solve` furnizată de Maple, iar rezultatele sunt:

$$x = 0.1861198738, \; y = 0.3312302839, \; z = -0.4227129338.$$

2.6.3 Probleme propuse

Exerciţiul 2.6.2. Să se verifice dacă se poate aplica metoda iterativă a lui Jacobi, şi în caz afirmativ să se găsească primele 3 elemente ale şirului de soluţii parţiale. Comparaţi soluţia obţinută cu soluţia exactă:

a) $\begin{cases} x + 3y = -2 \\ 2x + y = 6 \end{cases}$

b) $\begin{cases} x + 2y + z = 1 \\ 3x - y + 5z = 14 \\ x + y - z = -2 \end{cases}$

2.6.4 Implementare

A. Algoritm

Observaţia 2.6.1. Deoarece metoda lui Jacobi este o metodă iterativă, trebuie specificată o condiţie de oprire a algoritmului. Algoritmul converge dacă şirul $(x^{(k)})$ este convergent. Convergenţa acestui şir poate fi descrisă în mod teoretic în diverse moduri. În practică, se foloseşte o variantă a criteriului lui Cauchy, şi anume: şirul $(x^{(k)})$ este convergent, dacă

$$\|x^{(k+1)} - x^{(k)}\| < \varepsilon$$

unde ε este o constantă dată.

În cazul nostru, vom considera că soluţiile sistemului au fost obţinute cu eroarea ε, adică

$$x_i^{(k+1)} \in [x_i^{(*)} - \varepsilon, x_i^{(*)} + \varepsilon].$$

De aici rezultă o condiţie de oprire a algoritmului:

$$\sqrt{\sum_{i=1}^{n}(x_i^{(k+1)} - x_i^{(k)})} < \varepsilon\sqrt{n}. \tag{2.33}$$

Date de intrare: un sistem de ecuaţii (o mulţime de ecuaţii), un punct iniţial, $x^{(0)}$, o eroare ε.

Date de ieşire: soluţia aproximativă a sistemului, obţinută în urma aplicării traiectoriei Jacobi vectorului $x^{(0)}$ până când este îndeplinită condiţia (2.33).

Algoritmul constă în următoarele etape:

1. generarea matricei A a sistemului şi a vectorului b
   - n - numărul de necunoscute (numărul de linii ale matricei A)
2. generarea matricelor L, D, U şi verificarea convergenţei metodei:

$$\rho(-D^{-1}(L + U)) < 1$$

3. construirea traiectoriei Jacobi

   repetă

$$x_i^{(k+1)} = \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij} \cdot x_j^{(k)} \right) \cdot \frac{1}{a_{ii}},\ i = \overline{1,n}$$

   până când $\sqrt{\sum_{i=1}^{n} (x_i^{(k+1)} - x_i^{(k)})} < \varepsilon\sqrt{n}$

B. Programe MAPLE şi rezultate

```
jacobi:=proc(eq::set(equation), init::vector, eps::float)
local var, n, AA, A, b, l, d, u, i, j, m, lst, xo, test, k, x;
var:=[op(indets(eq))];
n:=nops(var);
if vectdim(init)<>n then
  ERROR(‘numarul de necunoscute nu este egal cu
    dimensiunea vectorului initial‘)
fi;
AA:=genmatrix(eq, var, flag);
A:=delcols(AA,n+1..n+1);
b:=col(AA,n+1);
l:=matrix(n,n,0):
u:=matrix(n,n,0):
d:=matrix(n,n,0):
for i from 1 to n do
  for j from 1 to i-1 do
    l[i,j]:=A[i,j];
  od;
  d[i,i]:=A[i,i];
  for j from i+1 to n do
    u[i,j]:=A[i,j];
  od;
od;
# conditia de convergenta
m:=multiply(inverse(d),matadd(l,u,-1,-1));
lst:=[eigenvals(m)];
if evalf(max(seq(abs(lst[k]),k=1..nops(lst))))>=1 then
  ERROR(‘Algoritmul nu converge‘);
fi;
# algoritmul propriu-zis
for i from 1 to n do
  xo[i]:=init[i]
od;
test:=1;
```

```
while test>=evalf(eps*sqrt(n)) do
  for i from 1 to n do
    x[i]:=evalf(
          1/A[i,i]*( b[i]-sum(A[i,k]*xo[k],k=1..n)+A[i,i]*xo[i] )
          );
  od;
  test:=evalf(sqrt( sum( (x[k]-xo[k])^2, k=1..n ) ));
  for i from 1 to n do
    xo[i]:=x[i];
  od;
od;
RETURN(seq(var[i]=x[i],i=1..n));
end:

debug(jacobi):

jacobi({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.01);
```

```
{--> enter jacobi, args = {3*x+y = 5, x+2*y = 5}, array(1 ..
2,[(1)=0,(2)=0]), .1e-1
```

$$var := [x, y]$$

$$n := 2$$

$$AA := \begin{bmatrix} 3 & 1 & 5 \\ 1 & 2 & 5 \end{bmatrix}$$

$$A := \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$$

$$b := [5, 5]$$

$$l := \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$u := \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$d := \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$d_{1,1} := 3$$

$$u_{1,2} := 1$$

$$l_{2,1} := 1$$

$$d_{2,2} := 2$$

$$m := \begin{bmatrix} 0 & \frac{-1}{3} \\ \frac{-1}{2} & 0 \end{bmatrix}$$

$$lst := [\frac{\sqrt{6}}{6}, -\frac{\sqrt{6}}{6}]$$

$$xo_1 := 0$$

$$xo_2 := 0$$
$$test := 1$$
$$x_1 := 1.666666667$$
$$x_2 := 2.500000000$$
$$test := 3.004626063$$
$$xo_1 := 1.666666667$$
$$xo_2 := 2.500000000$$
$$x_1 := 0.8333333333$$
$$x_2 := 1.666666666$$
$$test := 1.178511303$$
$$xo_1 := 0.8333333333$$
$$xo_2 := 1.666666666$$
$$x_1 := 1.111111111$$
$$x_2 := 2.083333334$$
$$test := 0.5007710115$$
$$xo_1 := 1.111111111$$
$$xo_2 := 2.083333334$$
$$x_1 := 0.9722222220$$
$$x_2 := 1.944444444$$
$$test := 0.1964185512$$
$$xo_1 := 0.9722222220$$
$$xo_2 := 1.944444444$$
$$x_1 := 1.018518519$$
$$x_2 := 2.013888889$$
$$test := 0.08346183593$$
$$xo_1 := 1.018518519$$
$$xo_2 := 2.013888889$$
$$x_1 := 0.9953703703$$
$$x_2 := 1.990740740$$
$$test := 0.03273642604$$
$$xo_1 := 0.9953703703$$
$$xo_2 := 1.990740740$$
$$x_1 := 1.003086420$$
$$x_2 := 2.002314815$$
$$test := 0.01391030679$$
$$xo_1 := 1.003086420$$
$$xo_2 := 2.002314815$$

```
<-- exit jacobi (now at top level) = x = 1.003086420, y = 2.002314815}
```

$$x = 1.003086420,\ y = 2.002314815$$

Se pot compara rezultatele obţinute dacă eroarea ε se modifică:

```
> jacobi({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.01);
```

$$x = 1.003086420,\ y = 2.002314815$$

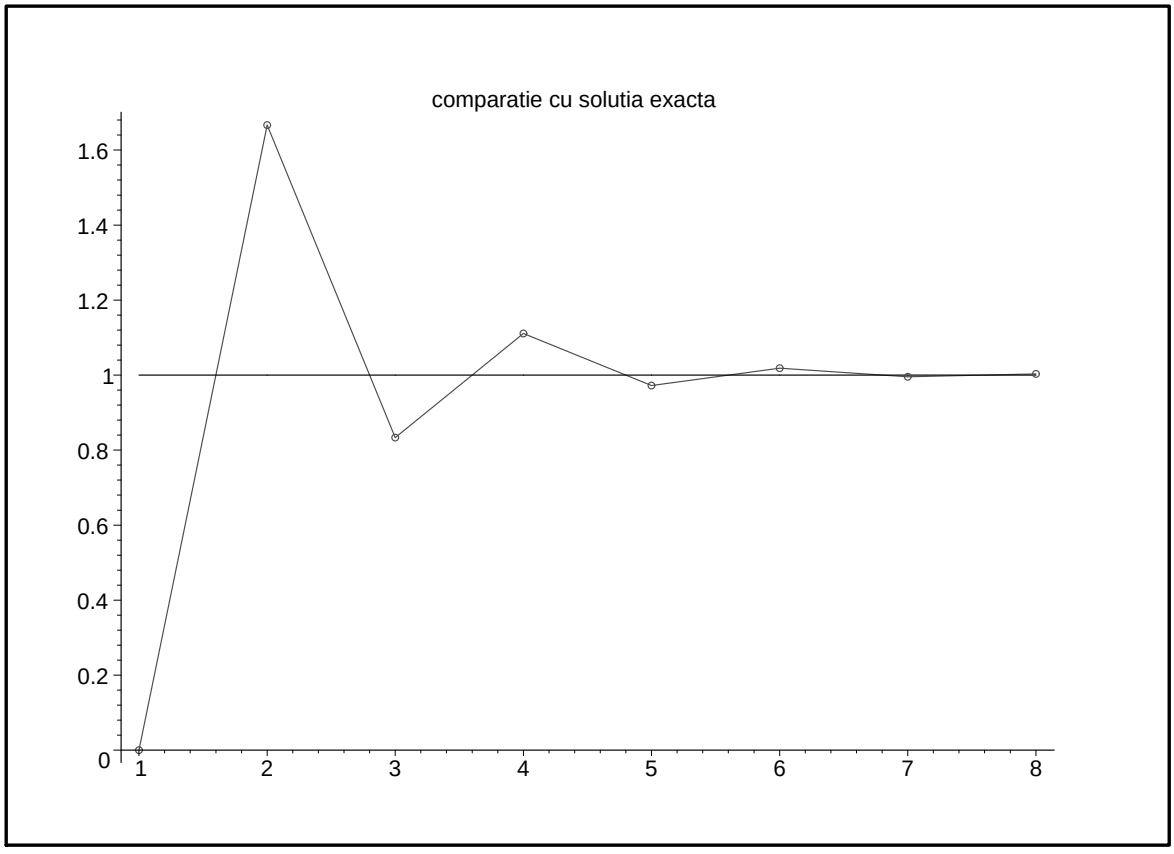
```
> jacobi({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.001);
```

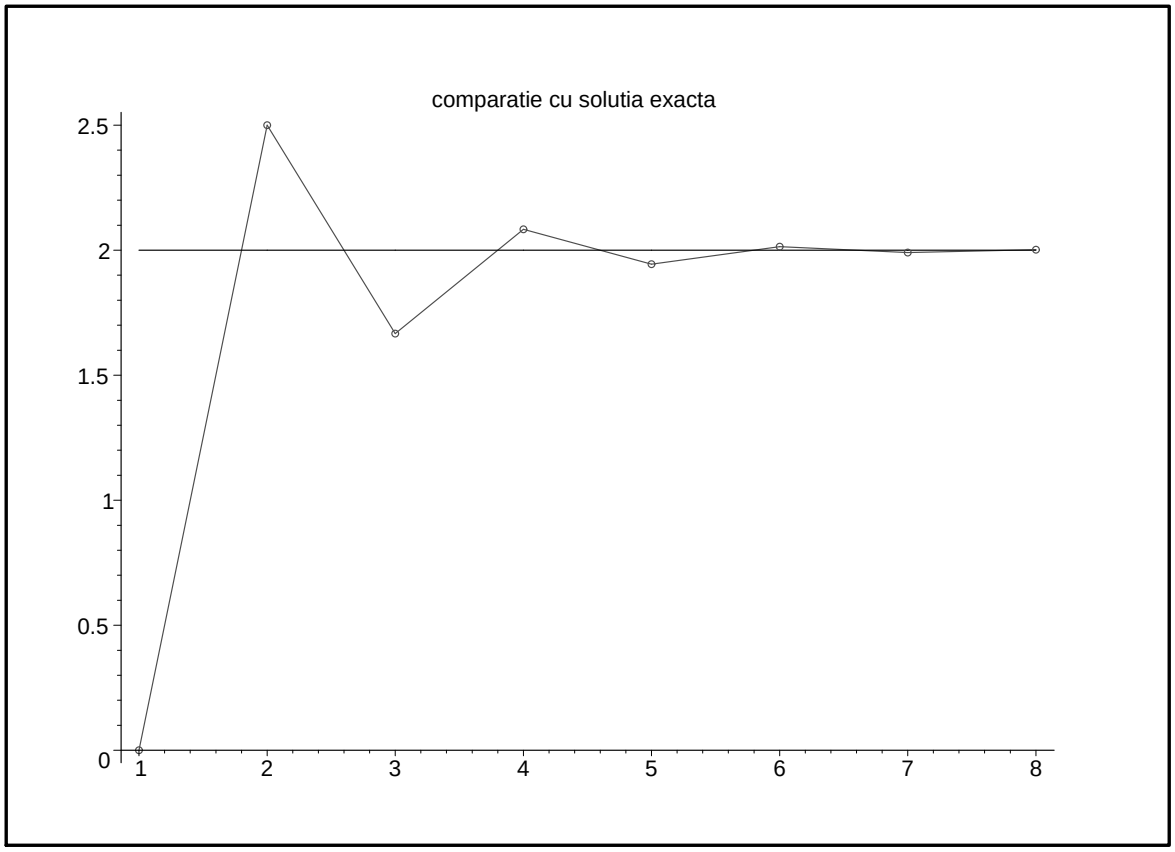
$$x = 0.9998713993,\ y = 1.999742798$$

```
> jacobi({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.00001);
```

$$x = 1.000002381,\ y = 2.000001786$$

De asemenea, se poate compara şirul soluţiilor parţiale cu soluţia exactă obţinută rezolvând sistemul $Ax = b$ cu ajutorul procedurii `linsolve`. Pentru sistemul considerat mai sus, a cărui soluţie exactă este $x = 1$, $y = 2$, obţinem următoarele grafice:





2.7 Metoda Gauss-Seidel

2.7.1 Breviar teoretic

Metoda Gauss-Seidel este o metodă de rezolvare numerică a sistemelor de tip Cramer, prin aproximaţii succesive. Matricea A a sistemului se descompune în suma $L+D+U$, unde L este matrice triunghiulară subdiagonală, D matrice diagonală şi U matrice triunghiulară supradiagonală.

Pentru un vector $x^{(0)} \in \mathbb{R}^n$, şirul de vectori $x^{(k)}$ definit prin:

$$x^{(k+1)} = (L + D)^{-1}(b - Ux^{(k)}) \tag{2.34}$$

se numeşte traiectoria Gauss-Seidel a vectorului $x^{(0)}$.

Traiectoria Gauss-Seidel a vectorului $x^{(0)}$ converge dacă şi numai dacă raza spectrală ρ a matricei

$$-(L + D)^{-1} U \tag{2.35}$$

este strict subunitară.

În caz de convergenţă, componentele $x_1^{(k+1)}$, ..., $x_n^{(k+1)}$ ale vectorului $x^{(k+1)}$, situat pe

traiectoria Gauss-Seidel a vectorului $x^{(0)}$, sunt date de relaţiile:

$$x_1^{(k+1)} = \left(b_1 - \sum_{j=2}^{n} a_{1j} \cdot x_j^{(k)} \right) \cdot \frac{1}{a_{11}} \tag{2.36}$$

$$x_i^{(k+1)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} \cdot x_j^{(k)} \right) \cdot \frac{1}{a_{ii}},\ i = 2, \ldots, n. \tag{2.37}$$

2.7.2 Problemă rezolvată

Exerciţiul 2.7.1. Să se determine primele 3 puncte de pe traiectoria Gauss-Seidel a vectorului $(0, 0)^T$ pentru sistemul următor:

$$\begin{cases} 4x + y = -1 \\ 4x + 3y = -2. \end{cases}$$

Rezolvare

Sistemul se poate scrie sub forma $A\mathrm{x} = b$, unde

$$A = \begin{pmatrix} 4 & 1 \\ 4 & 3 \end{pmatrix},\ b = \begin{pmatrix} -1 \\ -2 \end{pmatrix}$$

iar matricea A se descompune în suma $L + D + U$, după cum urmează:

$$L = \begin{pmatrix} 0 & 0 \\ 4 & 0 \end{pmatrix},\ D = \begin{pmatrix} 4 & 0 \\ 0 & 3 \end{pmatrix},\ U = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

Algoritmul converge dacă raza spectrală a matricei

$$M = -(L + D)^{-1}U = \begin{pmatrix} 0 & -\frac{1}{4} \\ -\frac{4}{3} & 0 \end{pmatrix}$$

este strict subunitară. Efectuând calculele, obţinem

$$\rho(M) = \frac{\sqrt{3}}{3} < 1$$

şi deci algoritmul este convergent.
În continuare, aplicăm formulele (2.36)-(2.37) şi, plecând de la punctele $x^{(0)} = 0$, $y^{(0)} = 0$, obţinem:

$$x^{(1)} = -0.2500000000$$
$$y^{(1)} = -0.3333333333$$
$$x^{(2)} = -0.1666666667$$
$$y^{(2)} = -0.4444444443$$
$$x^{(3)} = -0.1388888889$$
$$y^{(3)} = -0.4814814813$$

Pentru comparaţie, determinăm soluţia exactă a sistemului considerat:

$$x = -\frac{1}{8},\ y = -\frac{1}{2}$$

adică $x = -0.125$, $y = -0.5$.

2.7.3 Probleme propuse

Exerciţiul 2.7.2. Să se verifice dacă se poate aplica metoda iterativă a lui Gauss-Seidel, şi în caz afirmativ să se găsească primele 3 elemente ale şirului de soluţii parţiale. Comparaţi cu soluţia exactă şi cu şirul de soluţii parţiale obţinut prin metoda lui Jacobi:

a) $\begin{cases} x + 3y = -2 \\ 2x + y = 6 \end{cases}$

b) $\begin{cases} x + 2y + z = 1 \\ 3x - y + 5z = 14 \\ x + y - z = -2 \end{cases}$

2.7.4 Implementare

A. Algoritm

Observaţia 2.7.1. Deoarece metoda Gauss-Seidel este o metodă iterativă, trebuie specificată o condiţie de oprire a algoritmului. În continuare vom folosi aceeaşi condiţie de oprire a algoritmului ca şi cea prezentată în paragraful 2.6.4:

$$\sqrt{\sum_{i=1}^{n}(x_i^{(k+1)} - x_i^{(k)})} < \varepsilon\sqrt{n}. \tag{2.38}$$

Date de intrare: un sistem de ecuaţii (o mulţime de ecuaţii), un punct iniţial, $x^{(0)}$, o eroare ε.

Date de ieşire: soluţia aproximativă a sistemului, obţinută în urma aplicării traiectoriei Gauss-Seidel vectorului $x^{(0)}$ până când este îndeplinită condiţia (2.38).

Algoritmul constă în următoarele etape:

1. generarea matricei A a sistemului şi a vectorului b
   - n - numărul de necunoscute (numărul de linii ale matricei A)
2. generarea matricelor L, D, U şi verificarea convergenţei metodei:
   $$\rho(-(L + D)^{-1} U) < 1$$
3. construcţia traiectoriei Gauss-Seidel

   repetă

$$x_1^{(k+1)} = \left(b_1 - \sum_{j=2}^{n} a_{1j} \cdot x_j^{(k)}\right) \cdot \frac{1}{a_{11}}$$
$$x_i^{(k+1)} = \left(b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} \cdot x_j^{(k)}\right) \cdot \frac{1}{a_{ii}},\ i = \overline{2, n}.$$

   până când $\sqrt{\sum_{i=1}^{n}(x_i^{(k+1)} - x_i^{(k)})} < \varepsilon\sqrt{n}$

B. Programe MAPLE şi rezultate

Comparativ, prezentăm rezultatele obţinute cu ajutorul metodei Gauss-Seidel pentru acelaşi sistem şi vector iniţial al traiectoriei, dar pentru diferite valori ale erorii:

```
> gseidel({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.01);
```

$x = 1.000514403,\ y = 1.999742798$

```
> gseidel({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.001);
```

$x = 1.000085734,\ y = 1.999957133$

```
> gseidel({3*x+y=5, x+2*y=5}, vector(2,[0,0]),0.00001);
```

$x = 1.000002381,\ y = 1.999998810$

2.8 Metoda relaxării succesive

2.8.1 Breviar teoretic

Metoda relaxării succesive este o metodă de rezolvare numerică a sistemelor de tip Cramer, prin aproximaţii succesive. Această metodă se deosebeşte de metoda Gauss-Seidel prin aceea că se introduc corecţiile

$$\Delta x^{(k)} = x^{(k+1)} - x^{(k)},\ k = 0, 1, 2, ... \tag{2.39}$$

Matricea A a sistemului se descompune în suma $L + D + U$, unde L este matrice triunghiulară subdiagonală, D matrice diagonală şi U matrice triunghiulară supradiagonală.

Pentru un vector $x^{(0)} \in \mathbb{R}^n$, şirul de vectori $x^{(k)}$ definit prin:

$$x^{(k+1)} = \left(L + \frac{1}{\omega} D\right)^{-1} \left\{b - \left[\left(1 - \frac{1}{\omega}\right) D + U\right] x^{(k)}\right\},\ k = 0, 1, 2, \ldots \tag{2.40}$$

se numeşte traiectoria vectorului $x^{(0)}$ obţinută prin relaxări succesive.

Traiectoria vectorului $x^{(0)}$ obţinută prin relaxări succesive converge dacă şi numai dacă raza spectrală ρ a matricei

$$-\left(L + \frac{1}{\omega} D\right)^{-1} \left[\left(1 - \frac{1}{\omega}\right) D + U\right] \tag{2.41}$$

este strict subunitară.

În caz de convergenţă, componentele $x_1^{(k+1)}$, ..., $x_n^{(k+1)}$ ale vectorului $x^{(k+1)}$ situat pe traiectoria vectorului $x^{(0)}$ obţinută prin relaxări succesive sunt date de relaţiile:

$$x_1^{(k+1)} = (1 - \omega) \cdot x_1^{(k)} + \frac{\omega}{a_{11}} \left[b_1 - \sum_{j=1}^{n} a_{1j} \cdot x_j^{(k)}\right] \tag{2.42}$$

$$x_i^{(k+1)} = (1 - \omega) \cdot x_i^{(k)} + \frac{\omega}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij} \cdot x_j^{(k)}\right],\ i = 2, \ldots, n. \tag{2.43}$$

Observaţia 2.8.1. Metoda Gauss-Seidel este un caz particular al metodei relaxării succesive, pentru care $\omega = 1$.

2.8.2 Problemă rezolvată

Exerciţiul 2.8.1. Să se găsească primele 3 elemente ale traiectoriei vectorului $(0,0)^T$ folosind metoda relaxării succesive cu $\omega = 0.5$, pentru sistemul:

$$\begin{cases} 4x + y = -1 \\ 4x + 3y = -2. \end{cases}$$

Rezolvare

Sistemul se poate scrie sub forma $A\mathrm{x} = b$, unde

$$A = \begin{pmatrix} 4 & 1 \\ 4 & 3 \end{pmatrix}\ ,\ b = \begin{pmatrix} -1 \\ -2 \end{pmatrix}.$$

Matricea A se descompune în suma $L + D + U$, cu

$$L = \begin{pmatrix} 0 & 0 \\ 4 & 0 \end{pmatrix}\ ,\ D = \begin{pmatrix} 4 & 0 \\ 0 & 3 \end{pmatrix}\ ,\ U = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

Algoritmul converge dacă raza spectrală a matricei

$$M = -\left(L + \frac{1}{\omega}\,D\right)^{-1}\left[\left(1 - \frac{1}{\omega}\right)\,D + U\right] = \begin{pmatrix} 0.500 & -0.125 \\ 0.333 & 0.583 \end{pmatrix}$$

este strict subunitară. Efectuând calculele, obţinem

$$\rho(M) = 0.75 < 1$$

şi deci algoritmul este convergent.
În continuare, aplicăm formulele (2.42)-(2.43), plecând de la punctele $x^{(0)} = 0$, $y^{(0)} = 0$, şi obţinem:

$$\begin{aligned} x^{(1)} &= -0.1250000000 \\ y^{(1)} &= -0.5000000000 \\ x^{(2)} &= -0.1250000000 \\ y^{(2)} &= -0.7500000000 \\ x^{(3)} &= -0.0937500000 \\ y^{(3)} &= -0.9166666667 \end{aligned}$$

Pentru comparaţie, determinăm soluţia exactă a sistemului considerat:

$$x = -0.125\ ,\ y = -0.5.$$

2.8.3 Probleme propuse

Exerciţiul 2.8.2. Să se verifice dacă se poate aplica metoda relaxării succesive, şi în caz afirmativ să se găsească primele 3 elemente ale şirului de soluţii parţiale, folosind o subrelaxare şi o suprarelaxare. Comparaţi rezultatele obţinute cu soluţia exactă:

a) $\begin{cases} x + 3y = -2 \\ 2x + y = 6 \end{cases}$

b) $\begin{cases} x + 2y + z = 1 \\ 3x - y + 5z = 14 \\ x + y - z = -2 \end{cases}$

2.8.4 Implementare

A. Algoritm

Observaţia 2.8.2. Deoarece metoda relaxării succesive este o metodă iterativă, trebuie specificată o condiţie de oprire a algoritmului. Această condiţie este similară cu cea folosită în paragrafele anterioare, şi anume:

$$\sqrt{\sum_{i=1}^{n}(x_i^{(k+1)} - x_i^{(k)})} < \varepsilon\sqrt{n}. \tag{2.44}$$

Date de intrare: un sistem de ecuaţii (o mulţime de ecuaţii), un punct iniţial, $x^{(0)}$, o relaxare, ω, o eroare ε.

Date de ieşire: soluţia aproximativă a sistemului, obţinută în urma aplicării traiectoriei vectorului $x^{(0)}$ obţinută prin relaxări succesive până când este îndeplinită condiţia (2.44).

Algoritmul constă în următoarele etape:

1. generarea matricei A a sistemului şi a vectorului b
   - n - numărul de necunoscute (numărul de linii ale matricei A)
2. generarea matricelor L, D, U şi verificarea convergenţei metodei:

$$\rho\left(-\left(L+\frac{1}{\omega}D\right)^{-1}\left[\left(1-\frac{1}{\omega}\right)D+U\right]\right)<1$$

   şi

$$0<\omega<2$$

3. construcţia traiectoriei relaxărilor succesive

   repetă

$$x_1^{(k+1)} = (1-\omega)\cdot x_1^{(k)} + \frac{\omega}{a_{11}}\left[b_1 - \sum_{j=1}^{n} a_{1j}\cdot x_j^{(k)}\right] \tag{2.45}$$

$$x_i^{(k+1)} = (1-\omega)\cdot x_i^{(k)} + \frac{\omega}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}\cdot x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij}\cdot x_j^{(k)}\right],\ i=\overline{2,n} \tag{2.46}$$

   până când $\sqrt{\sum_{i=1}^{n}(x_i^{(k+1)} - x_i^{(k)})} < \varepsilon\sqrt{n}$

B. Programe MAPLE şi rezultate

Prezentăm comparativ rezultatele aplicării metodei relaxării succesive pentru diferite valori ale lui ω:

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 0.99, 0.0001);
```

$x = 0.9919288541,\ y = 2.024277742$

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 0.9, 0.0001);
```

$x = 0.9091141587,\ y = 2.272710330$

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 1.01, 0.0001);
```

$x = 1.007912178,\ y = 1.976281288$

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 1.1, 0.0001);
```

$x = 1.071425885,\ y = 1.785716899$

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 0.7, 0.0001);
```

$x = 0.6250722364,\ y = 3.124921273$

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 1.3, 0.0001);
```

$x = 1.176464330,\ y = 1.470600344$

```
> relsucc({3*x+y=5, x+2*y=5}, vector(2,[0,0]), 1, 0.0001); #
> gauss-seidel
```

$x = 1.000014289,\ y = 1.999992856$

Capitolul 3

Rezolvarea ecuaţiilor şi a sistemelor de ecuaţii neliniare

Fie sistemul neliniar

$$F(x) = 0 \tag{3.1}$$

unde $F(x)$ este vectorul $(f_1(x), ..., f_n(x))^T$, funcţiile $f_1, ..., f_n : D \in \mathbb{R}^n \to \mathbb{R}^1$ sunt considerate cunoscute, iar vectorul $x = (x_1, ..., x_n)^T$ este necunoscut.

3.1 Metoda punctului fix

3.1.1 Breviar teoretic

Soluţiile sistemului neliniar (3.1) se caută printre punctele fixe $x^{(*)}$, adică printre soluţiile sistemului

$$G(x) = x \tag{3.2}$$

obţinut din sistemul iniţial prin alegerea

$$G(x) = x - [F'(x)]^{-1} \cdot F(x) \tag{3.3}$$

unde $F'(x)$ este matricea Jacobi asociată vectorului F, matrice despre care s-a presupus că este continuă şi inversabilă.

Presupunem că operatorul G are un punct fix $x^{(*)}$.

Definiţia 3.1.1. *Vom spune că $x^{(*)}$ este un punct de atracţie dacă există o sferă deschisă $S(x^{(*)}, r) = \{x \in \mathbb{R}^n \mid \|x - x^{(*)}\| < r\}$ cu următoarele proprietăţi:*

1. *$S(x^{(*)}, r) \subset D$ şi $x^{(k)}$ generat de*

$$x^{(k+1)} = G(x^{(k)}) \tag{3.4}$$

   este un şir bine definit pentru $\forall\, x^{(0)} \in S(x^{(*)}, r)$;

2. *$\forall\, x^{(0)} \in S(x^{(*)}, r)$ şirul $x^{(k)}$ definit de (3.4) aparţine lui D şi $x^{(k)} \xrightarrow[k\to\infty]{} x^{(*)}$.*

Teorema 3.1.1. *Fie $G : D \subset \mathbb{R}^n \to \mathbb{R}^n$ un operator neliniar şi $x^{(*)} \in D$ un punct fix al lui G. Dacă G este de clasă $\mathcal{C}^1$ pe D şi raza spectrală ρ a matricei Jacobi a lui G în $x^{(*)}$ este strict subunitară ($\rho < 1$), atunci $x^{(*)}$ este un punct de atracţie şi*

$$\limsup_{k\to\infty} \|x^{(k)} - x^{(*)}\|^{1/k} = \rho.$$

Teorema 3.1.2. *Fie $G : D \subset \mathbb{R}^n \to \mathbb{R}^n$ un operator neliniar şi $x^{(*)} \in D$ un punct fix al lui G. Dacă G este de clasă $\mathcal{C}^1$ pe D şi norma μ a matricei Jacobi a lui G în $x^{(*)}$ este strict subunitară ($\mu < 1$), atunci $x^{(*)}$ este punct de atracţie şi*

$$\limsup_{k\to\infty} \|x^{(k)} - x^{(*)}\|^{1/k} = \mu.$$

Teorema 3.1.3. *(Unicitatea punctului fix) Fie $G : D \subset \mathbb{R}^n \to \mathbb{R}^n$ un operator neliniar. Dacă G este de clasă $\mathcal{C}^1$ pe D, şi dacă norma μ a matricei Jacobi asociată operatorului G este strict subunitară ($\mu < 1$) pentru orice $x \in D$, atunci pentru orice x^0 şirul de aproximaţii succesive*

$$x^{(k+1)} = G(x^k), \quad k = 0, 1, 2, \ldots \qquad x^{(0)} \in D \tag{3.5}$$

converge la un unic punct fix $x^{(*)} \in D$.

Observaţia 3.1.1. Norma unei matrice este dată de:

$$\|A\| = \max_{\|x\|\neq 0} \frac{\|Ax\|}{\|x\|} \tag{3.6}$$

3.1.2 Problemă rezolvată

Exerciţiul 3.1.1. Să se găsească primii trei termeni ai şirului de aproximaţii succesive folosind metoda punctului fix, pentru sistemul următor:

$$\begin{cases} \dfrac{x_1^2 + x_2}{6} - x_1 = 0 \quad \text{pe domeniul} \quad D = [0,1] \times [1,2] \\ \dfrac{x_1 + x_2^2}{8} - x_2 = 0 \end{cases}$$

Rezolvare

Sistemul se mai scrie $G(x) = x$, unde

$$G = \left(\frac{x_1^2 + x_2}{6}, \frac{x_1 + x_2^2}{8} \right)^T, \; x = (x_1, x_2)^T$$

Calculăm jacobianul lui G, şi obţinem:

$$G' = \begin{pmatrix} \frac{x_1}{3} & \frac{1}{6} \\ \frac{1}{8} & \frac{x_2}{4} \end{pmatrix}$$

a cărui normă este:

$$\max \left(\frac{1}{8}, \frac{1}{4} \, |x_2| \, , \, \frac{1}{6}, \frac{1}{3} \, |x_1| \right)$$

Deoarece $x_1 \in [0,1]$ şi $x_2 \in [1,2]$, rezultă că norma matricei G' este $\frac{2}{3} < 1$, şi deci pentru orice punct iniţial $x^{(0)} \in D$, algoritmul converge.

Fie $x^{(0)} = (0.5, 0.5)^T$. Aplicând formula (3.5), obţinem valorile pentru primii trei termeni ai şirului de aproximaţii succesive:

$$x_1^{(1)} = 0.1250000000$$
$$x_2^{(1)} = 0.0468750000$$
$$x_1^{(2)} = 0.0104166666$$
$$x_2^{(2)} = 0.0015767415$$
$$x_1^{(3)} = 0.0002808747$$
$$x_2^{(3)} = 0.0000354201$$

Pentru comparaţie, determinăm soluţia exactă a sistemului: $x_1^{(*)} = 0$, $x_2^{(*)} = 0$.

3.1.3 Probleme propuse

Exerciţiul 3.1.2. Să se găsească o condiţie în care se poate aplica metoda punctului fix asupra sistemului următor, precum şi primii 3 termeni ai şirului de aproximaţii succesive ale soluţiei:

$$\begin{cases} x^2 + y^2 - 1 = 0 \\ x^3 - y = 0 \end{cases}$$

3.1.4 Implementare

A. Algoritmi

Date de intrare: un sistem de ecuaţii (dat ca mulţime de ecuaţii), *eqn*, un punct iniţial, x_0 şi o eroare ε

Date de ieşire: soluţia sistemului *eqn* obţinută prin metoda punctului fix, plecând de la punctul iniţial x_0, cu eroarea ε.

Algoritmul constă în următorii paşi:

1. generarea matricei G şi verificarea condiţiei ca norma matricei G' să fie strict subunitară

2. găsirea soluţiei aproximative

   repetă

   $x^{(k+1)} = G(x^k)$

   până când $\max\limits_{i=\overline{1,n}} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon$

B. Programe MAPLE şi rezultate

```
fixedpoint:=proc(eqn::set(equation), x0::vector, eps::float)
local x, n, g, i, j, jac, yo, test, k, y, jac1, eig;
if nops(eqn) <> vectdim(x0) then
  ERROR(‘problema nu este bine pusa!‘)
fi;
n:=nops(eqn);
x:=[op(indets(eqn, name))];
if nops(x) <> n then
  ERROR(‘numarul de ecuatii nu coincide cu numarul de
    necunoscute!‘)
fi;
g:=vector(n,0);
for i from 1 to n do g[i]:=x[i]+lhs(eqn[i])-rhs(eqn[i]); od;
evalm(g);
jac:=Matrix(n,n,0);
for i from 1 to n do for j from 1 to n do
jac[i,j]:=jacobian(g,x)[i,j]; od;od;
jac1:=[];
for i from 1 to n do
  for j from 1 to n do
    jac1:=[op(jac1),jacobian(g,x)[i,j]];
  od;
od;
eig:=max(op(jac1));
WARNING(‘punctul de plecare trebuie ales astfel incat expresia
  \%1 sa fie strict subunitara‘,eig);
yo:=x0;
test:=1; k:=1;
while test>=eps and k<50 do
  for i from 1 to n do
    test:=0;
    y[i]:=evalf(subs(seq(x[k]=yo[k],k=1..n),g[i]));
    if abs(y[i]-yo[i])>test then test:=abs(y[i]-yo[i]); fi;
    yo[i]:=y[i];
    if test>10^10 then ERROR(‘Algoritmul nu converge!‘);fi;
  od;
  k:=k+1;
od;
if k>=50 then
  ERROR(‘sunt necesare mai mult de 50 de iteratii pentru gasirea
    solutiei‘)
fi;
RETURN(seq(x[k]=y[k],k=1..n));
end:

debug(fixedpoint):
```

```
fixedpoint({(x^2+y)/6-x=0, (x+y^2)/8-y=0},vector(2,[0.5,0.5]),
  0.0001);
```

```
{--> enter fixedpoint, args = {1/6*x^2+1/6*y-x = 0, 1/8*x+1/8*y^2-y =
0}, array(1 .. 2,[(1)=.5,(2)=.5]), .1e-3
```

$$n := 2$$

$$x := [x,\ y]$$

$$g := [0,\ 0]$$

$$g_1 := \frac{x^2}{6} + \frac{y}{6}$$

$$g_2 := \frac{x}{8} + \frac{y^2}{8}$$

$$\left[\frac{x^2}{6} + \frac{y}{6},\ \frac{x}{8} + \frac{y^2}{8}\right]$$

$$jac := \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$jac_{1,\,1} := \frac{x}{3}$$

$$jac_{1,\,2} := \frac{1}{6}$$

$$jac_{2,\,1} := \frac{1}{8}$$

$$jac_{2,\,2} := \frac{y}{4}$$

$$jac1 := []$$

$$jac1 := [\frac{x}{3}]$$

$$jac1 := [\frac{x}{3},\ \frac{1}{6}]$$

$$jac1 := [\frac{x}{3},\ \frac{1}{6},\ \frac{1}{8}]$$

$$jac1 := [\frac{x}{3},\ \frac{1}{6},\ \frac{1}{8},\ \frac{y}{4}]$$

$$eig := \max(\frac{1}{6},\ \frac{x}{3},\ \frac{y}{4})$$

```
Warning, punctul de plecare trebuie ales astfel incat expresia
max(1/6,1/3*x,1/4*y) sa fie strict subunitara
```

$$yo := [0.5,\ 0.5]$$

$$test := 1$$

$$k := 1$$

$$test := 0$$

$$y_1 := 0.1250000000$$

$$test := 0.3750000000$$

$$yo_1 := 0.1250000000$$

$$test := 0$$

$$y_2 := 0.04687500000$$
$$test := 0.4531250000$$
$$yo_2 := 0.04687500000$$
$$k := 2$$
$$test := 0$$
$$y_1 := 0.01041666667$$
$$test := 0.1145833333$$
$$yo_1 := 0.01041666667$$
$$test := 0$$
$$y_2 := 0.001576741537$$
$$test := 0.04529825846$$
$$yo_2 := 0.001576741537$$
$$k := 3$$
$$test := 0$$
$$y_1 := 0.0002808747470$$
$$test := 0.01013579192$$
$$yo_1 := 0.0002808747470$$
$$test := 0$$
$$y_2 := 0.00003542010761$$
$$test := 0.001541321429$$
$$yo_2 := 0.00003542010761$$
$$k := 4$$
$$test := 0$$
$$y_1 := 0.5916499705\,10^{-5}$$
$$test := 0.0002749582473$$
$$yo_1 := 0.5916499705\,10^{-5}$$
$$test := 0$$
$$y_2 := 0.7397192861\,10^{-6}$$
$$test := 0.00003468038832$$
$$yo_2 := 0.7397192861\,10^{-6}$$
$$k := 5$$

```
<-- exit fixedpoint (now at top level) = x = .5916499705e-5, y =
.7397192861e-6}
```

$$x = 0.5916499705\,10^{-5},\ y = 0.7397192861\,10^{-6}$$

3.2 Metoda lui Newton

3.2.1 Breviar teoretic

Soluţia $x^{(*)}$ a sistemului neliniar (3.1) este dată de limita şirului $x^{(k)}$, unde:

$$x^{(k+1)} = x^{(k)} - [F'(x^{(k)})]^{-1} \cdot F(x^{(k)}), \qquad k = 0, 1, 2, \ldots \tag{3.7}$$

în cazul şirului de iteraţii succesive clasic al lui Newton, respectiv

$$x^{(k+1)} = x^{(k)} - [F'(x^{(0)})]^{-1} \cdot F(x^{(k)}), \qquad k = 0, 1, 2, \ldots \tag{3.8}$$

în cazul şirului de iteraţii succesive simplificat al lui Newton.

Teorema 3.2.1. *Fie $F : D \subset \mathbb{R}^n \to \mathbb{R}^n$ şi ecuaţia $F(x) = 0$, despre care presupunem că are o soluţie $x^{(*)} \in D$. Dacă există o sferă deschisă $S(x^{(*)}, r) = \{x \in \mathbb{R}^n \mid \|x - x^{(*)}\| < r\}$ pe care F este de clasă $\mathcal{C}^1$ şi $F'(x^{(*)})$ este nesingulară atunci, în cazul metodei lui Newton clasice, $x^{(*)}$ este un punct de atracţie.*

Teorema 3.2.2. *În condiţiile teoremei precedente, dacă raza spectrală ρ a matricei*

$$I - [F'(x^{(0)})]^{-1} \cdot F'(x^{(*)})$$

este strict subunitară atunci, în cazul metodei lui Newton simplificată, $x^{(*)}$ este un punct fix atractiv.

Un caz particular al metodei lui Newton este acela pentru care $n = 1$. În acest caz ecuaţia (3.1) devine

$$f(x) = 0, \tag{3.9}$$

şirul de iteraţii succesive clasic al lui Newton se scrie

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}, \qquad k = 0, 1, 2, \ldots \tag{3.10}$$

iar şirul de iteraţii succesive simplificat al lui Newton se scrie

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(0)})}, \qquad k = 0, 1, 2, \ldots \tag{3.11}$$

3.2.2 Probleme rezolvate

Exerciţiul 3.2.1. Să se găsească primii 3 termeni ai şirului de iteraţii succesive (clasic şi simplificat) al lui Newton, cu $x^{(0)} = (0.7, 0.4)^T$ pentru sistemul:

$$\begin{cases} x^2 + y^2 - 1 = 0 \\ x^3 - y = 0. \end{cases}$$

Rezolvare

Sistemul se mai scrie $F(\mathrm{x}) = 0$, unde

$$F(\mathrm{x}) = (x^2 + y^2 - 1,\ x^3 - y)^T\ ,\ \mathrm{x} = (x, y)^T.$$

Jacobianul operatorului F este:

$$F'(\mathrm{x}) = \begin{pmatrix} 2x & 2y \\ 3x^2 & -1 \end{pmatrix}$$

A. Rezolvare folosind metoda lui Newton clasică

Determinăm produsul:

$$[F'(\mathrm{x})]^{-1} F(\mathrm{x}) = \left(\frac{(x^2 + y^2 - 1)}{2x(1 + 3yx)} + \frac{y(x^3 - y)}{x(1 + 3yx)}, \frac{3x(x^2 + y^2 - 1)}{2 + 6yx} - \frac{(x^3 - y)}{1 + 3yx} \right)^T$$

şi aplicăm formula (3.7), considerând

$$x^{(0)} = 0.7$$
$$y^{(0)} = 0.4.$$

Astfel, obţinem primii 3 termeni ai şirului de iteraţii succesive clasic al lui Newton:

$$x^{(1)} = 0.8535714287$$
$$y^{(1)} = 0.5800885499$$
$$x^{(2)} = 0.8267993467$$
$$y^{(2)} = 0.5637617094$$
$$x^{(3)} = 0.8260319270$$
$$y^{(3)} = 0.5636241719.$$

B. Rezolvare folosind metoda lui Newton simplificată

Aplicând formula (3.8), obţinem primii 3 termeni ai şirului de iteraţii succesive simplificat al lui Newton:

$$x^{(1)} = 0.8535714286$$
$$y^{(1)} = 0.5841767661$$
$$x^{(2)} = 0.8147421666$$
$$y^{(2)} = 0.5577269491$$
$$x^{(3)} = 0.8297474476$$
$$y^{(3)} = 0.5653470981.$$

Exerciţiul 3.2.2. Să se aplice metoda lui Newton (clasică şi simplificată), pentru aflarea soluţiei ecuaţiei

$$\sin(x) - x = 0,$$

plecând de la punctul $x^{(0)} = 0.2$.

Rezolvare

Se observă că o soluţie a ecuaţiei este $x^{(*)} = 0$.
Dacă notăm

$$f(x) = \sin(x) - x,$$

atunci $f'(x) = \cos(x) - 1$.

Plecând de la $x^{(0)} = 0.2$ şi aplicând formula (3.10), obţinem:

$$x^{(1)} = 0.1332443177$$
$$x^{(2)} = 0.0888032392$$
$$x^{(3)} = 0.0591943762$$
$$x^{(4)} = 0.0394606157$$
$$x^{(5)} = 0.0263064006 \text{ etc.}$$

După 13 iteraţii se obţine o soluţie aproximativă a ecuaţiei, cu eroarea $\varepsilon = 0.001$:

$$x^{(13)} = 0.001539688244.$$

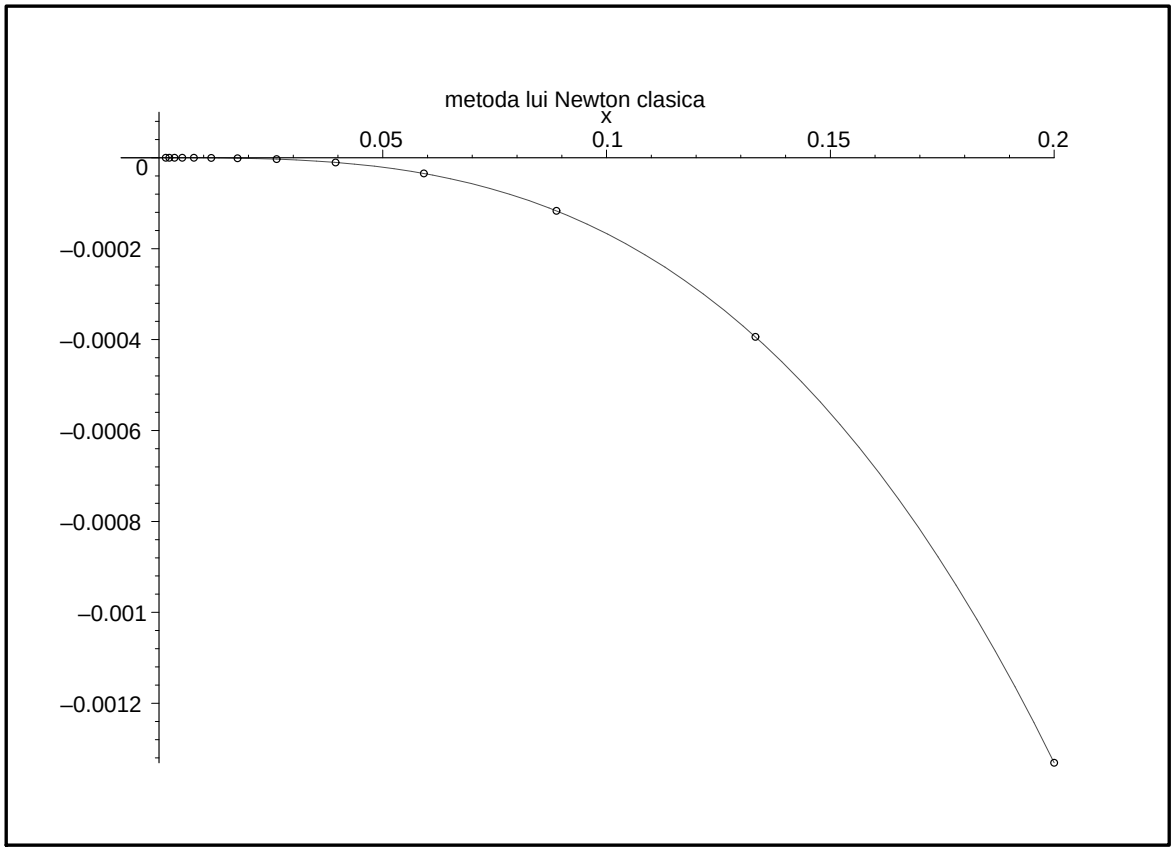
Aplicând iteraţia dată de formula (3.11), obţinem:

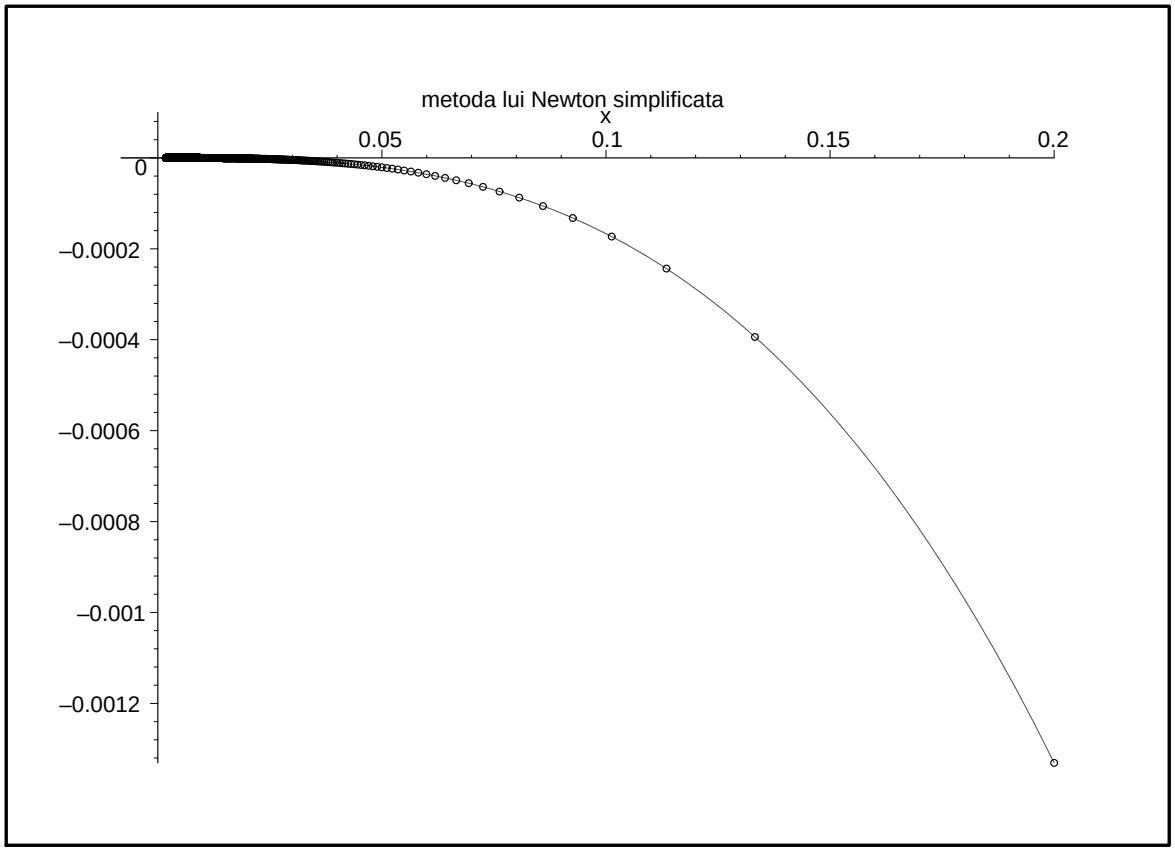
$$x^{(1)} = 0.1332443177$$
$$x^{(2)} = 0.1134824727$$
$$x^{(3)} = 0.1012708415$$
$$x^{(4)} = 0.0925912882$$
$$x^{(5)} = 0.0859570495 \text{ etc.}$$

După 18147 iteraţii se obţine o soluţie aproximativă a ecuaţiei, cu eroarea $\varepsilon = 0.00000005$:

$$x^{(18147)} = 0.001814907756.$$

În continuare, pentru ecuaţia considerată, prezentăm comparativ şirurile aproximaţiilor succesive clasică şi respectiv simplificată ale lui Newton, raportate la graficul funcţiei f.





3.2.3 Probleme propuse

Exerciţiul 3.2.3. Pornind de la punctul $(x_0, y_0) = (0.5, 0.5)$, să se găsească primii 3 termeni ai şirului de soluţii parţiale, folosind metoda lui Newton clasică şi metoda lui Newton simplificată:

$$\begin{cases} \dfrac{x^2+y}{6} - x = 0 \\ \dfrac{x+y^2}{8} - y = 0. \end{cases}$$

Exerciţiul 3.2.4. Pornind de la punctul $x_0 = 0.5$, să se găsească primii 3 termeni ai şirului de soluţii parţiale, folosind cele două variante ale metodei lui Newton:

$$x^3 + x = 0$$

3.2.4 Implementare

A. Algoritmi

A1. Cazul n-dimensional

Date de intrare: un sistem de ecuaţii eqn, un punct iniţial, x_0 şi o eroare ε

Date de ieşire: soluţia aproximativă a sistemului, obţinută folosind metoda lui Nexton clasică, respectiv metoda lui Newton simplificată

Algoritmul constă în următorii paşi:

1. generarea funcţiei F

2. a) metoda lui Newton clasică

   repetă

   $$x^{(k+1)} = x^{(k)} - [F'(x^{(k)})]^{-1} \cdot F(x^{(k)})$$

   până când $\max\limits_{i=\overline{1,n}} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon$

   b) metoda lui Newton simplificată

   repetă

   $$x^{(k+1)} = x^{(k)} - [F'(x^{(0)})]^{-1} \cdot F(x^{(k)})$$

   până când $\max\limits_{i=\overline{1,n}} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon$

A2. Cazul unidimensional

Date de intrare: o ecuaţie eqn, un punct iniţial, x_0 şi o eroare ε

Date de ieşire: soluţia aproximativă a ecuaţiei, obţinută folosind metoda lui Nexton clasică, respectiv metoda lui Newton simplificată

Algoritmul constă în următorii paşi:

1. generarea funcţiei f

2. a) metoda lui Newton clasică

   repetă

   $$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

   până când $|x^{(k+1)} - x^{(k)}| < \varepsilon$

   b) metoda lui Newton simplificată

   repetă

   $$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(0)})}$$

   până când $|x^{(k+1)} - x^{(k)}| < \varepsilon$

B. Programe MAPLE şi rezultate

Pentru metoda lui Newton clasică, respectiv simplificată în n dimensiuni, am construit procedurile `cnewton`, respectiv `snewton`.

```
cnewton:=proc(eqn::set(equation), x0::vector, eps::float)
local n,x,y,i,j,f,jc,y0,c,jc0,test;
n:=nops(eqn);
f:=vector(n,0);
y0:=vector(n,[seq(x0[i]+1,i=1..n)]);
```

```
x:=[op(indets(eqn, name))];
y:=x0;
for i from 1 to n do f[i]:=lhs(eqn[i])-rhs(eqn[i]); od;
jc:=evalf(multiply(inverse(jacobian(f,x)),f));
c:=1; test:=max(seq(abs(y0[i]-y[i]), i=1..n));
while test>eps do
  if c>1000 then ERROR(‘algoritmul nu converge‘) fi;
  for i from 1 to n do
    y0[i]:=y[i];
    jc0[i]:=evalf(subs(seq(x[k]=y[k],k=1..n),jc[i]));
    y[i]:=y[i]-jc0[i];
  od;
  test:=max(seq(abs(jc0[i]), i=1..n));
  c:=c+1;
od;
RETURN(seq(x[i]=y[i],i=1..n));
end:

debug(cnewton):

cnewton({x^2+y^2-1=0,x^3-y=0},vector(2,[0.7,0.4]),0.0001);
```

```
> cnewton({x^2+y^2-1=0,x^3-y=0},vector(2,[0.7,0.4]),0.0001);

{--> enter cnewton, args = {x^2+y^2-1 = 0, x^3-y = 0}, array(1 ..
2,[(1)=.7,(2)=.4]), .1e-3
```

$$n := 2$$

$$f := [0,\ 0]$$

$$y0 := [1.7,\ 1.4]$$

$$x := [x,\ y]$$

$$y := [0.7,\ 0.4]$$

$$f_1 := x^2 + y^2 - 1$$

$$f_2 := x^3 - y$$

$$jc := \left[\frac{0.5000000000\,(x^2 + y^2 - 1.)}{x\,(1. + 3.\,y\,x)} + \frac{y\,(x^3 - 1.\,y)}{x\,(1. + 3.\,y\,x)},\right.$$
$$\left.\frac{1.500000000\,x\,(x^2 + y^2 - 1.)}{1. + 3.\,y\,x} - \frac{1.\,(x^3 - 1.\,y)}{1. + 3.\,y\,x}\right]$$

$$c := 1$$

$$test := 1.0$$

$$y0_1 := 0.7$$

$$jc0_1 := -0.1535714287$$

$$y_1 := 0.8535714287$$

$$y0_2 := 0.4$$

$$jc0_2 := -0.1800885499$$

$$y_2 := 0.5800885499$$

$$test := 0.1800885499$$
$$c := 2$$
$$y0_1 := 0.8535714287$$
$$jc0_1 := 0.02677208199$$
$$y_1 := 0.8267993467$$
$$y0_2 := 0.5800885499$$
$$jc0_2 := 0.01632684047$$
$$y_2 := 0.5637617094$$
$$test := 0.02677208199$$
$$c := 3$$
$$y0_1 := 0.8267993467$$
$$jc0_1 := 0.0007674197500$$
$$y_1 := 0.8260319270$$
$$y0_2 := 0.5637617094$$
$$jc0_2 := 0.0001375374562$$
$$y_2 := 0.5636241719$$
$$test := 0.0007674197500$$
$$c := 4$$
$$y0_1 := 0.8260319270$$
$$jc0_1 := 0.5693512518\,10^{-6}$$
$$y_1 := 0.8260313576$$
$$y0_2 := 0.5636241719$$
$$jc0_2 := 0.9765710647\,10^{-8}$$
$$y_2 := 0.5636241621$$
$$test := 0.5693512518\,10^{-6}$$
$$c := 5$$

```
<-- exit cnewton (now at top level) = x = .8260313576, y =
.5636241621}
```

$$x = 0.8260313576,\ y = 0.5636241621$$

```
snewton({x^2+y^2-1=0,x^3-y=0},vector(2,[0.7,0.4]),0.0001);
```

$$x = 0.8260165866,\ y = 0.5636182992$$

Pentru metoda lui Newton clasică, respectiv simplificată într-o dimensiune, am construit procedurile `cnewton1d`, respectiv `snewton1d`.

```
cnewton1d:=proc(eqn::equation, x0::numeric, eps::float)
local f,x,fp,y,c,test,y0;
x:=op(indets(eqn, name));
f:=lhs(eqn)-rhs(eqn);
fp:=diff(f,x);
y:=x0;
c:=1; test:=1;
```

```
while test>eps do
  if c>1000 then ERROR(‘Algoritmul nu converge!‘) fi;
  y0:=y;
  y:=evalf(y-subs(x=y,f)/subs(x=y,fp));
  test:=abs(y-y0);
  c:=c+1;
od;
RETURN(x=y);
end:

debug(cnewton1d):

cnewton1d(sin(xx)-xx=0, 0.2,0.001);
```

```
> cnewton1d(sin(xx)-xx=0, 0.2,0.001);
```

```
{--> enter cnewton1d, args = sin(xx)-xx = 0, .2, .1e-2
```

$$x := xx$$
$$f := \sin(xx) - xx$$
$$fp := \cos(xx) - 1$$
$$y := 0.2$$
$$c := 1$$
$$test := 1$$
$$y0 := 0.2$$
$$y := 0.1332443177$$
$$test := 0.0667556823$$
$$c := 2$$
$$y0 := 0.1332443177$$
$$y := 0.08880323922$$
$$test := 0.04444107848$$
$$c := 3$$
$$y0 := 0.08880323922$$
$$y := 0.05919437624$$
$$test := 0.02960886298$$
$$c := 4$$
$$y0 := 0.05919437624$$
$$y := 0.03946061575$$
$$test := 0.01973376049$$
$$c := 5$$
$$y0 := 0.03946061575$$
$$y := 0.02630640064$$
$$test := 0.01315421511$$
$$c := 6$$
$$y0 := 0.02630640064$$
$$y := 0.01753738944$$
$$test := 0.00876901120$$

$$c := 7$$
$$y0 := 0.01753738944$$
$$y := 0.01169155254$$
$$test := 0.00584583690$$
$$c := 8$$
$$y0 := 0.01169155254$$
$$y := 0.007794289520$$
$$test := 0.003897263020$$
$$c := 9$$
$$y0 := 0.007794289520$$
$$y := 0.005196191723$$
$$test := 0.002598097797$$
$$c := 10$$
$$y0 := 0.005196191723$$
$$y := 0.003464143309$$
$$test := 0.001732048414$$
$$c := 11$$
$$y0 := 0.003464143309$$
$$y := 0.002309495886$$
$$test := 0.001154647423$$
$$c := 12$$
$$y0 := 0.002309495886$$
$$y := 0.001539688244$$
$$test := 0.000769807642$$
$$c := 13$$

<-- exit cnewton1d (now at top level) = xx = .1539688244e-2}

$$xx = 0.001539688244$$

Capitolul 4

Interpolare polinomială. Funcţii spline

În practică este des întâlnită situaţia în care se cunoaşte valoarea unei funcţii f în diferite puncte x_i şi se cere valoarea sa într-un punct intermediar. De exemplu, se poate cere valoarea temperaturii aerului la ora 14.30, cunoscându-se temperaturile aerului luate din oră în oră.

Astfel, se pune problema ca, plecând de la punctele date (x_i, y_i) să se determine o funcţie de interpolare al cărei grafic să treacă prin toate punctele date. Interpolarea se numeşte polinomială atunci când se caută funcţii polinomiale având proprietăţile menţionate.

4.1 Polinomul lui Newton cu diferenţe divizate

4.1.1 Breviar teoretic

Fie funcţia $f : X \to \mathbb{R}^1$ dată prin: $y_i = f(x_i), i = 0, 1, \ldots, m$.

Diferenţa divizată de ordinul întâi a lui f relativ la punctul x_r este numărul definit de fracţia:

$$(\mathcal{D}^1 f)(x_r) = \frac{f(x_{r+1}) - f(x_r)}{x_{r+1} - x_r}. \tag{4.1}$$

Diferenţa divizată de ordinul al doilea a funcţiei f relativ la punctul x_r este prin definiţie numărul:

$$(\mathcal{D}^2 f)(x_r) = \frac{(\mathcal{D}^1 f)(x_{r+1}) - (\mathcal{D}^1 f)(x_r)}{x_{r+2} - x_r} = \frac{[x_{r+1}, x_{r+2}, f] - [x_r, x_{r+1}, f]}{x_{r+2} - x_r} \tag{4.2}$$

Prin calcul se găseşte că diferenţa divizată de ordin k a lui f în x_r este:

$$(\mathcal{D}^k f)(x_r) = \sum_{i=0}^{k} \frac{f(x_{r+i})}{\prod\limits_{\substack{m=0 \\ m \neq i}}^{k} (x_{r+i} - x_{r+m})} \tag{4.3}$$

Polinomul lui Newton cu diferenţe divizate se defineşte ca fiind polinomul:

$$\begin{aligned} P_m(x) =& f(x_0) + (\mathcal{D}^1 f)(x_0)(x - x_0) + (\mathcal{D}^2 f)(x_0)(x - x_0)(x - x_1) + \dots \\ &+ (\mathcal{D}^m f)(x_0)(x - x_0)(x - x_1)\dots(x - x_{m-1}) \end{aligned} \tag{4.4}$$

Astfel, funcţia f se poate aproxima după cum urmează:

$$f(x) = P_m(x) + R_m(x), \tag{4.5}$$

unde

$$R_m(x) = \frac{f^{(m+1)}(\xi)}{(m+1)!}(x - x_0)(x - x_1)\dots(x - x_{m-1})(x - x_m) \tag{4.6}$$

este restul sau eroarea de aproximare la interpolarea polinomială.

În cazul în care $x_{i+1} - x_i = h = const$ (i.e. nodurile x_i sunt echidistante), se pot introduce diferenţele finite. Astfel,

$$\triangle f(x_k) = f(x_{k+1}) - f(x_k) \tag{4.7}$$

se numeşte diferenţă finită la dreapta, iar

$$\triangledown f(x_k) = f(x_k) - f(x_{k-1}) \tag{4.8}$$

se numeşte diferenţă finită la stânga.

Diferenţele finite de ordin superior se definesc recursiv, după cum urmează:

$$\triangle^n f(x_k) = \triangle^{n-1} f(x_{k+1}) - \triangle^{n-1} f(x_k) \tag{4.9}$$

$$\triangledown^n f(x_k) = \triangledown^{n-1} f(x_k) - \triangledown^{n-1} f(x_{k-1}) \tag{4.10}$$

Legătura dintre diferenţele divizate şi diferenţele finite este următoarea:

$$(\mathcal{D}^m f)(x_0) = \frac{\triangle^m f(x_0)}{m! h^m}, \qquad (\mathcal{D}^m f)(x_m) = \frac{\triangledown^m f(x_m)}{m! h^m}. \tag{4.11}$$

În acest fel, obţinem polinomul lui Newton cu diferenţe finite la dreapta:

$$\begin{aligned} p_m(x) =& f(x_0) + \frac{\triangle f(x_0)}{h}(x - x_0) + \frac{\triangle^2 f(x_0)}{2! h^2}(x - x_0)(x - x_1) + \dots + \\ &+ \frac{\triangle^m f(x_0)}{m! h^m}(x - x_0)\dots(x - x_{m-1}), \end{aligned} \tag{4.12}$$

respectiv polinomul lui Newton cu diferenţe finite la stânga:

$$\begin{aligned} p_m(x) =& f(x_m) + \frac{\triangledown f(x_m)}{h}(x - x_m) + \frac{\triangledown^2 f(x_m)}{2! h^2}(x - x_m)(x - x_{m-1}) + \dots \\ &+ \frac{\triangledown^m f(x_m)}{m! h^m}(x - x_m)\dots(x - x_1). \end{aligned} \tag{4.13}$$

4.1.2 Probleme rezolvate

Exerciţiul 4.1.1. Să se găsească polinomul de interpolare pentru următorul set de date:

x	1	2	3	4	5
$f(x)$	2	5	10	17	26

folosind

a) diferenţe divizate

b) diferenţe finite la dreapta

c) diferenţe finite la stânga.

Rezolvare

A. Polinomul lui Newton cu diferenţe divizate

1. Obţinerea diferenţelor divizate:

x	$f(x)$	$\mathcal{D}^1 f(x)$	$\mathcal{D}^2 f(x)$	$\mathcal{D}^3 f(x)$	$\mathcal{D}^4 f(x)$
1	2	$\frac{5-2}{2-1}=3$			
2	5	$\frac{10-5}{3-2}=5$	$\frac{5-3}{3-1}=1$	$\frac{1-1}{4-1}=0$	
3	10	$\frac{17-10}{4-3}=7$	$\frac{7-5}{4-2}=1$	$\frac{1-1}{5-2}=0$	$\frac{0-0}{5-1}=0$
4	17	$\frac{26-17}{5-4}=9$	$\frac{9-7}{5-3}=1$		
5	26				

Şirul diferenţelor divizate este şirul primelor valori de pe fiecare coloană, începând cu valorile funcţiei, adică:

$$[2, 3, 1, 0, 0].$$

2. Obţinerea polinomului de interpolare

$$\begin{aligned} P(x) =& 2 + 3 \cdot (x-1) + 1 \cdot (x-1)(x-2) + 0 \cdot (x-1)(x-2)(x-3) + \\ & + 0 \cdot (x-1)(x-2)(x-3)(x-4) = \\ =& 2 + 3x - 3 + x^2 - 3x + 2 = \\ =& x^2 + 1. \end{aligned}$$

B. Polinomul lui Newton cu diferenţe finite la dreapta

Această metodă este aplicabilă, deoarece $x_{i+1} - x_i = 1 \overset{not}{=} h$, $i = \overline{1,4}$.

1. Obţinerea diferenţelor finite la dreapta

x	$f(x)$	$\triangle^1 f(x)$	$\triangle^2 f(x)$	$\triangle^3 f(x)$	$\triangle^4 f(x)$
1	2	$5-2=3$	$5-3=2$	$2-2=0$	$0-0=0$
2	5	$10-5=5$	$7-5=2$	$2-2=0$	–
3	10	$17-10=7$	$9-7=2$	–	–
4	17	$26-17=9$	–	–	–
5	26	–	–	–	–

Şirul diferenţelor finite la dreapta este şirul rezultatelor de pe prima linie, începând cu valorile funcţiei, adică:

$$[2, 3, 2, 0, 0].$$

2. Obţinerea polinomului de interpolare

$$\begin{aligned} P(x) =& 2 + \frac{3}{1}(x-1) + \frac{2}{2! \cdot 1^2}(x-1)(x-2) + \frac{0}{3! \cdot 1^3}(x-1)(x-2)(x-3) + \\ &+ \frac{0}{4! \cdot 1^4}(x-1)(x-2)(x-3)(x-4) = \\ =& 2 + 3x - 3 + x^2 - 3x + 2 = \\ =& x^2 + 1. \end{aligned}$$

C. Polinomul lui Newton cu diferenţe finite la stânga

Această metodă este aplicabilă, deoarece $x_{i+1} - x_i = 1 \overset{not}{=} h$, $i = \overline{1,4}$.

1. Obţinerea diferenţelor finite la stânga

x	$f(x)$	$\triangledown^1 f(x)$	$\triangledown^2 f(x)$	$\triangledown^3 f(x)$	$\triangledown^4 f(x)$
1	2	–	–	–	–
2	5	$5-2=3$	–	–	–
3	10	$10-5=5$	$5-3=2$	–	–
4	17	$17-10=7$	$7-5=2$	$2-2=0$	–
5	26	$26-17=9$	$9-7=2$	$2-2=0$	$0-0=0$

Şirul diferenţelor finite la stânga este şirul rezultatelor de pe ultima linie, începând cu valorile funcţiei, adică:

$$[26, 9, 2, 0, 0].$$

2. Obţinerea polinomului de interpolare

$$\begin{aligned}P(x) =&26+\frac{9}{1}(x-5)+\frac{2}{2!\cdot 1^2}(x-5)(x-4)+\frac{0}{3!\cdot 1^3}(x-5)(x-4)(x-3)+\\&+\frac{0}{4!\cdot 1^4}(x-5)(x-4)(x-3)(x-2)=\\=&26+9x-45+x^2-9x+20=\\=&x^2+1.\end{aligned}$$

4.1.3 Probleme propuse

Exerciţiul 4.1.2. Să se găsească polinomul de interpolare folosind diferenţe divizate, diferenţe finite la stânga şi diferenţe finite la dreapta pentru următoarele seturi de date:

a)

x	-1	0	1	2
$f(x)$	2	1	0	-1

b)

x	$\frac{\pi}{2}$	π	$\frac{3\pi}{2}$	2π	
$f(x)$	1	0	-1	0	1

4.1.4 Implementare

A. Algoritmi

Date de intrare: lista argumentelor x ale funcţiei, lista valorilor fx ale funcţiei, şi o variabilă z

Date de ieşire: valoarea polinomului de interpolare în acel număr, dacă z este un număr, respectiv expresia polinomului de interpolare, dacă z este o nedeterminată

Algoritm pentru diferenţe divizate

1. obţinerea diferenţelor divizate

   $aux = fx$

   $difdiv = [fx_1]$

   $n =$ numărul de elemente ale listei x

   $n_{aux} =$ numărul de elemente ale listei aux

   pentru $s = \overline{1, n-1}$

      calculează elementele listei dd, astfel: $dd_k = \frac{aux_k - aux_{k-1}}{x_{k+s-1}-x_{k-1}}$, $k = \overline{2, n_{aux}}$

      adaugă elementul dd_1 la lista $difdiv$

      $aux = dd$

2. obţinerea polinomului de interpolare

   $P = difdiv_n$

   pentru i de la $n-1$ la 1

      $P = P \cdot (z - x_i) + difdiv_i$

Algoritm pentru diferenţe finite la dreapta

1. obţinerea diferenţelor finite la dreapta

   $aux = fx$

   $dfs = [fx_1]$

   $n =$ numărul de elemente ale listei aux

   pentru $k = \overline{n-1, 1}$

   pentru $i = \overline{1, k}$

   $aux_i = aux_{i+1} - aux_i$

   elimină elementul aux_n din lista aux

   $n = n - 1$

   adaugă elementul aux_1 la lista dfd

2. obţinerea polinomului de interpolare

   $h = x_2 - x_1$

   $P = \dfrac{dfd_n}{(n-1)! \cdot h^{n-1}}$

   pentru i de la $n-1$ la 1

   $P = P \cdot (z - x_i) + \dfrac{dfd_i}{(i-1)! \cdot h^{i-1}}$

Algoritm pentru diferenţe finite la stânga

1. obţinerea diferenţelor finite la stânga

   $aux = fx$

   $n =$ numărul de elemente ale listei aux

   $dfs = [fx_n]$

   pentru $k = \overline{1, n-1}$

   pentru $i = \overline{1, n-k}$

   pentru $i = \overline{1, k}$

   $aux_i = aux_{i+1} - aux_i$

   elimină elementul aux_{n-k+1} din lista aux

   adaugă elementul aux_{n-k} la lista dfs

2. obţinerea polinomului de interpolare

   $h = x_2 - x_1$

   $P = \dfrac{dfs_n}{(n-1)! \cdot h^{n-1}}$

   pentru i de la $n-1$ la 1

   $P = P \cdot (z - x_i) + \dfrac{dfs_i}{(i-1)! \cdot h^{i-1}}$

B. Programe MAPLE şi rezultate

Pentru polinomul lui Newton cu diferenţe divizate, am folosit următoarele proceduri: `difdiv` care construieşte şirul diferenţelor divizate, şi `ddinterp` care construieşte polinomul de interpolare.

```
difdiv:=proc(x::list, fx::list)
local aux, dd, n, s, d1, k;
if nops(x)<>nops(fx) then
  ERROR(‘primul si al doilea argument trebuie sa aibe
    acelasi numar de elemente‘);
fi;
aux:=fx;
dd:=[fx[1]];
n:=nops(x);
for s from 1 to n-1 do
  d1:=[seq((aux[k]-aux[k-1])/(x[k+s-1]-x[k-1]),
       k=2..nops(aux))];
  dd:=[op(dd), d1[1]];
  aux:=d1;
od;
RETURN(dd);
end:

ddinterp:=proc(x::list, fx::list,xx)
local dd, pol, i;
if evalb(type(xx,numeric) and (xx<x[1] or xx>x[nops(x)])) then
  WARNING (‘Polinomul de interpolare da rezultate corecte doar
     pentru x in intervalul [%1,%2]‘, x[1],x[nops(x)]);
fi;
dd:=difdiv(x,fx);
pol:=dd[nops(dd)];
for i from nops(dd)-1 by -1 to 1 do
  pol:=pol*(xx-x[i])+dd[i];
od;
RETURN(simplify(pol));
end:
```

Testăm acest program pentru calculul diferenţelor divizate de la problema rezolvată anterior.

```
x:=[1,2,3,4,5]: fx:=[2,5,10,17,26]: difdiv(x,fx);
```

$$[2, 3, 1, 0, 0]$$

Pentru calculul polinomului de interpolare, avem două variante:

```
ddinterp(x,fx,10);

Warning, Polinomul de interpolare da rezultate corecte doar
pentru x in intervalul [1,5]
```


sau

```
ddinterp(x,fx,z);
```

$$z^2 + 1$$

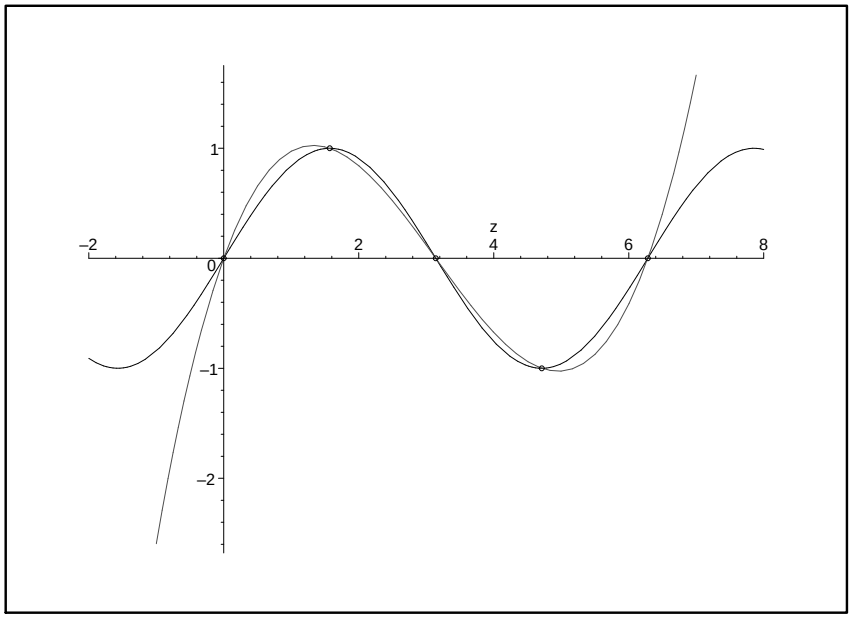
Observaţia 4.1.1. Polinomul de interpolare dă rezultate exacte pentru funcţii polinomiale de grad maxim n, unde n este numărul de puncte în care se cunoaşte valoarea funcţiei căutate.

Observaţia 4.1.2. Fie o funcţie tabelată, dată prin lista ordonată a variabilelor $[x_1 = min, x_2, ..., x_n = max]$, şi lista valorilor sale $[f_1, f_2, ..., f_n]$. Polinomul de interpolare dă rezultate apropiate de valoarea funcţiei doar pentru x în intervalul $[x_1, x_n]$. Pentru a ilustra acest fapt, considerăm funcţia $\sin x$ pe intervalul $[0, 2\pi]$, şi construim polinomul de interpolare cu diferenţe divizate. Pentru o vizualizare mai bună, am reprezentat grafic punctele de interpolare, funcţia $\sin x$ şi polinomul de interpolare.

Avem astfel:

```
x:=[0,Pi/2,Pi,3*Pi/2,2*Pi]: fx:=[0,1,0,-1,0]:
f:=ddinterp(x,fx,z);
p1:=plot(f, z=-2..8, color=red):
p2:=pointplot([seq([x[i],fx[i]],i=1..nops(x))], symbol=circle):
p3:=plot(sin(t), t=-2..8, color=green):
display(p1,p2,p3);
```

$$f := \frac{8(z^2 - 3z\pi + 2\pi^2)z}{3\pi^3}$$



Pentru polinomul lui Newton cu diferenţe finite la dreapta, respectiv la stânga, am folosit proceduri similare celor folosite pentru polinomul lui Newton cu diferenţe divizate.

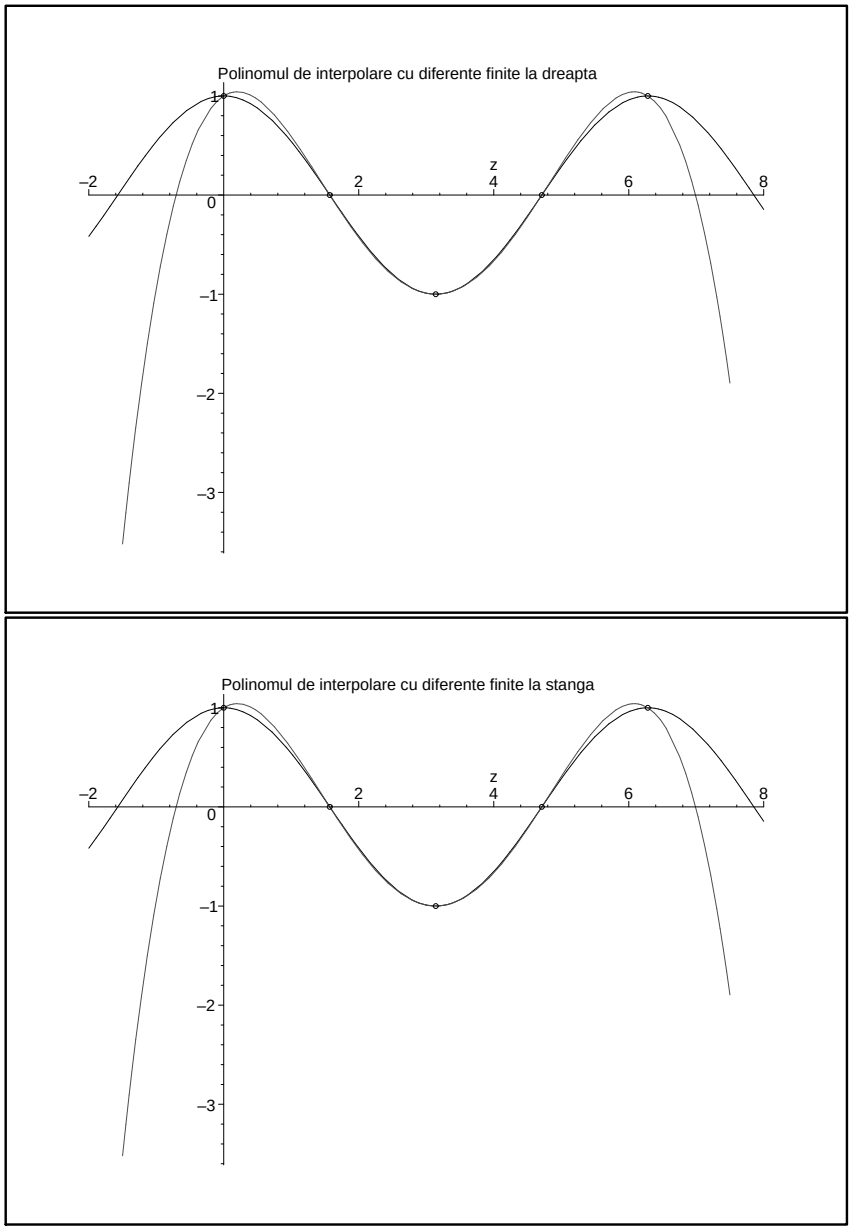
Observaţia 4.1.3. Fie o funcţie tabelată, dată prin lista ordonată a variabilelor, $[x_1 = min, x_2, ..., x_n = max]$, şi lista valorilor sale, $[f_1, f_2, ..., f_n]$. Polinomul de interpolare cu diferenţe finite nu aproximează bine funcţia, pentru valori ale lui x în afara intervalului $[x_1, x_n]$. Un exemplu intuitiv în acest sens îl constituie următoarea secvenţă de program:

```
x:=[0,Pi/2,Pi,3*Pi/2,2*Pi]: fx:=[1,0,-1,0,1]:
f:=dfdinterp(x,fx,z);
g:=dfsinterp(x,fx,z);
p1d:=plot(f, z=-1.5..7.5, color=red):
p1s:=plot(f, z=-1.5..7.5, color=red):
p2:=pointplot([seq([x[i],fx[i]],i=1..nops(x))], symbol=circle):
p3:=plot(cos(t), t=-2..8, color=black, thickness=2):
display(p1d,p2,p3);
display(p1s,p2,p3);
```

al cărei rezultat este:

$$f := -\frac{8\,z^4 - 32\,z^3\,\pi + 34\,z^2\,\pi^2 - 4\,z\,\pi^3 - 3\,\pi^4}{3\,\pi^4}$$

$$g := -\frac{8\,z^4 - 16\,z^3\,\pi + 10\,z^2\,\pi^2 - 8\,z\,\pi^3 - 3\,\pi^4}{3\,\pi^4}$$



4.2 Polinomul de interpolare Lagrange

4.2.1 Breviar teoretic

Fie o funcţie tabelată, dată prin lista ordonată a variabilelor $[x_0 = min, x_1, ..., x_m = max]$, şi lista valorilor sale $[f_0, f_1, ..., f_m]$.

Polinomul de interpolare Lagrange care aproximează funcţia f este

$$(L_m f)(x) = \sum_{i=0}^{m} l_i(x) \cdot f_i \tag{4.14}$$

unde

$$l_i(x) = \frac{(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_m)}{(x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_m)} \ , \ i = \overline{0, m} \tag{4.15}$$

se numesc polinoame de interpolare Lagrange fundamentale.

4.2.2 Probleme rezolvate

Exerciţiul 4.2.1. Să se găsească polinomul de interpolare Lagrange pentru funcţia

x	1	2	3	4	5
$f(x)$	2	5	10	17	26

Rezolvare

Polinoamele de interpolare Lagrange fundamentale sunt:

$$l_0(x) = \frac{(x-2)(x-3)(x-4)(x-5)}{(1-2)(1-3)(1-4)(1-5)}$$
$$l_1(x) = \frac{(x-1)(x-3)(x-4)(x-5)}{(2-1)(2-3)(2-4)(2-5)}$$
$$l_2(x) = \frac{(x-1)(x-2)(x-4)(x-5)}{(3-1)(3-2)(3-4)(3-5)}$$
$$l_3(x) = \frac{(x-1)(x-2)(x-3)(x-5)}{(4-1)(4-2)(4-3)(4-5)}$$
$$l_4(x) = \frac{(x-1)(x-2)(x-3)(x-4)}{(5-1)(5-2)(5-3)(5-4)}$$

Astfel, polinomul de interpolare Lagrange este:

$$(L_4 f)(x) = 2l_0(x) + 5l_1(x) + 10l_2(x) + 17l_4(x) + 26l_5(x)$$

ceea ce, după efectuarea calculelor, devine

$$(L_4 f)(x) = 1 + x^2.$$

4.2.3 Probleme propuse

Exerciţiul 4.2.2. Să se găsească polinomul de interpolare Lagrange pentru următoarele seturi de date şi să se compare cu rezultatul obţinut în paragraful anterior:

a)

x	-1	0	1	2
$f(x)$	2	1	0	-1

b)

x	$\frac{\pi}{2}$	π	$\frac{3\pi}{2}$	2π	
$f(x)$	1	0	-1	0	1

4.2.4 Implementare

A. Algoritm

Date de intrare: lista argumentelor funcţiei, x, lista valorilor funcţiei, fx, şi o variabilă z

Date de ieşire: valoarea polinomului de interpolare în acel număr, dacă z este un număr, respectiv expresia polinomului de interpolare, dacă z este o nedeterminată

Algoritm
n = numărul de elemente ale listei x
$s = 0$
pentru $i = \overline{1,n}$
  $p = 1$
  pentru $j = \overline{1,n}$
    dacă $i \neq j$ atunci $p = p \cdot \dfrac{z - x_j}{x_i - x_j}$
  $s = s + f_i \cdot p$
returnează s

B. Programe MAPLE şi rezultate

```
lagrange:=proc(x::list, f::list, xx)
local n,s,p,i,j;
if nops(x)<>nops(f) then
  ERROR(‘primul si al doilea argument trebuie sa aibe
    acelasi numar de elemente‘);
fi;
if evalb(type(xx,numeric) and (xx<x[1] or xx>x[nops(x)])) then
  WARNING (‘Polinomul de interpolare da rezultate corecte doar
     pentru x in intervalul [%1,%2]‘, x[1],x[nops(x)]);
fi;
n:=nops(x);
s:=0;
for i from 1 to n do
  p:=1;
  for j from 1 to n do
    if i<>j then p:=p*(xx-x[j])/(x[i]-x[j]) fi;
  od;
  s:=s+f[i]*p;
od;
RETURN(simplify(s));
end:

x:=[1,2,3,4,5]:fx:=[2,5,10,17,26]:lagrange(x,fx,z);
```

$$1 + z^2$$

Observaţia 4.2.1. Polinomul de interpolare dă rezultate exacte pentru funcţii polinomiale de grad maxim n, unde n este numărul de puncte în care se cunoaşte valoarea funcţiei căutate.

Observaţia 4.2.2. Şi în acest caz este valabilă observaţia că, în general, polinomul de interpolare nu dă rezultate corecte pentru valori ale parametrului z situate în afara intervalului care conţine argumentele funcţiei.

Observaţia 4.2.3. Aspecte practice ale interpolării polinomiale:

1. Polinoamele Newton şi Lagrange diferă numai prin formă; restul este acelaşi în ambele cazuri, dacă se consideră aceeaşi reţea de noduri. Din punct de vedere al calculului numeric, este preferată folosirea polinomului Newton deoarece acesta necesită un număr de operaţii aritmetice mai mic faţă de polinomul Lagrange. Necesarul de memorie este acelaşi pentru ambii algoritmi.

2. Dacă notăm cu α şi β cel mai mic, respectiv cel mai mare dintre nodurile de interpolare, atunci din punct de vedere computaţional, sunt convenabile următoarele polinoame de interpolare: pentru x apropiat de α este convenabilă utilizarea polinomului Newton cu diferenţe la dreapta (forward); pentru x apropiat de β este convenabilă utilizarea polinomului Newton cu diferenţe la stânga (backward).

4.3 Interpolare spline

4.3.1 Breviar teoretic

Fie o funcţie tabelată, dată prin lista ordonată a variabilelor, $[x_1 = min, x_2, ..., x_n = max]$, şi lista valorilor sale, $[f_1, f_2, ..., f_n]$.

Deoarece interpolarea polinomială globală pe tot intervalul $[x_1, x_n]$ (de exemplu, polinoamele de interpolare Newton şi Lagrange) nu converge întotdeauna, apare ideea de interpolare polinomială pe porţiuni (interpolare spline), la care pe fiecare subdiviziune a intervalului $[x_0, x_n]$ definim un alt polinom de interpolare.

Funcţia spline polinomială de ordinul întâi are expresia

$$S(x) = S_i(x) = s_{i,0} + s_{i,1}(x - x_i),\ x \in [x_{i-1}, x_i] \tag{4.16}$$

unde coeficienţii s_{ik} se determină din condiţiile:

$$\begin{aligned} &S(x_i) = y_i,\ i = \overline{0, n}; \\ &S_i(x_i) = S_{i+1}(x_i),\ i = \overline{1, n-2}. \end{aligned} \tag{4.17}$$

Funcţia spline polinomială de ordinul al doilea are expresia

$$S(x) = S_i(x) = s_{i,0} + s_{i,1}(x - x_i) + s_{i,2}(x - x_i)^2,\ x \in [x_{i-1}, x_i] \tag{4.18}$$

unde coeficienţii s_{ik} se determină din condiţiile:

$$\begin{aligned} &S(x_i) = y_i,\ i = \overline{0, n}; \\ &S_i(x_i) = S_{i+1}(x_i),\ i = \overline{1, n-1}; \\ &S_i'(x_i) = S_{i+1}'(x_i),\ i = \overline{1, n-1}. \end{aligned} \tag{4.19}$$

Deoarece avem $3n$ coeficienţi şi $3n - 1$ condiţii, pentru a determina în mod unic funcţia spline polinomială de ordinul al doilea mai avem nevoie de o condiţie suplimentară. Această condiţie suplimentară se referă la prima derivată, şi are următoarea seminficaţie:

unul din capetele funcţiei spline de ordinul al doilea trebuie să fie punct de extrem local. De regulă, se alege

$$S_0{}'(x_0) = 0. \tag{4.20}$$

Pentru diferite condiţii suplimentare, se obţin diferite funcţii spline.

Funcţia spline polinomială cubică (de ordinul al treilea) are expresia

$$S(x) = S_i(x) = s_{i,0} + s_{i,1}(x - x_i) + s_{i,2}(x - x_i)^2 + s_{i,3}(x - x_i)^3,\ x \in [x_{i-1}, x_i] \tag{4.21}$$

unde coeficienţii s_{ik} se determină din condiţiile:

$$\begin{aligned} &S(x_i) = y_i,\ i = \overline{0, n};\\ &S_i(x_i) = S_{i+1}(x_i),\ i = \overline{1, n-1};\\ &S_i'(x_i) = S_{i+1}'(x_i),\ i = \overline{1, n-1};\\ &S_i''(x_i) = S_{i+1}''(x_i),\ i = \overline{1, n-1}. \end{aligned} \tag{4.22}$$

Deoarece avem $4n$ coeficienţi şi $4n - 2$ condiţii, pentru determinarea în mod unic a funcţiei spline polinomiale cubice este nevoie de două condiţii suplimentare. Aceste condiţii suplimentare pot fi

- libere (sau naturale):

$$S_1''(x_0) = S_{n-1}''(x_{n-1}) = 0 \tag{4.23}$$

caz în care vorbim de o funcţie spline cubică naturală;

- "clamped":

$$S_1'(x_0) = y_0',\ S_{n-1}'(x_{n-1}) = y_{n-1}'. \tag{4.24}$$

4.3.2 Probleme rezolvate

Exerciţiul 4.3.1. Să se găsească funcţiile spline de interpolare de ordinul întâi, doi şi trei pentru următorul set de date (funcţia sinus):

x	0	$\frac{\pi}{2}$	π	$\frac{3\pi}{2}$	2π
$f(x)$	0	1	0	-1	0

Rezolvare

Avem 5 puncte, deci $n = 4$ intervale.

A. Funcţia spline de ordinul întâi

Funcţia căutată va avea expresia:

$$S(x) = \begin{cases} s_{10} + s_{11}(x - 0) & , x \in [0, \frac{\pi}{2}] \\ s_{20} + s_{21}(x - \frac{\pi}{2}) & , x \in [\frac{\pi}{2}, \pi] \\ s_{30} + s_{31}(x - \pi) & , x \in [\pi, \frac{3\pi}{2}] \\ s_{40} + s_{41}(x - \frac{3\pi}{2}) & , x \in [\frac{3\pi}{2}, 2\pi]. \end{cases}$$

Notăm:

$$\begin{cases} S_1 = s_{10} + s_{11}(x - 0) \\ S_2 = s_{20} + s_{21}(x - \frac{\pi}{2}) \\ S_3 = s_{30} + s_{31}(x - \pi) \\ S_4 = s_{40} + s_{41}(x - \frac{3\pi}{2}) \end{cases}$$

Condiţiile care se pun sunt următoarele:

- funcţia trece prin puncte

$$S_1(0) = 0 \qquad \Rightarrow s_{10} = 0$$
$$S_2(\frac{\pi}{2}) = 1 \qquad \Rightarrow s_{20} = 1$$
$$S_3(\pi) = 0 \qquad \Rightarrow s_{30} = 0$$
$$S_4(\frac{3\pi}{2}) = -1 \qquad \Rightarrow s_{40} = -1$$
$$S_4(2\pi) = 0 \qquad \Rightarrow s_{41} = \frac{2}{\pi}$$

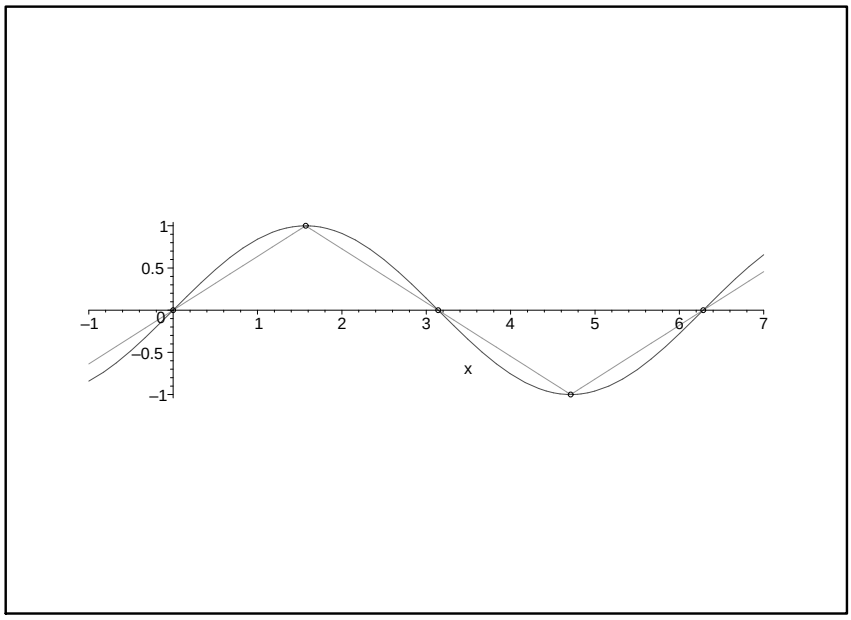
- funcţia este continuă

$$S_1(\frac{\pi}{2}) = S_2(\frac{\pi}{2}) \qquad \Rightarrow s_{11} = \frac{2}{\pi}$$
$$S_2(\pi) = S_3(\pi) \qquad \Rightarrow s_{21} = -\frac{2}{\pi}$$
$$S_3(\frac{3\pi}{2}) = S_4(\frac{3\pi}{2}) \qquad \Rightarrow s_{31} = -\frac{2}{\pi}$$

Cu acestea, după efectuarea calculelor, funcţia căutată devine:

$$S(x) = \begin{cases} \frac{2}{\pi}x & , x \in [0, \frac{\pi}{2}] \\ -\frac{2}{\pi}x + 2 & , x \in [\frac{\pi}{2}, \pi] \\ -\frac{2}{\pi}x + 2 & , x \in [\pi, \frac{3\pi}{2}] \\ \frac{2}{\pi}x - 4 & , x \in [\frac{3\pi}{2}, 2\pi]. \end{cases}$$

În figura următoare am reprezentat graficul funcţiei spline S în raport cu funcţia iniţială $(\sin x)$:



B. Funcţia spline de ordinul al doilea

Funcţia căutată va avea expresia:

$$S(x) = \begin{cases} s_{10} + s_{11}(x-0) + s_{12}(x-0)^2 & , x \in [0, \frac{\pi}{2}] \\ s_{20} + s_{21}(x-\frac{\pi}{2}) + s_{22}(x-\frac{\pi}{2})^2 & , x \in [\frac{\pi}{2}, \pi] \\ s_{30} + s_{31}(x-\pi) + s_{32}(x-\pi)^2 & , x \in [\pi, \frac{3\pi}{2}] \\ s_{40} + s_{41}(x-\frac{3\pi}{2}) + s_{42}(x-\frac{3\pi}{2})^2 & , x \in [\frac{3\pi}{2}, 2\pi]. \end{cases}$$

Notăm:

$$\begin{cases} S_1 = s_{10} + s_{11}(x-0) + s_{12}(x-0)^2 \\ S_2 = s_{20} + s_{21}(x-\frac{\pi}{2}) + s_{22}(x-\frac{\pi}{2})^2 \\ S_3 = s_{30} + s_{31}(x-\pi) + s_{32}(x-\pi)^2 \\ S_4 = s_{40} + s_{41}(x-\frac{3\pi}{2}) + s_{42}(x-\frac{3\pi}{2})^2 \end{cases}$$

Condiţiile care se pun sunt următoarele:

- funcţia trece prin puncte

  $S_1(0) = 0 \Rightarrow s_{10} = 0$

  $S_2(\frac{\pi}{2}) = 1 \Rightarrow s_{20} = 1$

  $S_3(\pi) = 0 \Rightarrow s_{30} = 0$

  $S_4(\frac{3\pi}{2}) = -1 \Rightarrow s_{40} = -1$

  $S_4(2\pi) = 0$
- funcţia este continuă

  $S_1(\frac{\pi}{2}) = S_2(\frac{\pi}{2})$

  $S_2(\pi) = S_3(\pi)$

  $S_3(\frac{3\pi}{2}) = S_4(\frac{3\pi}{2})$
- derivata este continuă

  $S_1'(\frac{\pi}{2}) = S_2'(\frac{\pi}{2})$

  $S_2'(\pi) = S_3'(\pi)$

  $S_3'(\frac{3\pi}{2}) = S_4'(\frac{3\pi}{2})$
- condiţie suplimentară

  $S_1'(0) = 0 \Rightarrow s_{11} = 0$

Înlocuind valorile cunoscute în celelalte ecuaţii, obţinem următorul sistem liniar

$$\begin{cases} -1 + s_{41}\frac{\pi}{2} + s_{42}(\frac{\pi}{2})^2 = 0 \\ s_{21}\frac{\pi}{2} + s_{22}(\frac{\pi}{2})^2 = -1 \\ s_{31}\frac{\pi}{2} + s_{32}(\frac{\pi}{2})^2 = -1 \\ 2s_{12}\frac{\pi}{2} - s_{21} = 0 \\ s_{12}(\frac{\pi}{2})^2 = 1 \\ s_{21} + 2s_{22}\frac{\pi}{2} - s_{31} = 0 \\ s_{31} + 2s_{32}\frac{\pi}{2} - s_{41} = 0 \end{cases}$$

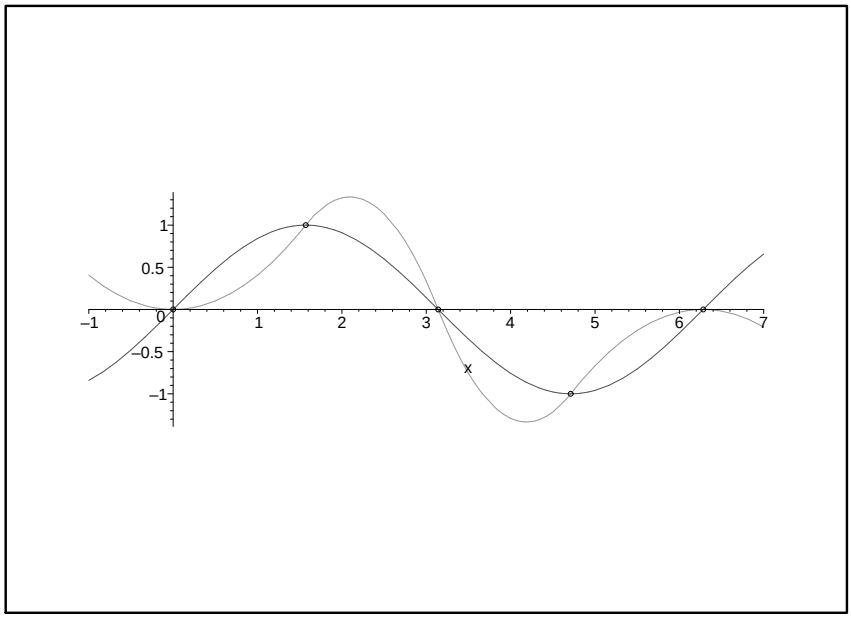
cu soluţia

$$s_{12} = \frac{4}{\pi^2}\ ,\ s_{22} = -\frac{12}{\pi^2}\ s_{32} = \frac{12}{\pi^2}\ ,\ s_{42} = -\frac{4}{\pi^2}\ ,\ s_{21} = \frac{4}{\pi}\ ,\ s_{31} = -\frac{8}{\pi}\ ,\ s_{41} = \frac{4}{\pi}.$$

În acest fel, am determinat funcţia spline polinomială de ordinul al doilea care aproximează funcţia sinus pe intervalul $[0, 2\pi]$:

$$S(x) = \begin{cases} \frac{4}{\pi^2}x^2 & , x \in [0, \frac{\pi}{2}] \\ -\frac{12}{\pi^2}x^2 + \frac{16}{\pi}x - 4 & , x \in [\frac{\pi}{2}, \pi] \\ \frac{12}{\pi^2}x^2 - \frac{32}{\pi}x + 20 & , x \in [\pi, \frac{3\pi}{2}] \\ -\frac{4}{\pi^2}x^2 + \frac{16}{\pi}x - 16 & , x \in [\frac{3\pi}{2}, 2\pi]. \end{cases}$$

În figura următoare am reprezentat graficul funcţiei spline S în raport cu funcţia iniţială $(\sin x)$:



C. Funcţia spline cubică

Funcţia căutată va avea expresia:

$$S(x) = \begin{cases} s_{10} + s_{11}(x-0) + s_{12}(x-0)^2 + s_{13}(x-0)^3 & , x \in [0, \frac{\pi}{2}] \\ s_{20} + s_{21}(x-\frac{\pi}{2}) + s_{22}(x-\frac{\pi}{2})^2 + s_{23}(x-\frac{\pi}{2})^3 & , x \in [\frac{\pi}{2}, \pi] \\ s_{30} + s_{31}(x-\pi) + s_{32}(x-\pi)^2 + s_{33}(x-\pi)^3 & , x \in [\pi, \frac{3\pi}{2}] \\ s_{40} + s_{41}(x-\frac{3\pi}{2}) + s_{42}(x-\frac{3\pi}{2})^2 + s_{43}(x-\frac{3\pi}{2})^3 & , x \in [\frac{3\pi}{2}, 2\pi]. \end{cases}$$

Notăm:

$$S(x) = \begin{cases} S_1 = s_{10} + s_{11}(x-0) + s_{12}(x-0)^2 + s_{13}(x-0)^3 \\ S_2 = s_{20} + s_{21}(x-\frac{\pi}{2}) + s_{22}(x-\frac{\pi}{2})^2 + s_{23}(x-\frac{\pi}{2})^3 \\ S_3 = s_{30} + s_{31}(x-\pi) + s_{32}(x-\pi)^2 + s_{33}(x-\pi)^3 \\ S_4 = s_{40} + s_{41}(x-\frac{3\pi}{2}) + s_{42}(x-\frac{3\pi}{2})^2 + s_{43}(x-\frac{3\pi}{2})^3 \end{cases}$$

Condiţiile care se pun sunt următoarele:

- funcţia trece prin puncte
  $S_1(0) = 0$
  $S_2(\frac{\pi}{2}) = 1$
  $S_3(\pi) = 0$
  $S_4(\frac{3\pi}{2}) = -1$
  $S_4(2\pi) = 0$
- funcţia este continuă
  $S_1(\frac{\pi}{2}) = S_2(\frac{\pi}{2})$
  $S_2(\pi) = S_3(\pi)$
  $S_3(\frac{3\pi}{2}) = S_4(\frac{3\pi}{2})$
- derivata este continuă
  $S_1{}'(\frac{\pi}{2}) = S_2{}'(\frac{\pi}{2})$
  $S_2{}'(\pi) = S_3{}'(\pi)$
  $S_3{}'(\frac{3\pi}{2}) = S_4{}'(\frac{3\pi}{2})$
- derivata a doua este continuă
  $S_1{}''(\frac{\pi}{2}) = S_2{}''(\frac{\pi}{2})$
  $S_2{}''(\pi) = S_3{}''(\pi)$
  $S_3{}''(\frac{3\pi}{2}) = S_4{}''(\frac{3\pi}{2})$
- condiţii suplimentare
  $S_1{}''(0) = 0$
  $S_4{}''(2\pi) = 0$

Soluţia acestui sistem va fi:

$$s_{10} = 0 \qquad s_{11} = \frac{3}{\pi} \qquad s_{12} = 0 \qquad s_{13} = \frac{4}{\pi^3}$$
$$s_{20} = 1 \qquad s_{21} = 0 \qquad s_{22} = -\frac{6}{\pi^2} \qquad s_{23} = \frac{4}{\pi^3}$$

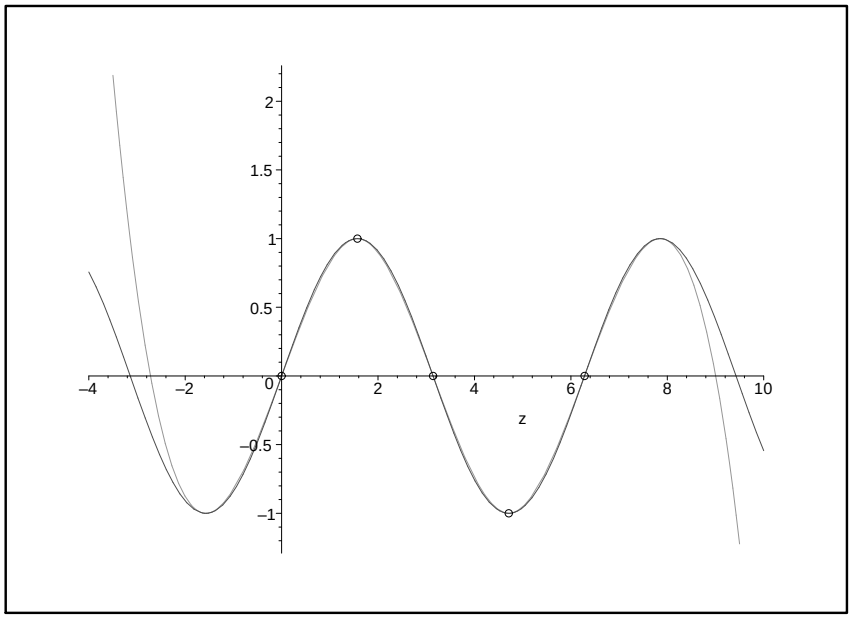
$$s_{30} = 0 \qquad s_{31} = -\frac{3}{\pi} \qquad s_{32} = 0 \qquad s_{13} = \frac{4}{\pi^3}$$
$$s_{40} = -1 \qquad s_{41} = 0 \qquad s_{42} = \frac{6}{\pi^2} \qquad s_{13} = -\frac{4}{\pi^3}$$

de unde, după efectuarea simplificărilor, rezultă că expresia funcţiei spline cubice naturale

care interpolează setul de date iniţial este:

$$S(x) = \begin{cases} -\dfrac{x(-3\pi^2+4x^2)}{\pi^3} & , x \in [0, \frac{\pi}{2}] \\ \dfrac{(-\pi^3-12\pi x^2+9x\pi^2+4x^3)}{\pi^3} & , x \in [\frac{\pi}{2}, \pi] \\ \dfrac{(x-\pi)(\pi^2+4x^2-8x\pi)}{\pi^3} & , x \in [\pi, \frac{3\pi}{2}] \\ -\dfrac{(-26\pi^3-24\pi x^2+45x\pi^2+4x^3)}{\pi^3} & , x \in [\frac{3\pi}{2}, 2\pi]. \end{cases}$$

Redăm în figura următoare graficul funcţiei spline cubice S în raport cu funcţia iniţială $(\sin x)$:



4.3.3 Probleme propuse

Exerciţiul 4.3.2. Să se găsească funcţia spline de ordinul întâi si funcţia spline cubică naturală pentru următoarele seturi de date:

a)

x	-1	0	1	2
$f(x)$	2	1	0	-1

b)

x	$\frac{\pi}{2}$	π	$\frac{3\pi}{2}$	2π	
$f(x)$	1	0	-1	0	1

4.3.4 Implementare

A. Algoritmi

Prezentăm în continuare algoritmul pentru obţinerea funcţiei spline cubice naturale.

Date de intrare lista argumentelor funcţiei, x, lista valorilor funcţiei, f, si variabila z

Date de ieşire dacă z este număr, atunci returnează valoarea funcţiei spline în acel număr, dacă z este simbol, atunci returnează expresiile pe subintervale ale funcţiei spline.

Algoritm

n = numărul de puncte, A - matrice pătratică de ordinul $n-2$, b - vector cu $n-2$ linii

pentru $i = \overline{1, n-1}$

$h_i = x_{i+1} - x_i$

pentru $i = \overline{1, n-2}$

$A_{i,i} = 2(h_i + h_{i+1})$

pentru $i = \overline{2, n-2}$

$A_{i,i-1} = h_i$

$A_{i-1,i} = h_i$

pentru $i = \overline{1, n-2}$

$b_i = 6\left(\dfrac{f_{i+2} - fi + 1}{h_{i+1}} - \dfrac{f_{i+1} - f_i}{h_i}\right)$

$c_1 = 0$

$c_n = 0$

pentru $i = \overline{2, n-1}$

$c_i =$ a $i-1$-a soluţie a sistemului tridiagonal $Ac = b$

pentru $i = \overline{1, n-1}$

$s_{i0} = f_i$

$s_{i1} = \dfrac{f_{i+1} - f_i}{h_i} - \dfrac{2h_ic_i + h_ic_{i+1}}{6}$

$s_{i2} = \dfrac{c_i}{2}$

$s_{i3} = \dfrac{c_{i+1} - c_i}{6h_i}$

B. Programe MAPLE şi rezultate

Programul Maple pentru determinarea funcţiei spline cubice naturale este următorul:

```
cubicspline:=proc(x::list, f::list, xx)
local n,i,h, A, b,c,aa,bb,cc,dd;
n:=nops(x);
A:=matrix(n-2, n-2, 0);
b:=vector(n-2);
for i from 1 to n-1 do
  h[i]:=x[i+1]-x[i];
od;
for i from 1 to n-2 do
  A[i,i]:=2*(h[i]+h[i+1]);
od;
for i from 2 to n-2 do
  A[i,i-1]:=h[i];
  A[i-1,i]:=h[i];
od;
for i from 1 to n-2 do
  b[i]:=6*( (f[i+2]-f[i+1])/h[i+1] - (f[i+1]-f[i])/h[i] );
od;
```

```
c:=linsolve(A,b);
c:=[0,seq(c[i],i=1..n-2),0];
for i from 1 to n-1 do
  aa[i]:=f[i];
  bb[i]:=(f[i+1]-f[i])/h[i]-(2*c[i]*h[i]+c[i+1]*h[i])/6;
  cc[i]:=c[i]/2;
  dd[i]:=(c[i+1]-c[i])/(6*h[i]);
od;
if type(xx,numeric) then
  if evalb(evalf(xx)<evalf(x[1]) or evalf(xx)>evalf(x[n])) then
  WARNING(‘rezultatele interpolarii nu sunt exacte decat pentru
    x in intervalul [%1,%2]‘, x[1],x[n]) fi;
  if (evalf(xx)<evalf(x[1])) then
    RETURN(aa[1]+bb[1]*(xx-x[1])+cc[1]*(xx-x[1])^2+dd[1]*(xx-x[1])^3);
  else
    i:=1;
    while evalb(evalf(x[i])<=evalf(xx) and i<n) do i:=i+1; od;
    if i>=n then
      RETURN(aa[n-1]+bb[n-1]*(xx-x[n-1])+cc[n-1]*(xx-x[n-1])^2+
        dd[n-1]*(xx-x[n-1])^3);
    else
      i:=i-1;
    fi;
    RETURN(aa[i]+bb[i]*(xx-x[i])+cc[i]*(xx-x[i])^2+dd[i]*(xx-x[i])^3);
  fi;
fi;
if type(xx, name) then
  printf(‘Functia spline cubice care interpoleaza datele x=%a si
    f(x)=%a este\n‘, x, f);
  for i from 1 to n-1 do
    printf(‘  %a+%a*(%a-%a)+%a*(%a-%a)^2+%a*(%a-%a)^3 , pentru
      %a in intervalul [%a,%a] \n‘, aa[i],bb[i],xx,x[i],cc[i],
      xx,x[i],dd[i],xx,x[i],xx,x[i],x[i+1]);
  od;
fi;
RETURN(seq(simplify(f[i]+bb[i]*(xx-x[i])+cc[i]*(xx-x[i])^2+
  dd[i]*(xx-x[i])^3), i=1..n-1));
end:
```

Testăm această procedură pe exemplul rezolvat anterior.

```
x:=[0,Pi/2,Pi,3*Pi/2,2*Pi]: fx:=[0,1,0,-1,0]:
x1:=cubicspline(x,fx,10);
x2:=cubicspline(x,fx,3);
cubicspline(x,fx,z);
```

```
Warning, rezultatele interpolarii nu sunt exacte decat
pentru x in intervalul [0,2*Pi]
```

$$x_1 := -1 + \frac{6\left(10 - \frac{3\pi}{2}\right)^2}{\pi^2} - \frac{4\left(10 - \frac{3\pi}{2}\right)^3}{\pi^3}$$

$$x_2 := 1 + \frac{6\left(3 - \frac{\pi}{2}\right)^2}{\pi^2} + \frac{4\left(3 - \frac{\pi}{2}\right)^3}{\pi^3}$$

```
Functia spline cubice care interpoleaza datele
  x=[0, 1/2*Pi, Pi, 3/2*Pi, 2*Pi] si f(x)=[0, 1, 0, -1, 0] este

  0+3/Pi*(z-0)+0*(z-0)^2+-4/Pi^3*(z-0)^3 , pentru z in
    intervalul [0,1/2*Pi]
  1+0*(z-1/2*Pi)+-6/Pi^2*(z-1/2*Pi)^2+4/Pi^3*(z-1/2*Pi)^3 ,
    pentru z in intervalul [1/2*Pi,Pi]
  0+-3/Pi*(z-Pi)+0*(z-Pi)^2+4/Pi^3*(z-Pi)^3 , pentru z in
    intervalul [Pi,3/2*Pi]
  -1+0*(z-3/2*Pi)+6/Pi^2*(z-3/2*Pi)^2+-4/Pi^3*(z-3/2*Pi)^3 ,
    pentru z in intervalul [3/2*Pi,2*Pi]
```

$$-\frac{z\left(-3\,\pi^2+4\,z^2\right)}{\pi^3},\ \frac{-\pi^3-12\,\pi\,z^2+9\,z\,\pi^2+4\,z^3}{\pi^3},$$
$$\frac{(z-\pi)\left(\pi^2+4\,z^2-8\,z\,\pi\right)}{\pi^3},\ -\frac{-26\,\pi^3-24\,\pi\,z^2+45\,z\,\pi^2+4\,z^3}{\pi^3}$$

4.4 Polinoame Bernstein

4.4.1 Breviar teoretic

Polinomul Bernstein de grad m de aproximare a funcţiei f pe intervalul $[0,1]$ este dat de relaţia:

$$(B_m f)(x) = \sum_{k=0}^{m} C_m^k \cdot x^k \cdot (1-x)^{m-k} \cdot f\left(\frac{k}{m}\right),\ x \in [0,1]. \tag{4.25}$$

Cu ajutorul polinoamelor Bernstein se poate construi curba Bezier asociată punctelor $P_i(x_i, y_i)$, $i = \overline{0,n}$. Aceasta este dată prin ecuaţiile parametrice:

$$\begin{aligned} x(t) &= \sum_{i=0}^{n} C_n^i \cdot t^i (1-t)^{n-i}\, x_i, \\ y(t) &= \sum_{i=0}^{n} C_n^i \cdot t^i (1-t)^{n-i}\, y_i\ ,\ t \in [0,1]. \end{aligned} \tag{4.26}$$

4.4.2 Probleme rezolvate

Exerciţiul 4.4.1. Găsiţi curba Bezier determinată de punctele $P_1(0,0)$, $P_2(1,2)$, $P_3(2,1)$, $P_4(3,2)$, $P_5(2,4)$.

Rezolvare

Ecuaţiile parametrice ale curbei Bezier sunt date de:

$$\begin{aligned}x(t) =& \sum_{i=0}^{5} C_n^i \cdot t^i(1-t)^{n-i}\, x_i = C_4^0 t^0(1-t)^4 \cdot 0 + C_4^1 t^1(1-t)^3 \cdot 1 + \\ &+ C_4^2 t^2(1-t)^2 \cdot 2 + C_4^3 t^3(1-t)^1 \cdot 3 + C_4^4 t^4(1-t)^0 \cdot 2 = \\ =& 4t(1-t)^3 + 12t^2(1-t)^2 + 12t^3(1-t) + 2t^4 \\ y(t) =& \sum_{i=0}^{5} C_n^i \cdot t^i(1-t)^{n-i}\, x_i = C_4^0 t^0(1-t)^4 \cdot 0 + C_4^1 t^1(1-t)^3 \cdot 2 + \\ &+ C_4^2 t^2(1-t)^2 \cdot 1 + C_4^3 t^3(1-t)^1 \cdot 2 + C_4^4 t^4(1-t)^0 \cdot 4 = \\ =& 8t(1-t)^3 + 6t^2(1-t)^2 + 8t^3(1-t) + 4t^4\end{aligned}$$

4.4.3 Probleme propuse

Exerciţiul 4.4.2. Să se scrie ecuaţia curbei Bezier determinată de punctele $(0,0)$, $(0,2)$, $(2,1)$, $(2,2)$.

Exerciţiul 4.4.3. Să se scrie ecuaţia curbei Bezier determinată de punctele $(1,0)$, $(0,2)$, $(1,1)$, $(2,0)$, $(3,1)$.

4.4.4 Implementare

A. Algoritmi

Date de intrare o listă de puncte în plan, $P_i(x_i, y_i)$, $i = \overline{1,n}$

Date de ieşire ecuaţiile parametrice ale curbei Bezier determinată de aceste puncte

Algoritm pentru $i = \overline{0,n}$

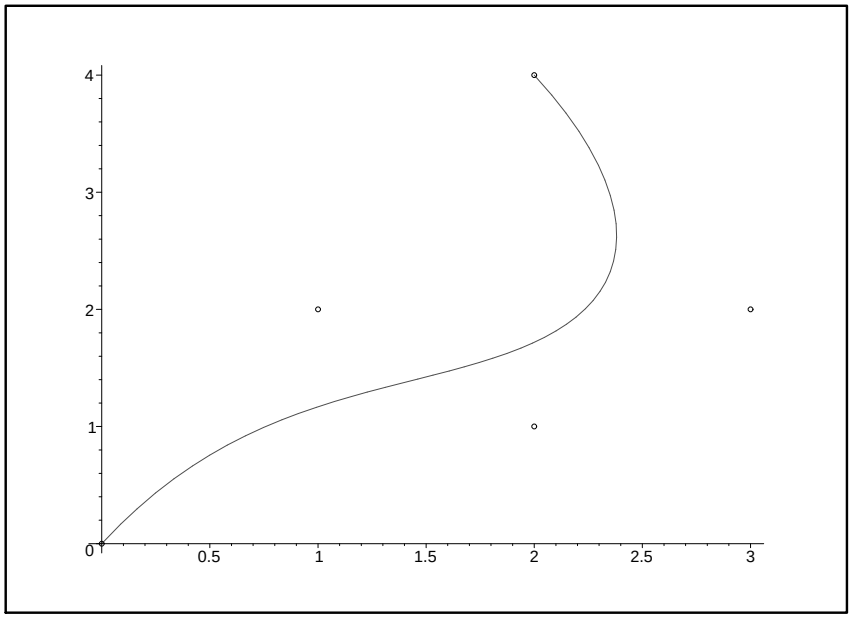
$$\begin{aligned}x &= x + C_n^i \cdot t^i(1-t)^{n-i}\, x_{i+1} \\ y &= y + C_n^i \cdot t^i(1-t)^{n-i}\, y_{i+1}\end{aligned}$$

B. Programe MAPLE şi rezultate

Testăm această procedură pe exemplul rezolvat anterior.

```
pct:=[[0,0],[1,2],[2,1],[3,2],[2,4]];
bezier(pct);
with(plots):
p1:=pointplot(pct,symbol=circle):
p2:=plot([bezier(pct)[1],bezier(pct)[2], t=0..1], thickness=2):
display(p1, p2);
```

$$4t(1-t)^3 + 12t^2(1-t)^2 + 12t^3(1-t) + 2t^4,\ 8t(1-t)^3 + 6t^2(1-t)^2 + 8t^3(1-t) + 4t^4$$


4
3
2
1
0
0.5
1
1.5
2
2.5
3

Capitolul 5

Derivare numerică

Necesitatea derivării numerice apare în practică în următoarele două situaţii:

a) expresia analitică a funcţiei date f conduce la o derivare formală dificilă, caz în care derivata se aproximează prin diferenţe finite;

b) expresia analitică a funcţiei f nu este cunoscută (i.e. funcţia f este dată tabelat), caz în care derivata se aproximează prin derivata unui polinom de interpolare

5.1 Aproximarea derivatei prin diferenţe finite

5.1.1 Breviar teoretic

Aproximarea derivatei întâi:
a) cu diferenţe finite la dreapta:

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{f''(\xi)}{2}h,\ \xi \in (x, x+h) \tag{5.1}$$

b) cu diferenţe finite la stânga:

$$f'(x) = \frac{f(x) - f(x-h)}{h} + \frac{f''(\xi)}{2}h,\ \xi \in (x-h, x) \tag{5.2}$$

Aproximarea derivatei a doua:
a) cu diferenţe finite la dreapta:

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} - \frac{h^2}{12}f^{(4)}(\xi),\ \xi \in (x-h, x+h) \tag{5.3}$$

b) cu diferenţe finite la stânga:

$$f''(x) = \frac{f(x) - 2f(x-h) + f(x-2h)}{h^2} + \frac{h^2}{12}f^{(4)}(\xi),\ \xi \in (x-2h, x) \tag{5.4}$$

5.1.2 Probleme rezolvate

Exerciţiul 5.1.1. Să se găsească prima derivată a funcţiei $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2+3x+2$ cu diferenţe finite la stânga şi la dreapta, în punctul $x_0 = 2$.

Rezolvare

Precizia cu care este returnată valoarea derivatei este legată de valoarea pasului h. Pentru a exemplifica acest lucru, folosind varianta clasică cu formula de derivare calculăm valoarea derivatei în punctul $x_0 = 2$, şi apoi o comparăm cu diversele valori obţinute pentru diferite valori ale lui h.

Astfel, folosind formula de derivare avem:

$$f'(x) = 2x + 3$$

de unde rezultă că $f'(2) = 7$.

Folosind aproximarea derivatei întâi cu diferenţe finite la dreapta, respectiv la stânga, pentru $h = 0.1$, avem:

$$f'(2) = \frac{f(2 + 0.1) - f(2)}{0.1} = 7.1$$

respectiv

$$f'(2) = \frac{f(2) - f(2 - 0.1)}{0.1} = 6.9,$$

iar pentru $h = 0.01$, avem

$$f'(2) = \frac{f(2 + 0.01) - f(2)}{0.01} = 7.01$$

respectiv

$$f'(2) = \frac{f(2) - f(2 - 0.01)}{0.01} = 6.99,$$

şi aşa mai departe.

Se observă că, rezultatele numerice obţinute diferă de rezultatul exact cu $\pm h$.

5.1.3 Probleme propuse

Exerciţiul 5.1.2. Să se calculeze prima şi a doua derivată folosind diferenţe finite la stânga, şi apoi la dreapta pentru următoarele funcţii:

a) $f(x) = x^3 + 2$ în punctul $x_0 = 2$ cu eroarea $h = 0.1$

b) $f(x) = \sin x$ în punctul $x_0 = \pi$ cu eroarea $h = 0.01$.

Comparaţi cu valorile exacte.

5.1.4 Implementare

A. Algoritmi

Date de intrare expresia unei funcţii, f, punctul x_0 în care dorim să calculăm derivata (respectiv a doua derivată) şi precizia h cu care dorim să calculăm această derivată.

Date de ieşire valoarea $f'(x_0)$ calculată cu diferenţe finite la dreapta (respectiv la stânga), respectiv valoarea lui $f''(x_0)$ calculată cu diferenţe finite la dreapta

Algoritm

a) derivata întâi cu diferenţe finite la dreapta:

$$f'(x) = \frac{f(x+h) - f(x)}{h}$$

b) derivata întâi cu diferenţe finite la stânga:

$$f'(x) = \frac{f(x) - f(x-h)}{h}$$

c) derivata a doua cu diferenţe finite la dreapta:

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

B. Programe MAPLE şi rezultate

Toate cele trei variante de program iau ca argument o expresie algebrică, f, o variabilă x (care poate să fie un număr, sau un simbol) şi o eroare, h.

Testăm aceste programe astfel:

```
derivdr(x^2+3*x+2,2,0.1);
derivst(x^2+3*x+2,2,0.1);
derivdr(x^2+3*x+2,2,0.01);
derivst(x^2+3*x+2,2,0.01);
derivdr(x^2+3*x+2,2,0.00001);
derivst(x^2+3*x+2,2,0.00001);
derivdoi(x^3+5*x^2+1,x,0.01);
derivdr(sin(x),0,0.001);
```

7.100000000

6.900000000

7.010000000

6.990000000

7.

6.

$6.x + 10.$

0.9999998333

5.2 Aproximarea derivatei prin derivata unei funcţii de interpolare

5.2.1 Breviar teoretic

În cazul în care expresia analitică a funcţiei f nu este cunoscută (funcţia f este dată tabelat), derivata funcţiei se mai poate obţine prin derivarea unui polinom de interpolare (Newton sau Lagrange). Derivata acestui polinom de interpolare se poate obţine fie formal, fie folosind diferenţe finite (vezi paragraful anterior).

5.2.2 Probleme rezolvate

Exerciţiul 5.2.1. Se dă funcţia tabelată

x	1	2	3	4	5
$f(x)$	1	3	-1	0	1

a) folosind un polinom de interpolare, să se găsească funcţia f precum şi derivata acesteia

b) folosind un polinom de interpolare, să se găsească derivata întâi a funcţiei f în punctul $x_0 = 2.5$.

Rezolvare

Cu ajutorul polinomului de interpolare Lagrange, obţinem o valoare aproximativă pentru funcţia f:

$$f(z) = -\frac{2}{3}z^4 + \frac{17}{2}z^3 - \frac{112}{3}z^2 + \frac{129}{2}z - 34.$$

Derivata sa formală este:

$$f'(z) = -\frac{8}{3}z^3 + \frac{51}{2}z^2 - \frac{224}{3}z + \frac{129}{2},$$

iar valoarea derivatei în punctul $x_0 = 2.5$ este -4.45833340.

Această valoare se poate obţine şi dacă folosim derivata cu diferenţe finite la dreapta a funcţiei f, de exemplu cu o eroare de 0.01:

$$f'(2.5) = \frac{f(2.5 + 0.01) - f(2.5)}{0.01} = -4.443980000.$$

5.2.3 Probleme propuse

Exerciţiul 5.2.2. Calculaţi derivata întâi a funcţiei

x	-1	0	1	2
$f(x)$	2	1	0	-1

în punctul $x_0 = 0.5$ folosind derivata polinomului de interpolare Lagrange.

Exerciţiul 5.2.3. Calculaţi derivata întâi a funcţiei

x	$\frac{\pi}{2}$	π	$\frac{3\pi}{2}$	2π	
$f(x)$	1	0	-1	0	1

în punctul $x_0 = \pi$ folosind derivata polinomului de interpolare al lui Newton cu diferenţe divizate.

5.2.4 Implementare

A. Algoritmi

Funcţia de interpolare a funcţiei tabelate f se obţine folosind unul din algoritmii prezentaţi în secţiunile 4.2, 4.1 sau 4.3. Derivata funcţiei de interpolare se obţine în Maple folosind procedura `diff`.

B. Programe MAPLE şi rezultate

Exemplificăm aplicarea acestei metode pentru funcţia tabelată

x	1	2	3	4	5
$f(x)$	1	3	-1	0	1

folosind polinomul de interpolare Lagrange.

```
> x:=[1,2,3,4,5]:  fx:=[1,3,-1,0,1]:  # functia tabelata
> f:=lagrange(x,fx,z); # polinomul de interpolare
```

$$f := -\frac{2}{3}\,z^4 + \frac{17}{2}\,z^3 - \frac{112}{3}\,z^2 + \frac{129}{2}\,z - 34$$

```
> diff(f,z); # prima derivata
```

$$-\frac{8}{3}\,z^3 + \frac{51}{2}\,z^2 - \frac{224}{3}\,z + \frac{129}{2}$$

```
> diff(f,z$2); # derivata a doua
```

$$-8\,z^2 + 51\,z - \frac{224}{3}$$

```
> diff(f,z$3); # derivata a treia
```

$$-16\,z + 51$$

Capitolul 6

Integrare numerică

Fie $f : [a, b] \to \mathbb{R}$ o funcţie integrabilă Riemann-Darboux şi nodurile echidistante $x_i = a + ih$ ale intervalului $[a, b]$, $i = \overline{0, N}$, $h = \dfrac{b-a}{N}$.

6.1 Formule de tip Newton-Cotes

6.1.1 Breviar teoretic

Formula trapezelor (pentru care $N = 1$):

$$\int_a^b f(x)dx \approx \frac{b-a}{2} \cdot [f(b) + f(a)] = \frac{h}{2} \cdot [f(b) + f(a)]. \tag{6.1}$$

Formula generală a trapezelor (formula trapezelor aplicată pe n subintervale ale intervalului dat):

$$\int_a^b f(x)dx \approx \frac{h}{2} \cdot \left[f(a) + 2 \cdot \sum_{i=1}^{n-1} f(x_i) + f(b) \right]. \tag{6.2}$$

Formula lui Simpson (pentru care $N = 2$):

$$\int_a^b f(x)dx \approx \frac{h}{3} \cdot [f(a) + 4f(a + h) + f(b)]. \tag{6.3}$$

Observaţia 6.1.1. Din teorema de medie, rezultă că formulele de integrare de tip Newton-Cotes sunt exacte pentru funcţii polinomiale de grad maxim $2N - 1$.

6.1.2 Probleme rezolvate

Exerciţiul 6.1.1. Să se calculeze

$$\int_0^1 x^3 + 1 \, dx$$

folosind:

a) metoda clasică (formală);

b) formula trapezelor;

c) formula generală a trapezelor împărţind intervalul $[0,1]$ în 2, 4, 8 subintervale;

d) formula lui Simpson.

Rezolvare

Rezultatul exact, obţinut prin metoda formală, este:

$$\int_0^1 x^3+1\,dx = \frac{x^4}{4}\Big|_0^1 + x|_0^1 = \frac{1}{4}+1 = \frac{5}{4} = 1.25.$$

Deoarece funcţia considerată este o funcţie polinomială de gradul 3, rezultă că formula trapezelor (pentru care $N=1$) nu este exactă, dar formula lui Simpson (pentru care $N=2$) este exactă. Aceasta se poate remarca şi din comparaţia rezultatelor obţinute cu rezultatul exact (adică 1.25) obţinut prin metoda formală.
Folosind formula trapezelor, obţinem:

$$\int_0^1 x^3+1\,dx = \frac{1-0}{2}[f(0)+f(1)] = \frac{1}{2}(1+2) = \frac{3}{2} = 1.5.$$

Dacă împărţim intervalul $[0,1]$ în 2 subintervale şi aplicăm formula generală a trapezelor avem:

$$\int_0^1 x^3+1\,dx = \frac{1}{4}[f(0)+2f(\frac{1}{2})+f(1)] = \frac{1}{4}(1+\frac{9}{4}+2) = \frac{21}{16} = 1.3125.$$

Pentru $n=4$ subintervale, avem:

$$\begin{aligned}\int_0^1 x^3+1\,dx =& \frac{1}{8}[f(0)+2\sum_{i=1}^{3} f(0+\frac{i}{4})+f(1)] = \\ =& \frac{1}{8}(1+2\frac{65}{64}+2\frac{9}{8}+2\frac{91}{64}+2) = \frac{81}{64} = 1.265625.\end{aligned}$$

Pentru $n=8$ subintervale avem:

$$\int_0^1 x^3+1\,dx = \frac{1}{16}[f(0)+2\sum_{i=1}^{7} f(0+\frac{i}{8})+f(1)] = \frac{1}{16}\cdot\frac{321}{16} = \frac{321}{256} = 1.25390625.$$

Folosind formula lui Simpson, obţinem:

$$\int_0^1 x^3+1\,dx = \frac{\frac{1-0}{2}}{3}(f(0)+4f(\frac{1}{2})+f(1)) = \frac{1}{6}\cdot\frac{15}{2} = \frac{15}{12} = 1.25.$$

6.1.3 Probleme propuse

Exerciţiul 6.1.2. Să se calculeze integralele următoare folosind formula generală a trapezelor pentru $n = 4$ subintervale şi formula lui Simpson:

a) $\int_0^8 x^3 + 1\, dx$

b) $\int_0^1 \cos \pi x\, dx$

c) $\int_{-2}^2 3x^2 + 1\, dx$

Comparaţi rezultatul obţinut cu soluţia exactă.

6.1.4 Implementare

A. Algoritmi

Algoritmii folosiţi pentru integrarea numerică se bazează pe formulele de integrare prezentate anterior.

B. Programe MAPLE şi rezultate

```
trapeze:=proc(f::algebraic, ab::range,N::numeric)
local i, a, b, h, xx;
a:=lhs(ab);
b:=rhs(ab);
h:=(b-a)/N;
xx:=op(indets(f, name));
evalf( h/2*( subs(xx=a, f) + 2*sum(subs(xx=a+i*h,f), i=1..N-1) +
             subs(xx=b,f)
           )
     );
end:

simpson:=proc(f::algebraic, ab::range)
local i, a, b, xx;
a:=lhs(ab);
b:=rhs(ab);
xx:=op(indets(f, name));
evalf( (b-a)/6*( subs(xx=a,f) + 4*subs(xx=a+(b-a)/2,f) +
                 subs(xx=b,f)
               )
     );
end:
```

Exemplificăm în continuare calculul integralei

$$\int_0^1 \sin x + 1\, dx.$$

```
> f:=x->sin(x)+1;
```

$$f := x \to \sin(x) + 1$$

```
> evalf(int(f(x),x=0..1));
```

$$1.459697694$$

```
> trapeze(f(x),0,1,1);
```

$$1.420735492$$

```
> trapeze(f(x),0,1,2);
```

$$1.450080515$$

```
> trapeze(f(x),0,1,4);
```

$$1.457300938$$

```
> trapeze(f(x),0,1,8);
```

$$1.459098974$$

```
> simpson(f(x),0,1);
```

$$1.459862190$$

6.2 Formule de tip Gauss

6.2.1 Breviar teoretic

Formula de integrare numerică de tip Gauss pentru două puncte ($N = 2$):

$$\int_{-1}^{1} f(x)dx \approx f\left(-\frac{\sqrt{3}}{3}\right) + f\left(\frac{\sqrt{3}}{3}\right). \tag{6.4}$$

Formula de integrare numerică de tip Gauss pentru trei puncte ($N = 3$):

$$\int_{-1}^{1} f(x)dx \approx \frac{5}{9} f\left(-\frac{\sqrt{15}}{5}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\frac{\sqrt{15}}{5}\right). \tag{6.5}$$

Formula de integrare numerică de tip Gauss pentru patru puncte ($N = 4$):

$$\begin{aligned}\int_{-1}^{1} f(x)dx \approx & \frac{18-\sqrt{30}}{36} f\left(-\frac{\sqrt{525+70\sqrt{30}}}{35}\right) + \\ & + \frac{18+\sqrt{30}}{36} f\left(-\frac{\sqrt{525-70\sqrt{30}}}{35}\right) + \\ & + \frac{18+\sqrt{30}}{36} f\left(\frac{\sqrt{525-70\sqrt{30}}}{35}\right) + \\ & + \frac{18-\sqrt{30}}{36} f\left(\frac{\sqrt{525+70\sqrt{30}}}{35}\right).\end{aligned} \tag{6.6}$$

Deoarece în formulele de integrare numerică de tip Gauss limitele de interpolare sunt -1 şi 1, în cazul în care dorim să integrăm o funcţie definită pe un interval $[a, b]$, facem schimbarea de variabilă

$$x = \frac{(b-a)t + b + a}{2} \tag{6.7}$$

care va transforma intervalul $[a, b]$ al variabilei x în intervalul $[0, 1]$ corespunzător noii variabile t. Astfel, se obţine:

$$\int_a^b f(x)dx = \frac{b-a}{2} \int_{-1}^{1} f\left(\frac{(b-a)t + a + b}{2}\right) dt. \tag{6.8}$$

Observaţia 6.2.1. Spre deosebire de formulele de integrare de tip Newton-Cotes, care sunt exacte pentru polinoame de grad maxim $2N-1$, formulele de integrare de tip Gauss sunt exacte pentru polinoame de grad maxim $2N+1$.

6.2.2 Probleme rezolvate

Exerciţiul 6.2.1. Să se calculeze $\int_{-1}^{1} x^2 + 1\, dx$ şi $\int_0^2 x^3 - 1\, dx$ folosind o formulă de cuadratură de tip Gauss.

Rezolvare

Folosim formula pentru 2 puncte, care dă rezultate exacte pentru polinoame până la gradul 5. Avem:

$$\int_{-1}^{1} x^2 + 1\, dx = \left(-\frac{\sqrt{3}}{3}\right)^2 + 1 + \left(\frac{\sqrt{3}}{3}\right)^2 + 1 = \frac{8}{3}$$

$$\int_0^2 x^3 - 1\, dx = \left(-\frac{\sqrt{3}}{3} + 1\right)^3 - 1 + \left(\frac{\sqrt{3}}{3} + 1\right)^3 - 1 = 2.$$

6.2.3 Probleme propuse

Exerciţiul 6.2.2. Să se calculeze integralele următoare folosind formula lui Gauss pentru 2, respectiv pentru 3 puncte:

a) $\int_0^8 x^3 + 1\, dx$

b) $\int_0^1 \cos \pi x\, dx$

c) $\int_{-2}^{2} 3x^2 + 1\, dx$

Comparaţi rezultatul obţinut cu soluţia exactă şi cu rezultatele obţinute în urma integrării prin metodele de tip Newton-Cotes.

6.2.4 Implementare

A. Algoritmi

Algoritmii folosiţi pentru integrarea numerică se bazează pe formulele de integrare prezentate anterior.

B. Programe MAPLE şi rezultate

Următorul program `MAPLE` ia ca argument o funcţie f, capetele intervalului pe care se integrează, a şi b, şi ordinul n (care poate fi 2, 3 sau 4).

```
gauss:=proc(f::algebraic, ab::range, n::numeric)
local xx, a, b g;
xx:=op(indets(f, name));
a:=lhs(ab);
b:=rhs(ab);
g:=subs(xx=(2*yy-a-b)/(b-a), f);
if n=2 then
  evalf( (b-a)/2*(subs(yy=-1/3*sqrt(3),g) +
    subs(yy=1/3*sqrt(3),g)) );
# secventa de program pentru n=3 si n=4 figureaza pe lista de proiecte
else ERROR(‘Ordinul n al formulei de cuadratura trebuie sa
  fie 2, 3 sau 4!‘)
fi;
end:
```

Exemplificăm acest program:

```
> gauss(x^8+1, 0..2,4);
```

$$58.87727878$$

```
> evalf(int(x^8+1, x=0..2));
```

$$58.88888889$$

```
> f:=x->sin(x)+1;
```

$$f := x \to \sin(x) + 1$$

```
> gauss(f(x),0..1,2);
```

$$0.6598837714$$

```
> gauss(f(x),0..1,3);
```

$$0.6159146322$$

```
> gauss(f(x),0..1,4);
```

$$1.234908772$$

Capitolul 7

Ecuaţii diferenţiale

Vom prezenta în continuare metode numerice de rezolvare a problemelor cu date iniţiale pentru ecuaţii diferenţiale de ordinul întâi, precum şi metode numerice de rezolvare a unor probleme la limită liniare pentru ecuaţii diferenţiale de ordinul al doilea.

Rezolvarea problemelor cu date iniţiale pentru ecuaţii diferenţiale de ordinul întâi

Fie problema cu date iniţiale

$$\begin{cases} y' & = & f(x,y) \\ y(x_0) & = & y_0 \end{cases} \tag{7.1}$$

unde $f : (\alpha, \beta) \times (\gamma, \delta) \to \mathbb{R}^1$ este o funcţie de clasă $\mathcal{C}^1$ şi $x_0 \in (\alpha, \beta)$, $y_0 \in (\gamma, \delta)$.

Pentru rezolvarea numerică a acestei probleme vom trece în vedere:

- metode unipas: metoda diferenţelor finite, metoda lui Taylor, metoda Runge-Kutta;
- metode multipas: metoda Adams-Bashforth, metoda Adams-Moulton, metoda predictor-corector

Considerăm punctele

$$x_{i+1} = x_i + h = x_0 + (i+1)h \ , \ i = 0, 1, \ldots, N-1 \tag{7.2}$$

$h > 0$, şi admitem că $x_i \in (\alpha, \beta)$ pentru $i = \overline{0, N-1}$. Notăm cu $a = x_0$ şi $b = x_N = x_0 + Nh$.

7.1 Metoda diferenţelor finite

7.1.1 Breviar teoretic

Formula lui Euler cu diferenţe finite înainte:

$$y(x_{i+1}) = y(x_i) + hf(x_i, y(x_i)),\ i = \overline{0, N-1} \tag{7.3}$$

Formula lui Euler cu diferenţe finite înapoi:

$$y(x_{i+1}) = y(x_i) + h \cdot f(x_{i+1}, y(x_{i+1})),\ i = \overline{0, N-1} \tag{7.4}$$

Formula punctului de mijloc:

$$y(x_{i+1}) = y(x_{i-1}) + 2h \cdot f(x_i, y(x_i)),\ i = \overline{1, N-1} \tag{7.5}$$

unde $y(x_1)$ trebuie găsită cu altă metodă.

7.1.2 Probleme rezolvate

Exerciţiul 7.1.1. Să se rezolve problema cu date iniţiale

$$\begin{cases} y'(x) = 1 + x^2 \\ y(0) = 2 \end{cases}$$

pe intervalul $[0, 1]$, folosind una formulele lui Euler, pentru $h = 0.1$.

Rezolvare

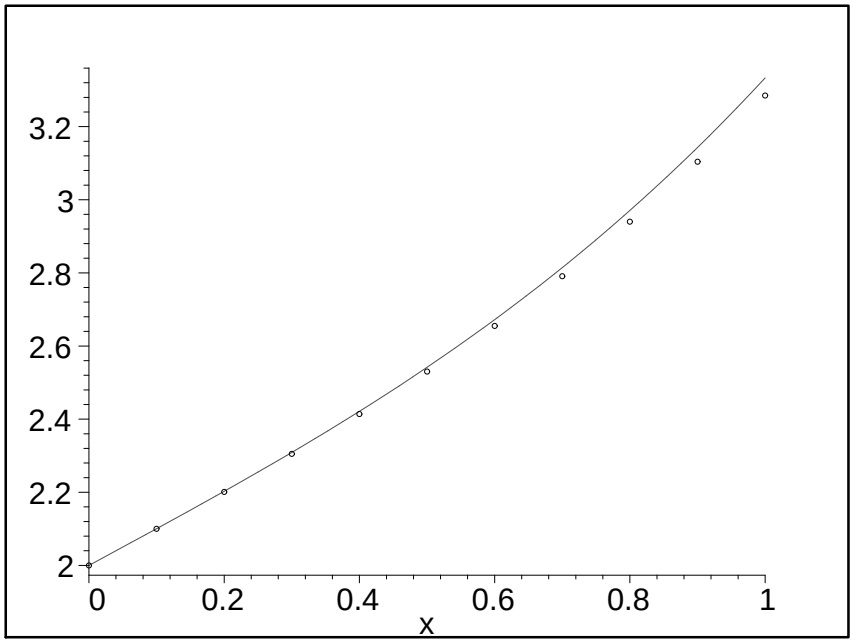
Folosim formula lui Euler cu diferenţe finite înainte. Aplicând formula (7.3) pentru $x_0 = 0$ şi $y_0 = y(0) = 2$, obţinem succesiv:

$$\begin{array}{ll} x_1 = 0.1000000000 & y_1 = 2.100000000 \\ x_2 = 0.2000000000 & y_2 = 2.201000000 \\ x_3 = 0.3000000000 & y_3 = 2.305000000 \\ x_4 = 0.4000000000 & y_4 = 2.414000000 \\ x_5 = 0.5000000000 & y_5 = 2.530000000 \\ x_6 = 0.6000000000 & y_6 = 2.655000000 \\ x_7 = 0.7000000000 & y_7 = 2.791000000 \\ x_8 = 0.8000000000 & y_8 = 2.940000000 \\ x_9 = 0.9000000000 & y_9 = 3.104000000 \\ x_{10} = 1 & y_{10} = 3.285000000 \end{array}$$

Soluţia exactă a problemei cu date iniţiale considerată este

$$y(x) = \frac{x^3}{3} + x + 2.$$

Reprezentăm grafic punctele obţinute folosind metoda lui Euler cu diferenţe finite înainte şi graficul soluţiei exacte a problemei considerate:



7.1.3 Probleme propuse

Exerciţiul 7.1.2. Folosind metoda lui Euler cu diferenţe finite înainte, să se determine soluţia aproximativă a problemei

$$\begin{cases} y'(x) = y(x) - x^2 + x \\ y(1) = 2 \end{cases}$$

împărţind intervalul $[1, 2]$ în $N = 4$. Comparaţi cu rezultatul exact.

7.1.4 Implementare

A. Algoritmi

Date de intrare: o ecuaţie $y'(x) = f(x, y(x))$, valorile iniţiale x_0 şi y_0, un interval $[a, b]$ şi numărul de puncte în care se împarte intervalul, N

Date de ieşire: soluţia aproximativă a problemei cu date iniţiale, dată sub forma unui şir de puncte, rez, folosind metoda lui Euler cu diferenţe finite înainte

Algoritmul constă în următorii paşi:

1. definirea funcţiei f
2. aplicarea formulei lui Euler cu diferenţe finite înainte:

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{0, N-1}$

   $y_{i+1} = y_i + h\, f(x_i, y_i)$

   $x_{i+1} = x_i + h$

   adaugă punctul (x_{i+1}, y_{i+1}) la liste rez

B. Programe MAPLE şi rezultate

```
dfieuler:=proc(eq::equation, x0::numeric, y0::numeric, ab::range,
  N::integer)
  local f,varx,h,rez,x1,y1,i;
  if StringTools[Has](convert(lhs(eq),string),"diff") then
    f:=rhs(eq);
  elif StringTools[Has](convert(rhs(eq),string),"diff") then
    f:=lhs(eq);
  else
    ERROR (‘ecuatia trebuie sa fie de tipul diff(y(x),x)=f(x,y)‘);
  fi;
  varx:=[op(indets(eq, name))];
  if nops(varx)=1 then varx:=[op(varx),0]; fi;
  h:=(rhs(ab)-lhs(ab))/N;
  rez:=[[x0,y0]];
  x1:=x0; y1:=y0;
  for i from 1 to N do
    y1:=evalf(y1+h*subs(varx[1]=x1,varx[2]=y1,f));
    x1:=evalf(x1+h);
    rez:=[op(rez),[x1,y1]];
  od;
RETURN(rez);
end:

debug(dfieuler):

dfieuler(diff(f(x),x)=x^2+1,0,2,0..1,10);
```

```
> dfieuler(diff(f(x),x)=x^2+1,0,2,0..1,10);
```

```
{--> enter dfieuler, args = diff(f(x),x) = x^2+1, 0, 2, 0 .. 1, 10
```

$$f := x^2 + 1$$

$$varx := [x]$$

$$varx := [x, 0]$$

$$h := \frac{1}{10}$$

$$rez := [[0, 2]]$$

$$x1 := 0$$

$$y1 := 2$$

$$y1 := 2.100000000$$

$$x1 := 0.1000000000$$

$$rez := [[0, 2], [0.1000000000, 2.100000000]]$$

$$y1 := 2.201000000$$

$$x1 := 0.2000000000$$

$$rez := [[0, 2], [0.1000000000, 2.100000000], [0.2000000000, 2.201000000]]$$

$$y1 := 2.305000000$$

$$x1 := 0.3000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000]]$

$$y1 := 2.414000000$$
$$x1 := 0.4000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000],\ [0.4000000000,\ 2.414000000]]$

$$y1 := 2.530000000$$
$$x1 := 0.5000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000],\ [0.4000000000,\ 2.414000000],$
$[0.5000000000,\ 2.530000000]]$

$$y1 := 2.655000000$$
$$x1 := 0.6000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000],\ [0.4000000000,\ 2.414000000],$
$[0.5000000000,\ 2.530000000],\ [0.6000000000,\ 2.655000000]]$

$$y1 := 2.791000000$$
$$x1 := 0.7000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000],\ [0.4000000000,\ 2.414000000],$
$[0.5000000000,\ 2.530000000],\ [0.6000000000,\ 2.655000000],$
$[0.7000000000,\ 2.791000000]]$

$$y1 := 2.940000000$$
$$x1 := 0.8000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000],\ [0.4000000000,\ 2.414000000],$
$[0.5000000000,\ 2.530000000],\ [0.6000000000,\ 2.655000000],$
$[0.7000000000,\ 2.791000000],\ [0.8000000000,\ 2.940000000]]$

$$y1 := 3.104000000$$
$$x1 := 0.9000000000$$

$rez := [[0,\ 2],\ [0.1000000000,\ 2.100000000],\ [0.2000000000,\ 2.201000000],$
$[0.3000000000,\ 2.305000000],\ [0.4000000000,\ 2.414000000],$
$[0.5000000000,\ 2.530000000],\ [0.6000000000,\ 2.655000000],$
$[0.7000000000,\ 2.791000000],\ [0.8000000000,\ 2.940000000],$
$[0.9000000000,\ 3.104000000]]$

$$y1 := 3.285000000$$
$$x1 := 1.000000000$$

rez := [[0, 2], [0.1000000000, 2.100000000], [0.2000000000, 2.201000000],
[0.3000000000, 2.305000000], [0.4000000000, 2.414000000],
[0.5000000000, 2.530000000], [0.6000000000, 2.655000000],
[0.7000000000, 2.791000000], [0.8000000000, 2.940000000],
[0.9000000000, 3.104000000], [1.000000000, 3.285000000]]

```
<-- exit dfieuler (now at top level) = [[0, 2], [.1000000000,
2.100000000], [.2000000000, 2.201000000], [.3000000000, 2.305000000],
[.4000000000, 2.414000000], [.5000000000, 2.530000000], [.6000000000,
2.655000000], [.7000000000, 2.791000000], [.8000000000, 2.940000000],
[.9000000000, 3.104000000], [1.000000000, 3.285000000]]}
```

[[0, 2], [0.1000000000, 2.100000000], [0.2000000000, 2.201000000],
[0.3000000000, 2.305000000], [0.4000000000, 2.414000000],
[0.5000000000, 2.530000000], [0.6000000000, 2.655000000],
[0.7000000000, 2.791000000], [0.8000000000, 2.940000000],
[0.9000000000, 3.104000000], [1.000000000, 3.285000000]]

Prezentăm în continuare aproximarea soluţiilor problemei

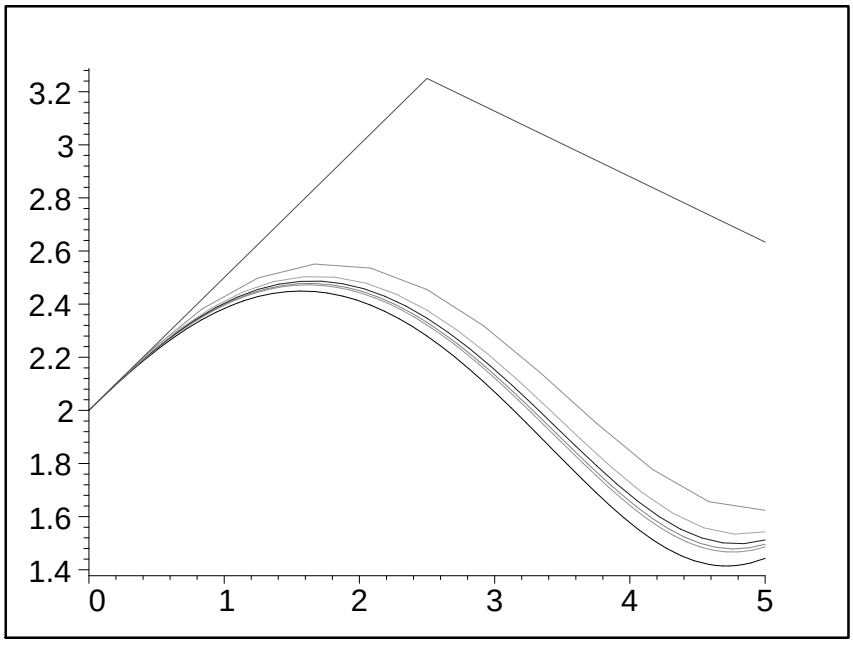
$$\begin{cases} y'(x) = \dfrac{\cos x}{y(x)} \\ y(0) = 2 \end{cases}$$

a cărei soluţie exactă este $y(x) = \sqrt{2\sin x + 4}$, pe intervalul $[0, 5]$, pentru diverse valori ale parametrului N:

```
> dsolve({diff(f(x),x)=cos(x)/f(x),f(0)=2},f(x));
```

$$\mathrm{f}(x) = \sqrt{2\sin(x) + 4}$$

```
> p1:=plot([seq(dfieuler(diff(f(x),x)=cos(x)/f,0,2,0..5,N),N=2..52,10)]
> ):
> p2:=plot((2*sin(x)+4)^(1/2),x=0..5, color=black, thickness=2):
> display(p1,p2);
```



7.2 Metoda lui Taylor

7.2.1 Breviar teoretic

Metoda lui Taylor de ordinul n pentru rezolvarea problemei cu date iniţiale (7.1), presupune găsirea şirului $(Y_n)_n$, unde:

$$\left\{ \begin{array}{lcl} Y_0 & = & y(x_0) \\ Y_{i+1} & = & Y_i + h \cdot T_n(x_i, Y_i, h) \quad \text{pentru} \quad i = 0, 1, \ldots, N-1, \end{array} \right. \tag{7.6}$$

iar

$$T_n(x_i, y_i, h) = f(x_i, y_i) + \frac{h}{2!}\, f'(x_i, y_i) + \ldots + \frac{h^{n-1}}{n!}\, f^{(n-1)}(x_i, y_i). \tag{7.7}$$

Pentru $n = 1$, metoda lui Taylor devine metoda lui Euler.

Pentru $n = 2$, obţinem următoarea formulă:

$$T_2(x_i, y_i, h) = f(x_i, y_i) + \frac{h}{2}\left[\frac{\partial f}{\partial x}(x_i, y_i) + \frac{\partial f}{\partial y}(x_i, y_i) \cdot f(x_i, y_i)\right] \tag{7.8}$$

iar pentru $n = 3$ obţinem:

$$\begin{aligned} T_3(x_i, y_i, h) =& f(x_i, y_i) + \frac{h}{2}\left[\frac{\partial f}{\partial x}(x_i, y_i) + \frac{\partial f}{\partial y}(x_i, y_i) \cdot f(x_i, y_i)\right] + \\ &+ \frac{h^2}{6}\left[\frac{\partial^2 f}{\partial x^2}(x_i, y_i) + 2f(x_i, y_i) \cdot \frac{\partial^2 f}{\partial x \partial y}(x_i, y_i) + \right. \\ &+ f^2(x_i, y_i) \cdot \frac{\partial^2 f}{\partial y^2}(x_i, y_i) + \frac{\partial f}{\partial x}(x_i, y_i) \cdot \frac{\partial f}{\partial y}(x_i, y_i) + \\ &+ \left. f(x_i, y_i) \cdot \frac{\partial^2 f}{\partial y^2}(x_i, y_i)\right]. \end{aligned} \tag{7.9}$$

7.2.2 Probleme rezolvate

Exerciţiul 7.2.1. Folosind metoda lui Taylor de ordinul 2, să se rezolve problema cu date iniţiale următoare

$$\left\{ \begin{array}{l} y'(x) = 1 + x^2 \\ y(0) = 2 \end{array} \right.$$

pe intervalul $[0, 1]$, pentru $h = 0.1$.

Rezolvare

În cazul nostru, $f(x, y) = 1 + x^2$, şi deci

$$T_2(x, y, h) = 1 + x^2 + \frac{h}{2} \cdot 2x.$$

Astfel, termenul general are forma:

$$y_{i+1} = y_i + h \cdot (1 + x_i^2 + hx_i)$$

Plecând de la $x_0 = 0$ şi $y_0 = y(0) = 2$ şi aplicând formula lui Taylor de ordinul 2, obţinem succesiv:

$$\begin{aligned}
x_1 &= 0.1000000000 & y_1 &= 2.120000000\\
x_2 &= 0.2000000000 & y_2 &= 2.232706000\\
x_3 &= 0.3000000000 & y_3 &= 2.350316071\\
x_4 &= 0.4000000000 & y_4 &= 2.475125294\\
x_5 &= 0.5000000000 & y_5 &= 2.609481021\\
x_6 &= 0.6000000000 & y_6 &= 2.755790277\\
x_7 &= 0.7000000000 & y_7 &= 2.916529651\\
x_8 &= 0.8000000000 & y_8 &= 3.094257797\\
x_9 &= 0.9000000000 & y_9 &= 3.291630711\\
x_{10} &= 1.000000000 & y_{10} &= 3.511419969
\end{aligned}$$

7.2.3 Probleme propuse

Exerciţiul 7.2.2. Folosind metoda lui Taylor de ordinul 2 şi 3, să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'(x) = -x^2 + x + 1 \\ y(1) = 2 \end{cases}$$

împărţind intervalul $[1, 2]$ în $N = 4$. Comparaţi cu rezultatul exact.

7.2.4 Implementare

A. Algoritmi

Date de intrare: o ecuatie $y'(x) = f(x, y(x))$, valorile initiale x_0 şi y_0, un interval $[a, b]$ şi numărul de puncte în care se împarte intervalul, N

Date de ieşire: soluţia aproximativă a problemei cu date iniţiale, dată sub forma unui şir de puncte, rez

Algoritmul constă în următorii paşi:

1. definirea funcţiei f
2. aplicarea formulei lui Taylor de ordinul 2 sau 3:

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{0, N-1}$

   $y_{i+1} = y_i + h\,T(x_i, y_i, h)$, unde T este dat de (7.8) sau (7.9)

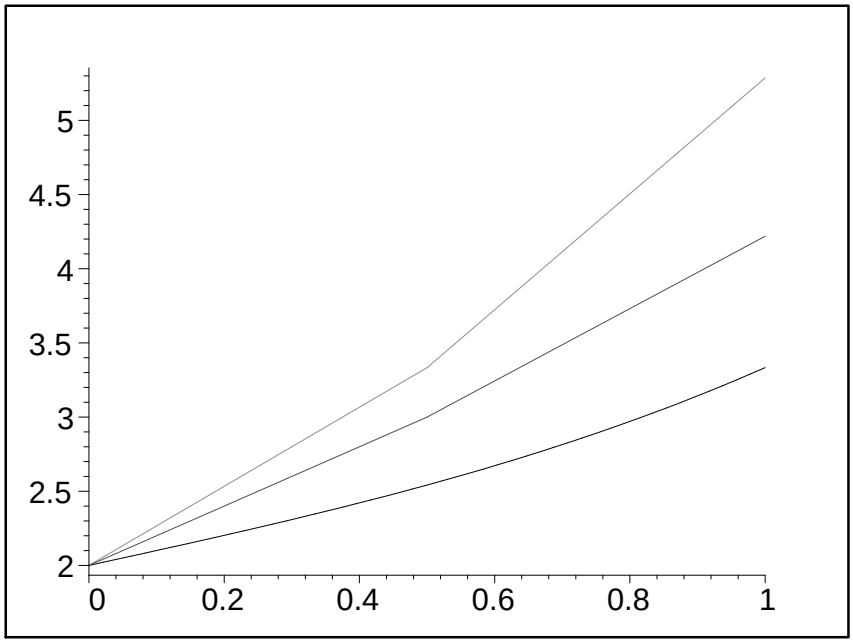
   $x_{i+1} = x_i + h$

   adaugă punctul (x_{i+1}, y_{i+1}) la liste rez

B. Programe MAPLE şi rezultate

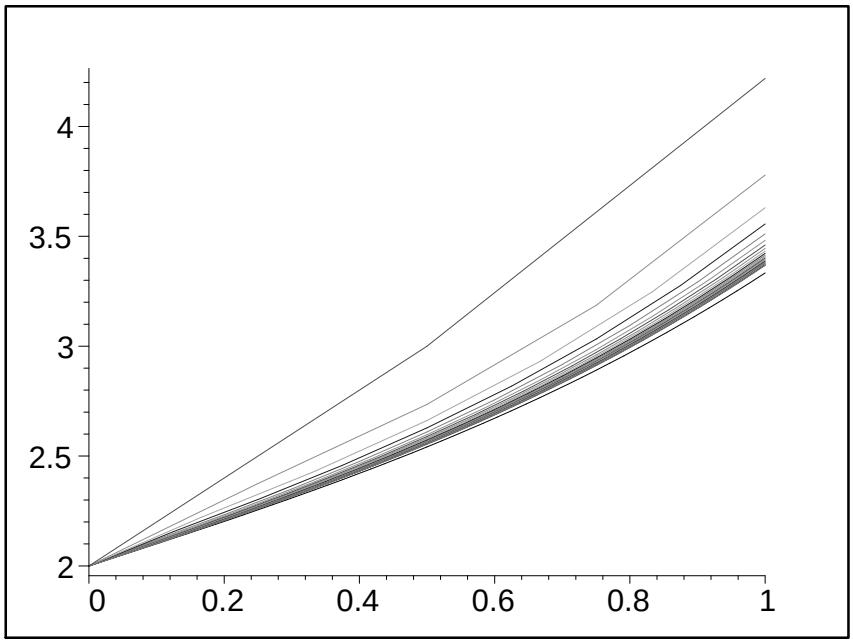
```
taylor2:=proc(eq::equation, x0::numeric, y0::numeric,ab::range,
  N::integer)
local h,f,varx,fx,fy,y1,x1,i,rez;
if StringTools[Has](convert(lhs(eq),string),"diff") then
    f:=rhs(eq);
  elif StringTools[Has](convert(rhs(eq),string),"diff") then
    f:=lhs(eq);
  else
    ERROR (‘ecuatia trebuie sa fie de tipul diff(y(x),x)=f(x,y)‘);
  fi;
varx:=[op(indets(eq, name))];
if nops(varx)=1 then varx:=[op(varx),0]; fi;
fx:=diff(f,x);
fy:=diff(f,y);
h:=(rhs(ab)-lhs(ab))/N;
y1:=y0;
x1:=x0;
rez:=[[x1,y1]];
for i from 1 to N do
  y1:=evalf(
      y1+
      h*(
       subs(varx[1]=x1,varx[2]=y1,f) +
       h/2*(
        subs(varx[1]=x1,varx[2]=y1,fx) +
        subs(varx[1]=x1,varx[2]=y1,fy) *
        subs(varx[1]=x1,varx[2]=y1,f)
        )
       )
      );
  x1:=evalf(x1+h);
  rez:=[op(rez),[x1,y1]];
od;
RETURN(rez);
end:

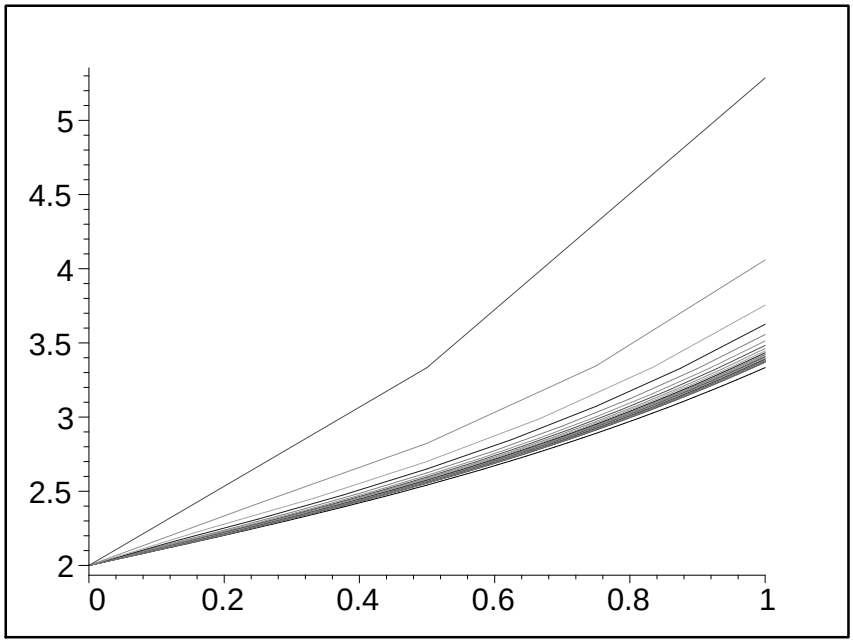
with(plots):
p1:=plot([taylor2(diff(f(x),x)=x^2+1,0,2,0..1,2),
          taylor3(diff(f(x),x)=x^2+1,0,2,0..1,2)], 0..1):
p2:=plot(1/3*x^3+x+2, x=0..1,color=black, thickness=2):
display(p1,p2);
```



În continuare prezentăm pe acelaşi sistem de coordonate soluţia exactă a problemei rezolvate anterior şi aproximarea soluţiei cu ajutorul formulei lui Taylor de ordinul 2, respectiv 3, pentru diferite valori ale parametrului N.

```
pt2:=plot([seq(taylor2(diff(f(x),x)=x^2+1,0,2,0..1,N),
           N=2..52,2)]):
pt3:=plot([seq(taylor3(diff(f(x),x)=x^2+1,0,2,0..1,N),
           N=2..52,2)]):
display(pt2,p2);display(pt3,p2);
```





7.3 Metoda Runge-Kutta

7.3.1 Breviar teoretic

Metoda Runge-Kutta de ordinul n pentru rezolvarea problemei cu date iniţiale (7.1) presupune găsirea şirului $(Y_n)_n$ care aproximează soluţia problemei date.

Pentru $n = 2$, obţinem:

$$Y_{i+1} = Y_i + \frac{h}{2}\left[f(x_i, Y_i) + f(x_{i+1}, Y_i + hf(x_i, Y_i))\right] \tag{7.10}$$

cu $Y_0 = y(x_0)$.

Pentru $n = 3$, obţinem

$$Y_{i+1} = Y_i + \frac{1}{6} \cdot (k_1 + 4k_2 + k_3) \tag{7.11}$$

cu $Y_0 = y(x_0)$ şi

$$\begin{aligned} k_1 &= h \cdot f(x_i, Y_i) \\ k_2 &= h \cdot f\left(x_i + \frac{h}{2}, Y_i + \frac{k_1}{2}\right) \\ k_3 &= h \cdot f(x_i + h, Y_i - k_1 + 2k_2) \end{aligned} \tag{7.12}$$

Pentru $n = 4$, obţinem

$$Y_{i+1} = Y_i + \frac{1}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4) \tag{7.13}$$

cu $Y_0 = y(x_0)$ şi

$$\begin{aligned} k_1 &= h \cdot f(x_i, Y_i) \\ k_2 &= h \cdot f\left(x_i + \frac{h}{2}, Y_i + \frac{k_1}{2}\right) \\ k_3 &= h \cdot f\left(x_i + \frac{h}{2}, Y_i + \frac{k_2}{2}\right) \\ k_4 &= h \cdot f(x_i + h, Y_i + k_3) \end{aligned} \tag{7.14}$$

7.3.2 Probleme rezolvate

Exerciţiul 7.3.1. Să se rezolve problema cu date iniţiale următoare

$$\begin{cases} y'(x) = 1 + x^2 \\ y(0) = 2 \end{cases}$$

pe intervalul $[0, 1]$, folosind una din metodele Runge-Kutta prezentate mai sus, pentru $h = 0.1$.

Rezolvare
Folosim metoda Runge-Kutta de ordinul 2.

Plecând de la $x_0 = 0$ şi $y_0 = y(0) = 2$ şi aplicând formula (7.10) obţinem:

$$\begin{aligned} x_1 &= 0.1000000000 & y_1 &= 2.100500000 \\ x_2 &= 0.2000000000 & y_2 &= 2.203000000 \\ x_3 &= 0.3000000000 & y_3 &= 2.309500000 \\ x_4 &= 0.4000000000 & y_4 &= 2.422000000 \\ x_5 &= 0.5000000000 & y_5 &= 2.542500000 \\ x_6 &= 0.6000000000 & y_6 &= 2.673000000 \\ x_7 &= 0.7000000000 & y_7 &= 2.815500000 \\ x_8 &= 0.8000000000 & y_8 &= 2.972000000 \\ x_9 &= 0.9000000000 & y_9 &= 3.144500000 \\ x_{10} &= 1.000000000 & y_{10} &= 3.335000000. \end{aligned}$$

7.3.3 Probleme propuse

Exerciţiul 7.3.2. Folosind metoda Runge-Kutta de ordinul 2, 3, 4, să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'(x) = x + 1 \\ y(1) = 2 \end{cases}$$

împărţind intervalul $[1, 2]$ în $N = 4$. Comparaţi cu soluţia exactă.

7.3.4 Implementare

A. Algoritmi

Date de intrare: o ecuaţie $y'(x) = f(x, y(x))$, valorile iniţiale x_0 şi y_0, un interval $[a, b]$ şi numărul de puncte în care se împarte intervalul, N

Date de ieşire: soluţia aproximativă a problemei cu date iniţiale, dată sub forma unui şir de puncte, rez, obţinută cu metoda Runge-Kutta

Algoritmul constă în următorii paşi:

1. definirea funcţiei f
2. a) metoda Runge-Kutta de ordinul 2:

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{0, N-1}$

   $y_{i+1} = y_i + \frac{h}{2}\left[f(x_i, y_i) + f(x_{i+1}, y_i + hf(x_i, y_i))\right]$

   $x_{i+1} = x_i + h$

   adaugă punctul (x_{i+1}, y_{i+1}) la liste rez

   b) metoda Runge-Kutta de ordinul 3

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{0, N-1}$

   calculează coeficienţii k_1, k_2, k_3 pe baza formulei (7.12)

   $y_{i+1} = y_i + \frac{1}{6} \cdot (k_1 + 4k_2 + k_3)$

   adaugă punctul (x_{i+1}, y_{i+1}) la liste rez

   c) metoda Runge-Kutta de ordinul 4

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{0, N-1}$

   calculează coeficienţii k_1, k_2, k_3, k_4 pe baza formulei (7.14)

   $y_{i+1} = y_i + \frac{1}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4)$

   adaugă punctul (x_{i+1}, y_{i+1}) la liste rez

B. Programe MAPLE şi rezultate

```
rungekutta2:=proc(eq::equation, x0::numeric, y0::numeric,
  ab::range, N::integer)
local f,varx,h,rez,x1,y1,i;
  if StringTools[Has](convert(lhs(eq),string),"diff") then
    f:=rhs(eq);
  elif StringTools[Has](convert(rhs(eq),string),"diff") then
```

```
    f:=lhs(eq);
  else
    ERROR (‘ecuatia trebuie sa fie de tipul diff(y(x),x)=f(x,y)‘);
  fi;
  varx:=[op(indets(eq, name))];
  if nops(varx)=1 then varx:=[op(varx),0]; fi;
  h:=(rhs(ab)-lhs(ab))/N;
  rez:=[[x0,y0]];
  x1:=x0; y1:=y0;
  for i from 1 to N do
    y1:=evalf(y1 + h/2*(
         subs(varx[1]=x1,varx[2]=y1,f) +
         subs(varx[1]=x1+h,
              varx[2]=y1+h*subs(varx[1]=x1,varx[2]=y1,f), f)
        ));
    x1:=evalf(x1+h);
    rez:=[op(rez),[x1,y1]];
  od;
RETURN(rez);
end:
```

```
> rungekutta2(diff(f(x),x)=x^2+1,1,2,0..1,3);
```

[[1, 2], [1.333333333, 2.796296296], [1.666666666, 3.888888888], [1.999999999, 5.351851850]]

```
> rungekutta3(diff(f(x),x)=x^2+1,1,2,0..1,3);
```

[[1, 2], [1.333333333, 2.790123457], [1.666666666, 3.876543210], [1.999999999, 5.333333332]]

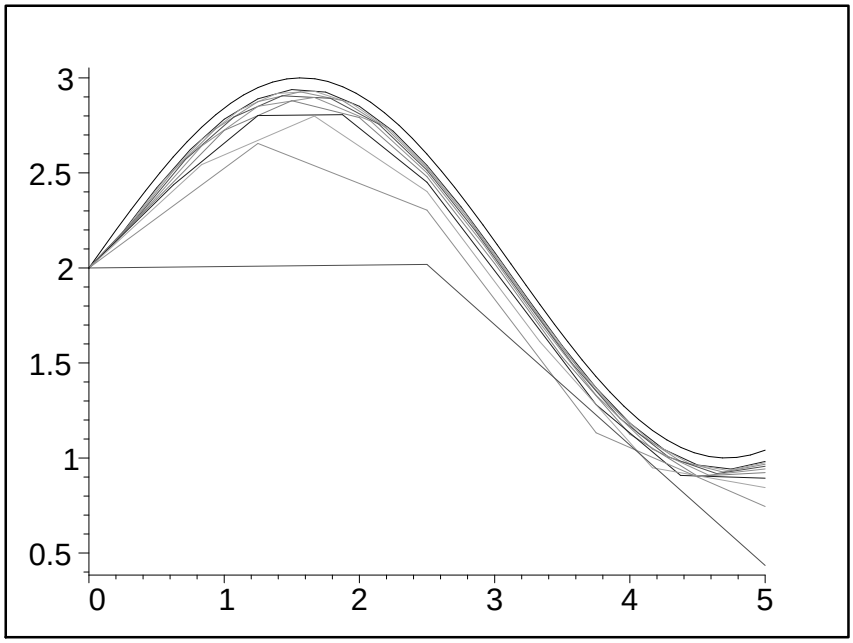
```
> rungekutta4(diff(f(x),x)=x^2+1,0,2,0..1,3);
```

[[0, 2], [0.3333333333, 2.345679012], [0.6666666666, 2.765432099], [0.9999999999, 3.333333334]]

Prezentăm în continuare diferite aproximări obţinute cu metoda Runge-Kutta de ordinul 4, pentru soluţia problemei

$$\begin{cases} y'(x) = \cos x \\ y(0) = 2. \end{cases},$$

folosind diverse valori ale parametrului N. Soluţia exactă a acestei probleme este $y(x) = \sin x + 2$.



7.4 Metoda Adams-Bashforth

7.4.1 Breviar teoretic

Metoda Adams-Bashforth este o metodă multipas de rezolvare a problemei (7.1), care pentru calculul valorii y_{i+1} foloseşte valorile obţinute în punctele x_i, ..., x_{i-k}.

Pentru $k = 1$, se obţine metoda Adams-Bashforth de ordinul 2, a cărui termen general y_{i+1} se găseşte din relaţia:

$$y_{i+1} = y_i + \frac{h}{2} \cdot [3\, f(x_i, y_i) - f(x_{i-1}, y_{i-1})] \tag{7.15}$$

unde $y_0 = y(x_0)$. Pentru ca relaţia (7.15) să aibe sens, trebuie ca indicii să fie toţi pozitivi, i.e. $i - 1 \geq 0$ deci $i \geq 1$. Însă aceasta ne permite să aflăm valorile aproximative ale soluţiei începând cu y_2. Valoarea y_1 trebuie calculată prin altă metodă (de exemplu, folosind metoda lui Euler cu diferenţe finite înainte).

Pentru $k = 2$, se obţine metoda Adams-Bashforth de ordinul 3, a cărui termen general y_{i+1} se găseşte din relaţia:

$$y_{i+1} = y_i + \frac{h}{12} \cdot \{23\, f(x_i, y_i) - 16\, f(x_{i-1}, y_{i-1}) + 5\, f(x_{i-2}, y_{i-2})\}. \tag{7.16}$$

unde $y_0 = y(x_0)$. Pentru ca relaţia (7.16) să aibe sens, trebuie ca indicii să fie toţi pozitivi, deci $i \geq 2$. Valorile y_1 şi y_2 trebuie calculate prin altă metodă (de exemplu, folosind metoda lui Euler cu diferenţe finite înainte).

Pentru $k = 3$, se obţine metoda Adams-Bashforth de ordinul 4, a cărui termen general y_{i+1} se găseşte din relaţia:

$$y_{i+1} = y_i + \frac{h}{24} \cdot \{55\, f(x_i, y_i) - 59\, f(x_{i-1}, y_{i-1}) + 37\, f(x_{i-2}, y_{i-2}) - 9\, f(x_{i-3}, y_{i-3})\}. \tag{7.17}$$

unde $y_0 = y(x_0)$. Pentru ca relaţia (7.17) să aibe sens, trebuie ca indicii să fie toţi pozitivi, deci $i \geq 3$. Valorile y_1, y_2 şi y_3 trebuie calculate prin altă metodă (de exemplu, folosind metoda lui Euler cu diferenţe finite înainte).

Pentru $k = 4$, se obţine metoda Adams-Bashforth de ordinul 5, a cărui termen general y_{i+1} se găseşte din relaţia:

$$y_{i+1} = y_i + \frac{h}{720} \cdot \{1901\, f(x_i, y_i) - 2774\, f(x_{i-1}, y_{i-1}) + 2616\, f(x_{i-2}, y_{i-2}) - 1274\, f(x_{i-3}, y_{i-3}) + 251\, f(x_{i-4}, y_{i-4})\}. \tag{7.18}$$

unde $y_0 = y(x_0)$. Pentru ca relaţia (7.18) să aibe sens, trebuie ca indicii să fie toţi pozitivi, deci $i \geq 4$. Valorile y_1, y_2, y_3 şi y_4 trebuie calculate prin altă metodă (de exemplu, folosind metoda lui Euler cu diferenţe finite înainte).

7.4.2 Probleme rezolvate

Exerciţiul 7.4.1. Folosind una din metodele Adams-Bashforth prezentate mai sus, să se rezolve următoarea problemă cu date iniţiale

$$\begin{cases} y'(x) = 1 + x^2 \\ y(0) = 2 \end{cases}$$

pe intervalul $[0, 1]$, pentru $h = 0.1$.

Rezolvare
Folosim metoda Adams-Bashforth de ordinul 2.

Avem $x_0 = 0$ şi $y_0 = 2$. Cu ajutorul formulei lui Euler cu diferenţe finite înainte, obţinem $x_1 = 0.1000000000$ şi $y_1 = 2.100000000$

Aplicând formula (7.15) obţinem:

$$\begin{aligned} x_2 &= 0.2000000000 & y_2 &= 2.201500000 \\ x_3 &= 0.3000000000 & y_3 &= 2.307000000 \\ x_4 &= 0.4000000000 & y_4 &= 2.418500000 \\ x_5 &= 0.5000000000 & y_5 &= 2.538000000 \\ x_6 &= 0.6000000000 & y_6 &= 2.667500000 \\ x_7 &= 0.7000000000 & y_7 &= 2.809000000 \\ x_8 &= 0.8000000000 & y_8 &= 2.964500000 \\ x_9 &= 0.9000000000 & y_9 &= 3.136000000 \\ x_{10} &= 1.000000000 & y_{10} &= 3.325500000. \end{aligned}$$

7.4.3 Probleme propuse

Exerciţiul 7.4.2. Folosind metoda Adams-Bashforth de ordinul 2 şi 3, să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'(x) = y(x) - x^2 + x \\ y(1) = 2 \end{cases}$$

împărţind intervalul $[1, 2]$ în $N = 4$. Comparaţi cu soluţia exactă.

7.4.4 Implementare

A. Algoritmi

Date de intrare: o ecuaţie $y'(x) = f(x, y(x))$, valorile iniţiale x_0 şi y_0, un interval $[a, b]$, ordinul metodei, m, şi numărul N de puncte în care se împarte intervalul

Date de ieşire: soluţia aproximativă a problemei cu date iniţiale, dată sub forma unui şir de puncte, rez, obţinută cu ajutorul metodei Adams-Bashforth

Algoritmul constă în următorii paşi:

1. identificarea funcţiei f
   - $h = \dfrac{b-a}{N}$
2. găsirea soluţiei aproximative

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{1, m-1}$

   $y_{i+1} = y_i + h\, f(x_i, y_i)$

   $x_{i+1} = x_i + h$

   adaugă punctul (x_{i+1}, y_{i+1}) la lista rez

   pentru $i = \overline{m, N}$

   obţine y_{i+1} din formula corespunzătoare ordinului((7.15), (7.16), (7.17) sau (7.18))

   $x_{i+1} = x_i + h$

   adaugă punctul (x_{i+1}, y_{i+1}) la lista rez

B. Programe MAPLE şi rezultate

```
adamsbashforth:=proc(eq::equation, x0::numeric, y0::numeric,
  ab::range, ordin::integer, N::integer)
local h,f,varx,y1,x1,i,rez;
if StringTools[Has](convert(lhs(eq),string),"diff") then
    f:=rhs(eq);
  elif StringTools[Has](convert(rhs(eq),string),"diff") then
    f:=lhs(eq);
  else
    ERROR (‘ecuatia trebuie sa fie de tipul
      diff(y(x),x)=f(x,y)‘);
  fi;
if ordin>N then
  WARNING(‘ordinul metodei,%1, trebuie sa fie mai mic decat
    numarul de puncte ales, %2‘, ordin, N)
fi;
varx:=[op(indets(eq, name))];
if nops(varx)=1 then varx:=[op(varx),0]; fi;
```

```
h:=(rhs(ab)-lhs(ab))/N;
y1:=evalf(y0);
x1:=evalf(x0);
rez:=[[x1,y1]];
if evalb(ordin in {2,3,4,5}) then
  for i from 1 to ordin-1 do
    y1:=evalf(y1+h*subs(varx[1]=rez[nops(rez)][1],
                        varx[2]=rez[nops(rez)][2],f));
    x1:=evalf(x1+h);
    rez:=[op(rez),[x1,y1]];
  od;
  if ordin=2 then
    for i from 2 to N do
      y1:=evalf(
          y1+h/2*( 3*subs(varx[1]=rez[nops(rez)][1],
                          varx[2]=rez[nops(rez)][2],f) -
                     subs(varx[1]=rez[nops(rez)-1][1],
                          varx[2]=rez[nops(rez)-1][2],f)
                 ));
      x1:=x1+h;
      rez:=[op(rez),[x1,y1]];
    od;
#secventa de program pentru n=3,4,5 constituie tema de proiect
else
  ERROR(‘ordinul metodei Adams-Bashforth poate fi 2, 3 sau 4
    sau 5‘);
fi;
RETURN(rez);
end:
```

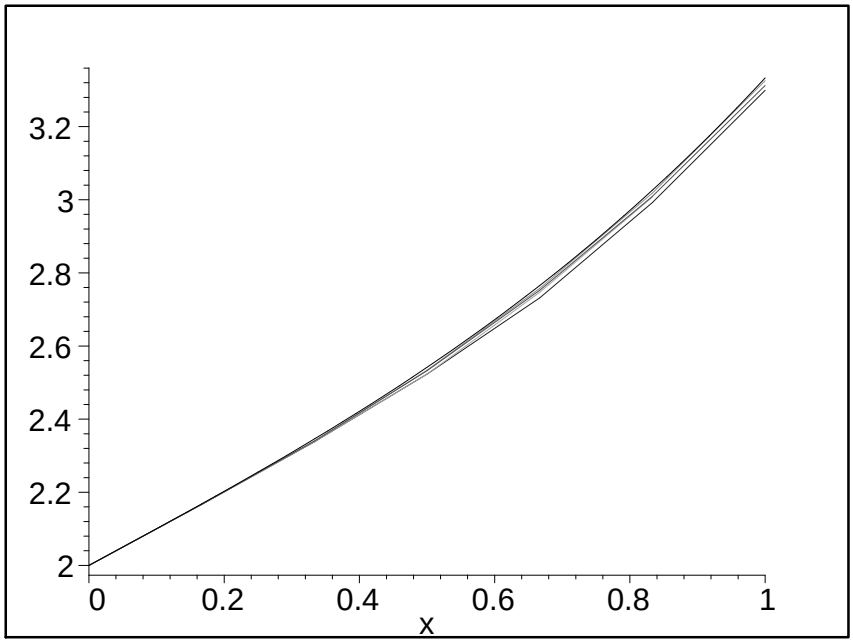
În continuareprezentăm pe acelaşi sistem de coordonate soluţia exactă a problemei rezolvate anterior şi soluţiile obţinute cu metoda Adams-Bashforth, în care am variat ordinul metodei dar am păstrat constant numărul de puncte în care s-a calculat soluţia.

```
with(plots):

p0:=plot(1/3*x^3+x+2, x=0..1,color=black, thickness=2):

p1:=plot([
       adamsbashforth(diff(f(x),x)=x^2+1,0,2,0..1,2,6),
       adamsbashforth(diff(f(x),x)=x^2+1,0,2,0..1,3,6),
       adamsbashforth(diff(f(x),x)=x^2+1,0,2,0..1,4,6),
       adamsbashforth(diff(f(x),x)=x^2+1,0,2,0..1,5,6)
    ]):

display(p0,p1);
```



7.5 Metoda Adams-Moulton

7.5.1 Breviar teoretic

Metoda Adams-Moulton foloseşte o formulă implicită pentru a găsi elementul y_{i+1} al soluţiei unei probleme cu date iniţiale.
Metoda Adams Moulton de ordinul 2:

$$y_{i+1} = y_i + \frac{h}{2}\left[f(x_i, y_i) + f(x_{i+1}, y_{i+1})\right]. \tag{7.19}$$

Metoda Adams-Moulton de ordinul 3:

$$y_{i+1} = y_i + \frac{h}{12}\left[5\, f(x_{i+1}, y_{i+1}) + 8\, f(x_i, y_i) - f(x_{i-1}, y_{i-1})\right]. \tag{7.20}$$

Metoda Adams-Moulton de ordinul 4:

$$y_{i+1} = y_i + \frac{h}{24}\left[9\, f(x_{i+1}, y_{i+1}) + 19\, f(x_i, y_i) - 5\, f(x_{i-1}, y_{i-1}) + f(x_{i-2}, y_{i-2})\right]. \tag{7.21}$$

Metoda Adams-Moulton de ordinul 5:

$$\begin{aligned} Y_{i+1} = Y_i + \frac{h}{720}\left[251\, f(x_{i+1}, Y_{i+1}) + 646\, f(x_i, Y_i) - 264\, f(x_{i-1}, Y_{i-1}) + \right. \\ \left. + 106\, f(x_{i-2}, Y_{i-2}) - 19\, f(x_{i-3}, Y_{i-3})\right] \end{aligned} \tag{7.22}$$

Observaţia 7.5.1. Formula Adams-Moulton de ordin n se foloseşte împreună cu o formulă Adams-Bashforth de ordin egal sau superior în metode de tip predictor-corector (vezi paragraful 7.6).

7.6 Metoda predictor-corector

7.6.1 Breviar teoretic

Combinaţia unei metode explicite folosită pentru predicţia valorii şi a unei metode implicite folosită pentru corectarea valorii, se numeşte metodă predictor-corector.

Dacă ordinul metodei predictor este cel puţin egal cu ordinul metodei corector, atunci este suficientă o singură iteraţie pentru a păstra acurateţea metodei corector.

Cea mai răspândită metodă predictor-corector este combinaţia formulei de ordinul patru a lui Adams-Bashforth ca predictor, cu formula de ordinul patru a lui Adams-Moulton ca şi corector:

$$\begin{cases} y_{i+1}^{(p)} &= y_i + \dfrac{h}{24}\left[55\, f(x_i, y_i) - 59\, f(x_{i-1}, y_{i-1}) + \right. \\ & \left. + 37\, f(x_{i-2}, y_{i-2}) - 9\, f(x_{i-3}, y_{i-3})\right] \\ y_{i+1} &= y_i + \dfrac{h}{24}\left[9\, f(x_{i+1}, y_{i+1}^{(p)}) + 19\, f(x_i, y_i) - \right. \\ & \left. - 5\, f(x_{i-1}, y_{i-1}) + f(x_{i-2}, y_{i-2})\right] \end{cases} \tag{7.23}$$

7.6.2 Probleme rezolvate

Exerciţiul 7.6.1. Folosind metoda predictor-corector, să se rezolve problema cu date iniţiale

$$\begin{cases} y'(x) = 1 + x^2 \\ y(0) = 2 \end{cases}$$

pe intervalul $[0, 1]$, pentru $h = 0.1$.

Rezolvare

Avem $x_0 = 0$ şi $y_0 = 2$. Cu ajutorul formulei lui Euler cu diferenţe finite înainte, obţinem

$$\begin{array}{ll} x_1 = 0.1000000000 & y_1 = 2.100000000 \\ x_2 = 0.2000000000 & y_2 = 2.201000000 \\ x_3 = 0.3000000000 & y_3 = 2.305000000 \end{array}$$

Aplicând formula (7.23) obţinem:

$$\begin{aligned}
x_4 &= 0.4000000000 & y_4^p &= 2.417333333\\
& & y_4 &= 2.417333333\\
x_5 &= 0.5000000000 & y_5^p &= 2.537666666\\
& & y_5 &= 2.537666666\\
x_6 &= 0.6000000000 & y_6^p &= 2.667999999\\
& & y_6 &= 2.667999999\\
x_7 &= 0.7000000000 & y_7^p &= 2.810333332\\
& & y_7 &= 2.810333332\\
x_8 &= 0.8000000000 & y_8^p &= 2.966666665\\
& & y_8 &= 2.966666665\\
x_9 &= 0.8000000000 & y_9^p &= 3.138999998\\
& & y_9 &= 3.138999998\\
x_{10} &= 1.000000000 & y_{10}^p &= 3.329333331\\
& & y_{10} &= 3.329333331.
\end{aligned}$$

7.6.3 Probleme propuse

Exerciţiul 7.6.2. Folosind metoda predictor-corector, să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'(x) = y(x) + x \\ y(1) = 2 \end{cases}$$

împărţind intervalul $[1, 2]$ în $N = 8$. Comparaţi cu soluţia exactă.

7.6.4 Implementare

A. Algoritmi

Date de intrare: o ecuaţie $y'(x) = f(x, y(x))$, valorile iniţiale x_0 şi y_0, un interval $[a, b]$ şi numărul de puncte N în care se împarte intervalul

Date de ieşire: soluţia aproximativă a problemei cu date iniţiale, dată sub forma unui şir de puncte, rez

Algoritmul constă în următorii paşi:

1. identificarea funcţiei f
   - $h = \dfrac{b-a}{N}$
2. găsirea soluţiei aproximative

   adaugă elementul (x_0, y_0) la lista rez

   pentru $i = \overline{1,3}$

   $y_{i+1} = y_i + h\, f(x_i, y_i)$

$x_{i+1} = x_i + h$

adaugă punctul (x_{i+1}, y_{i+1}) la lista rez

pentru $i = \overline{4, N}$

calculează y_i^p din ecuaţia (7.23)

calculează y_i din ecuaţia (7.23)

$x_{i+1} = x_i + h$

adaugă punctul (x_{i+1}, y_{i+1}) la lista rez

B. Programe MAPLE şi rezultate

```
predictorcorector:=proc(eq::equation, x0::numeric, y0::numeric,
  ab::range, N::integer)
local f,varx,h,y1,x1,rez,i;
if StringTools[Has](convert(lhs(eq),string),"diff") then
    f:=rhs(eq);
  elif StringTools[Has](convert(rhs(eq),string),"diff") then
    f:=lhs(eq);
  else
    ERROR (‘ecuatia trebuie sa fie de tipul diff(y(x),x)=f(x,y)‘);
fi;
varx:=[op(indets(eq, name))];
if nops(varx)=1 then varx:=[op(varx),0]; fi;
h:=(rhs(ab)-lhs(ab))/N;
y1:=evalf(y0);
x1:=evalf(x0);
rez:=[[x1,y1]];
for i from 1 to 3 do
  y1:=evalf(y1+h*subs(varx[1]=rez[nops(rez)][1],
                      varx[2]=rez[nops(rez)][2],f));
  x1:=evalf(x1+h);
  rez:=[op(rez),[x1,y1]];
od;
for i from 4 to N do
  y1:=evalf(
      y1+h/24*( 55*subs(varx[1]=rez[nops(rez)][1],
                        varx[2]=rez[nops(rez)][2],f) -
                59*subs(varx[1]=rez[nops(rez)-1][1],
                        varx[2]=rez[nops(rez)-1][2],f) +
                37*subs(varx[1]=rez[nops(rez)-2][1],
                        varx[2]=rez[nops(rez)-2][2],f) -
                 9*subs(varx[1]=rez[nops(rez)-3][1],
                        varx[2]=rez[nops(rez)-3][2],f)
           ));
  y1:=evalf(
```

```
      rez[nops(rez)][2]+h/24*(
                          9*subs(varx[1]=rez[nops(rez)][1]+h,
                                 varx[2]=y1,f) +
                         19*subs(varx[1]=rez[nops(rez)][1],
                                 varx[2]=rez[nops(rez)][2],f) -
                          5*subs(varx[1]=rez[nops(rez)-1][1],
                                 varx[2]=rez[nops(rez)-1][2],f) +
                            subs(varx[1]=rez[nops(rez)-2][1],
                                 varx[2]=rez[nops(rez)-2][2],f)
           ));
  x1:=x1+h;
  rez:=[op(rez),[x1,y1]];
od;
end:
```

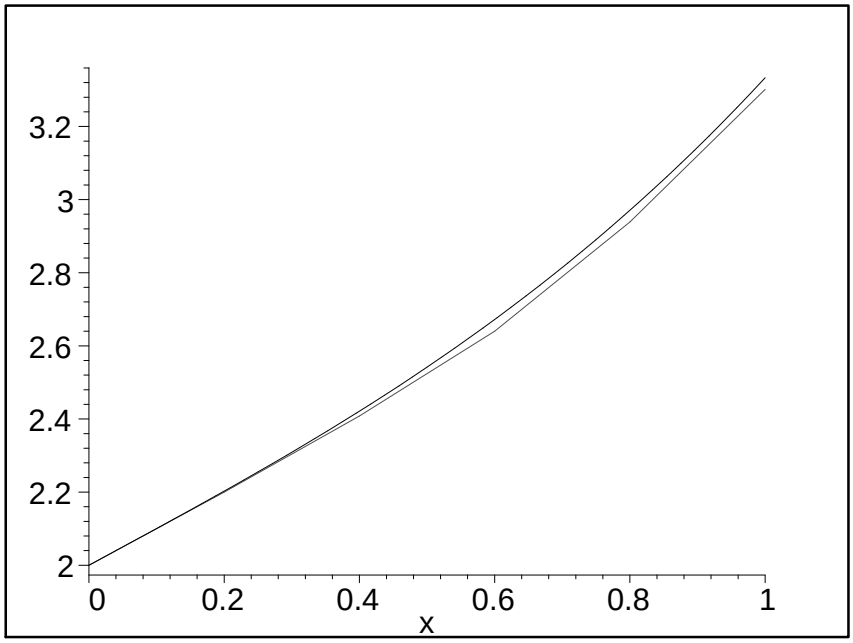
În continuare prezentăm în acelaşi sistem de coordonate soluţia exactă a problemei rezolvate anterior şi soluţiile obţinute cu metoda predictor-corector, în care am variat ordinul metodei dar am păstrat constant numărul de puncte în care s-a calculat soluţia.

```
with(plots):

p0:=plot(1/3*x^3+x+2, x=0..1,color=black, thickness=2):

p2:=plot([seq(predictorcorector(diff(f(x),x)=x^2+1,0,2,0..1,N),
          N=5..10)]):

display(p0,p1);
```



Rezolvarea problemelor la limită liniare pentru ecuaţii diferenţiale de ordinul al doilea

7.7 Metoda diferenţelor finite pentru rezolvarea unei probleme la limită liniare

7.7.1 Breviar teoretic

Se consideră problema

$$y'' = p(x) \cdot y' + q(x) \cdot y + r(x)\, x \in [a, b] \tag{7.24}$$

cu condiţiile la frontieră mixte:

$$\begin{cases} \gamma_1 \cdot y(a) + \gamma_2 \cdot y'(a) = \alpha \\ \gamma_3 \cdot y(b) + \gamma_4 \cdot y'(b) = \beta. \end{cases} \tag{7.25}$$

Dacă $\gamma_2, \gamma_4 = 0$, atunci vorbim de condiţii la limită Dirichlet, iar dacă $\gamma_1, \gamma_3 = 0$, atunci vorbim de condiţii la limită Neumann.

Dacă împărţim intervalul $[a, b]$ în $N + 1$ intervale de lungime $h = \dfrac{b-a}{N+1}$ şi notăm $Y_i = y(x_i)$, atunci determinarea valorilor Y_i se reduce la rezolvarea sistemului

$$AY = B \tag{7.26}$$

unde

$$A = \begin{bmatrix} b_1 - \frac{4a_1\gamma_2}{2h\gamma_1 - 3\gamma_2} & c_1 + \frac{a_1\gamma_2}{2h\gamma_1 - 3\gamma_2} & 0 & \dots & 0 & 0 \\ a_2 & b_2 & c_2 & \dots & 0 & 0 \\ 0 & a_3 & b_3 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & b_{N-1} & c_{N-1} \\ 0 & 0 & 0 & \dots & a_N - \frac{c_N\gamma_4}{2h\gamma_3 + 3\gamma_4} & b_N + \frac{4c_N\gamma_4}{2h\gamma_3 + 3\gamma_4} \end{bmatrix} \tag{7.27}$$

$$Y = (Y_1, Y_2, ..., Y_N)^T \tag{7.28}$$

$$B = \begin{bmatrix} h^2 r(x_1) - \dfrac{2h\alpha a_1}{2h\gamma_1 - 3\gamma_2} \\ h^2 r(x_2) \\ \vdots \\ h^2 r(x_{N-1}) \\ h^2 r(x_N) - \dfrac{2h\beta c_N}{2h\gamma_3 + 3\gamma_4} \end{bmatrix} \tag{7.29}$$

iar

$$a_i = 1 + \frac{h}{2}\, p(x_i) \tag{7.30}$$

$$b_i = -(2 + h^2 q(x_i)) \tag{7.31}$$

$$c_i = 1 - \frac{h}{2}\, p(x_i). \tag{7.32}$$

În capetele intervalului, valorile aproximative ale soluţiei sunt date de:

$$y(x_0) = \frac{h\alpha - \gamma_2 y_1}{h\gamma_1 + \gamma_2} \tag{7.33}$$

şi

$$y(x_{N+1}) = \frac{h\beta - \gamma_4 y_N}{h\gamma_3 + \gamma_4} \tag{7.34}$$

7.7.2 Problemă rezolvată

Exerciţiul 7.7.1. Să se găsească soluţia aproximativă a problemei următoare

$$\begin{cases} y'' = x \\ y(0) = 0 \\ y(1) = 1 \end{cases}$$

pe intervalul intervalul $[0, 1]$ $(N = 3)$.

Rezolvare Avem:

$$\begin{array}{lll} p(x) = 0 & q(x) = 0 & r(x) = x \\ \gamma_1 = 1 & \gamma_2 = 0 & \alpha = 0 \\ \gamma_3 = 1 & \gamma_4 = 0 & \beta = 1 \\ h = \dfrac{1-0}{3+1} = \dfrac{1}{4} & & \\ x_1 = \dfrac{1}{4} & x_2 = \dfrac{2}{4} & x_3 = \dfrac{3}{4} \end{array}$$

Înlocuind aceaste valori, se obţine sistemul

$$Ay = b$$

unde

$$A = \begin{pmatrix} -2 & 1 & 0 \\ 1 & -2 & 1 \\ 0 & 1 & -2 \end{pmatrix} \qquad b = \begin{pmatrix} \frac{1}{64} \\ \frac{2}{64} \\ -\frac{61}{64} \end{pmatrix}$$

şi a cărui soluţie este:

$$y = \begin{pmatrix} \frac{27}{128} \\ \frac{56}{128} \\ \frac{89}{128} \end{pmatrix}.$$

Valorile soluţiei aproximative în capetele intervalului sunt

$$y(0) = 0 \qquad y(1) = 1.$$

Astfel, se obţine soluţia aproximativă a problemei date sub forma unui şir de puncte:

$$\left[(0,0), (\frac{1}{4}, \frac{27}{128}), (\frac{2}{4}, \frac{56}{128}), (\frac{3}{4}, \frac{89}{128}), (1,1)\right].$$

7.7.3 Probleme propuse

Exerciţiul 7.7.2. Să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'' = 2y' + 3y - 1 \\ y(0) = 1 \\ y(1) = 2 \end{cases}$$

pe intervalul intervalul $[0, 1]$ ($N = 4$). Comparaţi cu soluţia exactă.

Exerciţiul 7.7.3. Să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'' = 2y' + 3y - 1 \\ 2y(0) + y'(0) = 1 \\ y'(1) = 1 \end{cases}$$

pe intervalul intervalul $[0, 1]$ ($N = 4$).

7.7.4 Implementare

A. Algoritmi

Date de intrare: ecuaţia $ec1$, condiţiile la frontieră $ec2$, $ec3$, şi numărul de puncte în care se caută soluţia, N

Date de ieşire: lista rez a punctelor care definesc soluţia

Algoritmul constă următoarele etape:

1. identificarea intervalului $[a, b]$,a funcţiilor p, q, r precum şi a coeficienţilor γ_1, γ_2, γ_3, γ_4, α, β
2. determinarea punctelor x_i în care se calculează soluţia, a matricei A şi a vectorului B
3. rezolvarea sistemului $AY = B$
4. returnarea listei de puncte de forma $[(x_i, Y_i)]$ care alcătuiesc soluţia numerică a problemei

B. Programe MAPLE şi rezultate

Observaţia 7.7.1. Având ecuaţia şi condiţiile pe frontieră, se pot determina: intervalul $[a, b]$, funcţiile p, q, r, numele variabilei în care se scrie ecuaţia, x, precum şi coeficienţii γ_1, γ_2, γ_3, γ_4, α, β. Pentru aceasta, am construit procedura `coeficienti`.

```
coeficienti:=proc(ec1::equation, ec2::equation, ec3::equation)
local f,xx,p,q,r,m,A,B,g1,g2,g3,g4;
#ecuatia ec1
if StringTools[Has](convert(lhs(ec1),string),"$2") or
   StringTools[Has](convert(lhs(ec1),string),"diff(diff")
```

```
  then f:=rhs(ec1);
  elif StringTools[Has](convert(rhs(ec1),string),"$2") or
       StringTools[Has](convert(lhs(ec1),string),"diff(diff")
  then f:=lhs(ec1);
  else
    ERROR (‘prima ecuatie trebuie sa fie de tipul \n
            diff(y(x),x$2)=p(x)*diff(y(x),x)+q(x)*y(x)+r(x)‘);
  fi;
if nops(indets(f,function)[1])=1 then
  xx:=op(indets(f,function)[1]);
  p:=coeff(f,indets(f,function)[2]);
  q:=coeff(f-p*indets(f,function)[2],indets(f,function)[1]);
  r:=f-p*indets(f,function)[2]-q*indets(f,function)[1];
else
  xx:=op(indets(f,function)[2]);
  p:=coeff(f,indets(f,function)[1]);
  q:=coeff(f-p*indets(f,function)[2],indets(f,function)[2]);
  r:=f-p*indets(f,function)[1]-q*indets(f,function)[2];
fi;
#capetele intervalului
m:=indets(lhs(ec2),{numeric,name});
A:=product(m[k],k=1..nops(m));
m:=indets(lhs(ec3),{numeric,name});
B:=product(m[k],k=1..nops(m));
#conditiile la frontiera
m:=[op(indets(ec2,function))];
if nops(m)=1 then m:=[op(m),0];fi;
if  not StringTools[Has](convert(m[1],string),"diff") then
  g1:=coeff(m[1],op(indets(m[1],function)));
  if m[2]=0 then g2:=0
  else g2:=coeff(m[2],op(indets(m[2],function)));
  fi;
else
  g2:=coeff(m[1],op(indets(m[1],function)));
  if m[2]=0 then g1:=0
  else g1:=coeff(m[2],op(indets(m[2],function)));
  fi;
fi;
m:=[op(indets(ec3,function))];
if nops(m)=1 then m:=[op(m),0];fi;
if  not StringTools[Has](convert(m[1],string),"diff") then
  g3:=coeff(m[1],op(indets(m[1],function)));
  if m[2]=0 then g4:=0
  else g4:=coeff(m[2],op(indets(m[2],function)));
  fi;
else
```

```
  g4:=coeff(m[1],op(indets(m[1],function)));
  if m[2]=0 then g3:=0
  else g3:=coeff(m[2],op(indets(m[2],function)));
  fi;
fi;
RETURN(A,B,xx,p,q,r,g1,g2,g3,g4,rhs(ec2),rhs(ec3));
end:
```

Procedura care returnează şirul de puncte care aproximează soluţia este `bvproblem`.

```
bvproblem:=proc(ec1::equation, ec2::equation, ec3::equation,
  N::integer)
local f,A,B,p,q,r,h,g1,g2,g3,g4,a,b,i,x,xx,AA,BB,yy;
f:=coeficienti(ec1,ec2,ec3);
A:=f[1];B:=f[2];xx:=f[3];
p:=f[4];q:=f[5];r:=f[6];
g1:=f[7];g2:=f[8];g3:=f[9];g4:=f[10];a:=f[11];b:=f[12];
h:=(B-A)/(N+1);
for i from 1 to N do
  x[i]:=evalf(A+i*h);
od;
AA:=matrix(N,N,0);
BB:=vector(N,0);
for i from 1 to N do
  AA[i,i]:=evalf(-(2+h^2*subs(xx=x[i],q)));
  BB[i]:=evalf(h^2*subs(xx=x[i],r));
od;
for i from 1 to N-1 do
  AA[i,i+1]:=evalf(1-h/2*subs(xx=x[i],p));
  AA[i+1,i]:=evalf(1+h/2*subs(xx=x[i+1],p));
od;
AA[1,1]:=evalf(AA[1,1]-4*(1+h/2*subs(xx=x[1],p))
                        *g2/(2*h*g1-3*g2));
AA[1,2]:=evalf(AA[1,2]+(1+h/2*subs(xx=x[1],p))
                              *g2/(2*h*g1-3*g2));
AA[N,N-1]:=evalf(AA[N,N-1]-(1-h/2*subs(xx=x[N],p))
                                 *g4/(2*h*g3+3*g4));
AA[N,N]:=evalf(AA[N,N]+4*(1-h/2*subs(xx=x[N],p))
                                *g4/(2*h*g3+3*g4));
BB[1]:=evalf(BB[1]-2*h*a*(1+h/2*subs(xx=x[1],p))/
             (2*h*g1-3*g2));
BB[N]:=evalf(BB[N]-2*h*b*(1-h/2*subs(xx=x[N],p))/
             (2*h*g3+3*g4));
evalm(AA);evalm(BB);
yy:=linsolve(AA,BB);
RETURN([
  [evalf(A), evalf(( h*a-g2*yy[1] )/(h*g1+g2)) ],
```

```
  seq([x[i],yy[i]],i=1..N),
  [evalf(B), evalf(( h*b-g4*yy[N] )/(h*g3+g4)) ]
]);
end:
```

Testăm această procedură pentru calculul soluţiei aproximative a problemei

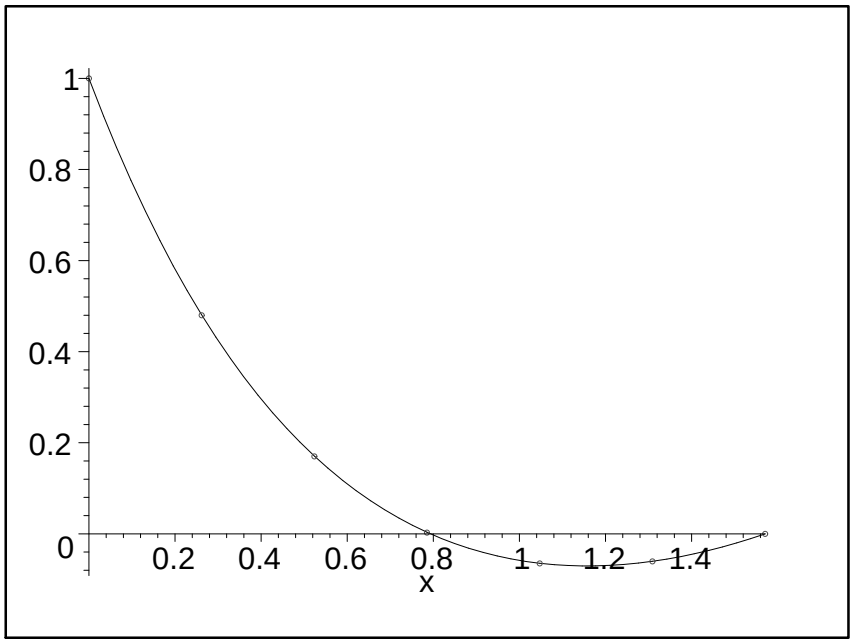
$$\begin{cases} y'' = -y' + y + 1 \\ y(0) = 1 \\ y'(\frac{\pi}{2}) = 0 \end{cases}$$

Pentru comparaţie, am reprezentat grafic în acelaşi sistem de coordonate soluţia formală a problemei, obţinută cu ajutorul procedurii predefinite `dsolve` şi punctele obţinute aplicând procedura `bvproblem`.

```
ec1:=diff(y(x),x$2)=-diff(y(x),x)+y(x)+1:
ec2:=y(0)=1:
ec3:=y(Pi/2)=0:

with(plots):
y1:=rhs(dsolve({ec1,ec2,ec3},y(x)));
p0:=plot(y1,x=0..Pi/2,thickness=2,color=black):
p1:=pointplot(bvproblem(ec1,ec2,ec3,5), symbol=circle,
  symbolsize=10, color=red):
display(p0,p1);
```

$$y1 := -\frac{e^{(\frac{(\sqrt{5}-1)\,x}{2})}\,(-1+2\,e^{(-\frac{(\sqrt{5}+1)\,\pi}{4})})}{e^{(\frac{(\sqrt{5}-1)\,\pi}{4})}-e^{(-\frac{(\sqrt{5}+1)\,\pi}{4})}}+\frac{e^{(-\frac{(\sqrt{5}+1)\,x}{2})}\,(2\,e^{(\frac{(\sqrt{5}-1)\,\pi}{4})}-1)}{e^{(\frac{(\sqrt{5}-1)\,\pi}{4})}-e^{(-\frac{(\sqrt{5}+1)\,\pi}{4})}}-1$$



7.8 Metoda colocaţiei şi metoda celor mai mici pătrate

7.8.1 Breviar teoretic

Se consideră problema

$$y'' + p(x) \cdot y' + q(x) \cdot y + r(x) = f(x), \quad x \in [a, b] \tag{7.35}$$

cu condiţiile la limită mixte,

$$\begin{cases} \gamma_1 \cdot y(a) + \gamma_2 \cdot y'(a) = \alpha \\ \gamma_3 \cdot y(b) + \gamma_4 \cdot y'(b) = \beta \end{cases} \tag{7.36}$$

Căutăm o soluţie a ecuaţiei (7.35) de forma

$$Y_N(x) = \Phi_0(x) + \sum_{i=1}^{N} c_i \cdot \Phi_i(x) \tag{7.37}$$

unde $\{\Phi_0, \Phi_1, \ldots, \Phi_N\}$ sunt funcţii de clasă $\mathcal{C}^2$ liniar independente, care verifică:

$$\begin{aligned} &\gamma_1 \cdot \Phi_0(a) + \gamma_2 \cdot \Phi_0'(a) = \alpha \quad \text{şi} \quad \gamma_3 \cdot \Phi_0(b) + \gamma_4 \cdot \Phi_0'(b) = \beta \\ &\gamma_1 \cdot \Phi_i(a) + \gamma_2 \cdot \Phi_i'(a) = 0 \quad \text{şi} \quad \gamma_3 \cdot \Phi_i(b) + \gamma_4 \cdot \Phi_i'(b) = 0, \ i = \overline{1, N}. \end{aligned} \tag{7.38}$$

Metoda colocaţiei presupune găsirea coeficienţilor c_i din sistemul de N ecuaţii cu N necunoscute

$$\begin{aligned} &\sum_{i=1}^{N} c_i [\Phi_i''(x_k) + p(x_k) \cdot \Phi_i'(x_k) + q(x_k) \cdot \Phi_i(x_k)] = \\ &\quad = f(x_k) - \Phi_0''(x_k) - p(x_k) \cdot \Phi_0'(x_k) - q(x_k) \cdot \Phi_0(x_k), \ k = \overline{1, N} \end{aligned} \tag{7.39}$$

Metoda celor mai mici pătrate presupune găsirea coeficienţilor c_i din sistemul de N ecuaţii cu N necunoscute

$$\begin{aligned} &\sum_{i=1}^{N} c_j \int_a^b [\Phi_j''(x) + p(x) \cdot \Phi_j'(x) + q(x) \cdot \Phi_j(x)] \cdot \\ &\quad \cdot [\Phi_i''(x) + p(x) \cdot \Phi_i'(x) q(x) \cdot \Phi_i(x)] dx = \\ &= - \int_a^b [\Phi_0''(x) + p(x) \cdot \Phi_0'(x) + q(x) \cdot \Phi_0(x) - f(x)] \cdot \\ &\quad \cdot [\Phi_i''(x) + p(x) \cdot \Phi_i'(x) q(x) \cdot \Phi_i(x)] dx. \end{aligned} \tag{7.40}$$

7.8.2 Probleme rezolvate

Exerciţiul 7.8.1. Să se găsească soluţia problemei

$$\begin{cases} y'' + y' = x \\ y(0) = 1 \\ y(1) = 1 \end{cases}$$

folosind:

a. metoda colocaţiei, cu $N = 3$

b. metoda celor mai mici pătrate, cu $N = 3$

Indicaţie: se consideră $\Phi_0(x) = 0$ şi $\Phi_i(x) = \sin i\pi x$, $i = \overline{1,3}$.

Rezolvare

Se observă că funcţiile $\Phi_0(x) = 0$ şi $\Phi_i(x) = \sin i\pi x$, $i = \overline{1,3}$ sunt funcţii de clasă $\mathcal{C}^2$, liniar independente, şi verifică condiţiile (7.38). De asemenea, avem:

$$p(x) = 1 \quad q(x) = 1 \quad f(x) = x.$$

şi $x_i = 0 + \frac{i}{4}$, $i = \overline{1,3}$.

A. Rezolvare folosind metoda colocaţiei

Soluţia problemei este de forma:

$$y_3(x) = \Phi_0(x) + c_1\,\Phi_1(x) + c_2\,\Phi_2(x) + c_3\,\Phi_3(x).$$

Înlocuind în sistemul (7.39), obţinem următorul sistem:

$$\begin{cases} \dfrac{1-\pi^2}{\sqrt{2}}c_1 + (1-4\pi^2)c_2 + \dfrac{1-9\pi^2}{\sqrt{2}}c_3 = \dfrac{1}{4} \\ (1-\pi^2)c_1 - (1-9\pi^2)c_3 = \dfrac{2}{4} \\ \dfrac{1-\pi^2}{\sqrt{2}}c_1 - (1-4\pi^2)c_2 + \dfrac{1-9\pi^2}{\sqrt{2}}c_3 = \dfrac{3}{4} \end{cases}$$

a cărui soluţie este:

$$c_1 = \frac{\sqrt{2}+1}{4(1-\pi^2)} \quad c_2 = -\frac{1}{4(1-4\pi^2)} \quad c_3 = \frac{\sqrt{2}-1}{4(1-9\pi^2)}$$

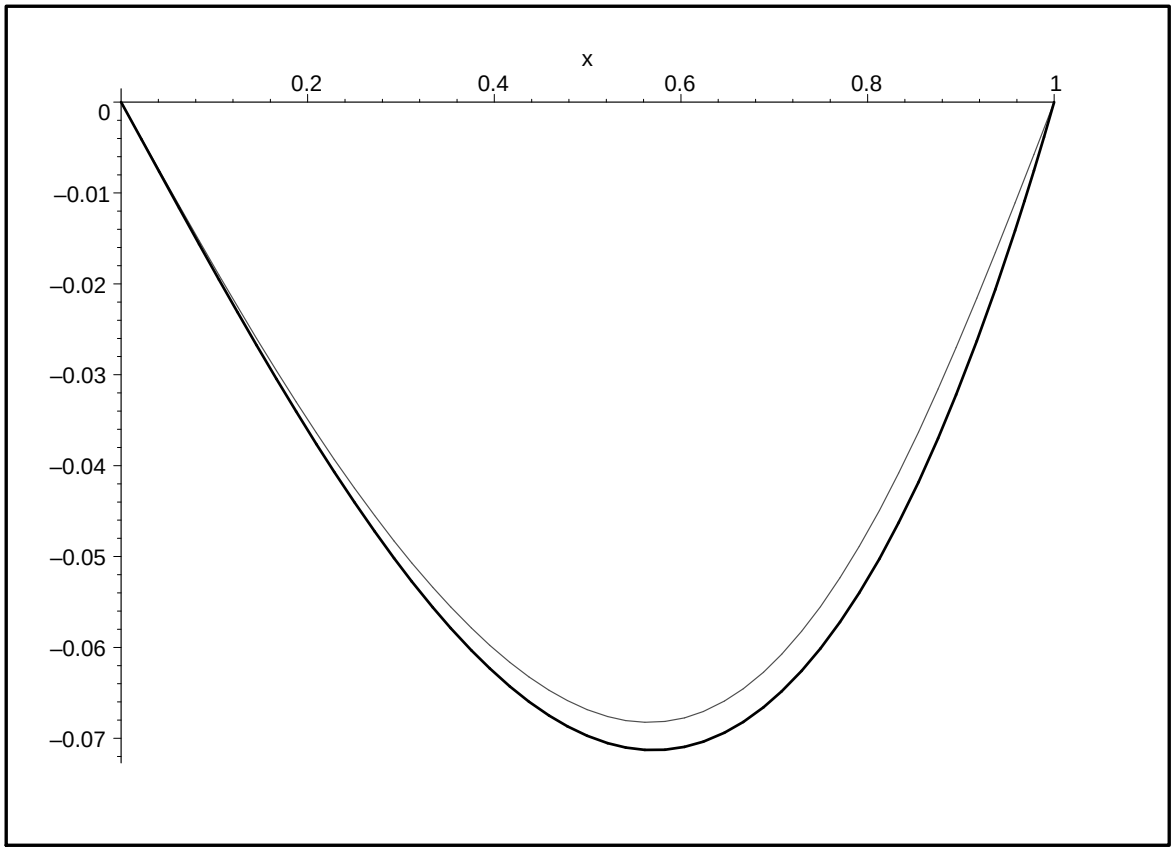
De aici rezultă că soluţia aproximativă a problemei este:

$$y_3(x) = \frac{\sqrt{2}+1}{4(1-\pi^2)}\,\sin\pi x - \frac{1}{4(1-4\pi^2)}\,\sin 2\pi x + \frac{\sqrt{2}-1}{4(1-9\pi^2)}\,\sin 3\pi x.$$

Soluţia exactă a problemei date este

$$y(x) = x - \frac{\sin x}{\sin 1}$$

Reprezentăm în continuare pe acelaşi sistem de coordonate, soluţia exactă a problemei iniţiale (cu linie îngroşată) şi soluţia obţinută folosind metoda colocaţiei:



B. Rezolvare folosind metoda celor mai mici pătrate

Notăm

$$f_j(x) = \Phi_j''(x) + p(x)\cdot\Phi_j'(x) + q(x)\cdot\Phi_j(x)\,,\ j = \overline{1,3}$$

Cu această notaţie, sistemul (7.40) devine:

$$\begin{cases} c_1\displaystyle\int_0^1 f_1(x)\cdot f_1(x) + c_2\int_0^1 f_2(x)\cdot f_1(x) + c_3\int_0^1 f_3(x)\cdot f_1(x) = \int_0^1 x\cdot f_1(x) \\ c_1\displaystyle\int_0^1 f_1(x)\cdot f_2(x) + c_2\int_0^1 f_2(x)\cdot f_2(x) + c_3\int_0^1 f_3(x)\cdot f_2(x) = \int_0^1 x\cdot f_2(x) \\ c_1\displaystyle\int_0^1 f_1(x)\cdot f_3(x) + c_2\int_0^1 f_2(x)\cdot f_3(x) + c_3\int_0^1 f_3(x)\cdot f_3(x) = \int_0^1 x\cdot f_3(x) \end{cases}$$

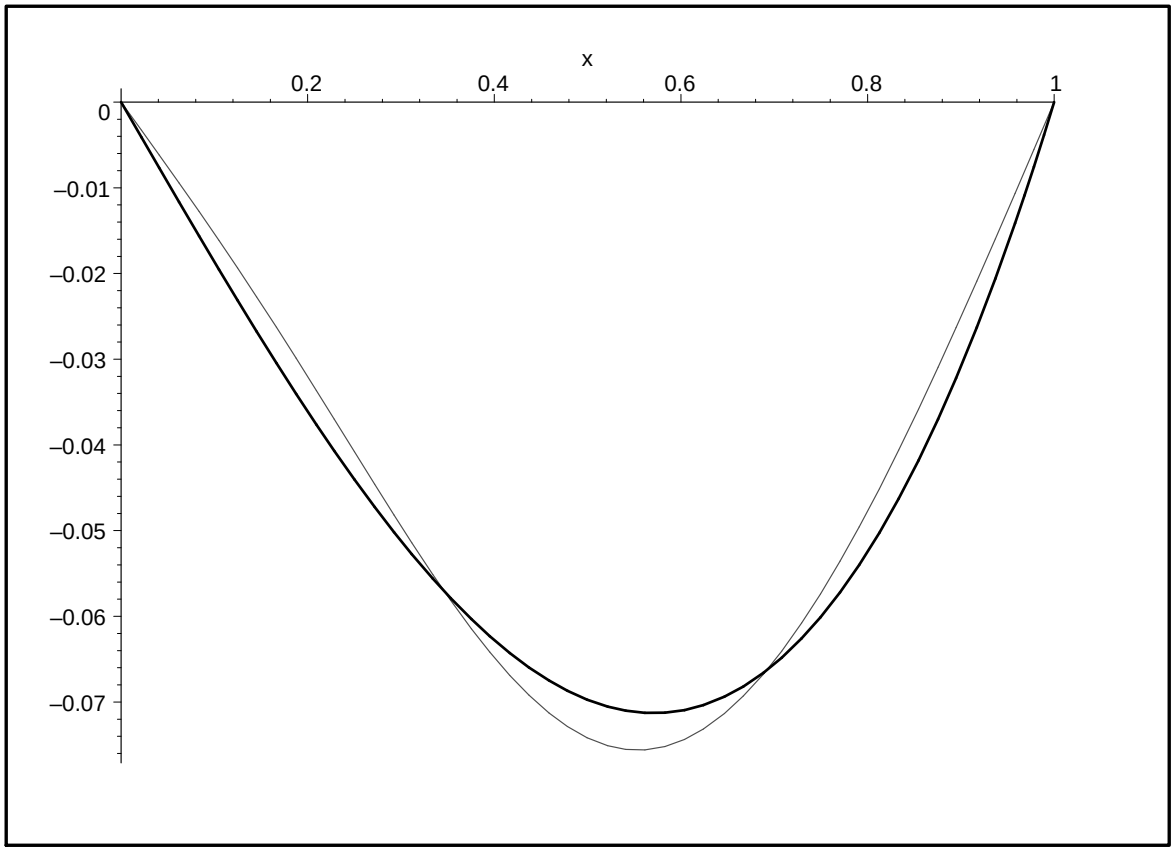
Soluţia acestui sistem este:

$$c_1 = \frac{2}{\pi(1-\pi^2)} \quad c_2 = -\frac{1}{\pi(1-4\pi^2)} \quad c_3 = -\frac{2}{3\pi(1-9\pi^2)}.$$

De aici rezultă că soluţia aproximativă a problemei, obţinută cu metoda celor mai mici pătrate, este:

$$y_3(x) = \frac{2}{\pi(1-\pi^2)}\sin\pi x - \frac{1}{\pi(1-4\pi^2)}\sin 2\pi x - \frac{2}{3\pi(1-9\pi^2)}\sin 3\pi x.$$

Reprezentăm în continuare pe acelaşi sistem de coordonate, soluţia exactă a problemei iniţiale (cu linie îngroşată) şi soluţia obţinută folosind metoda celor mai mici pătrate:



7.8.3 Probleme propuse

Exerciţiul 7.8.2. Să se găsească soluţia aproximativă a problemei

$$\begin{cases} y'' + y' = x \\ y(0) = 0 \\ y(1) = 0 \end{cases}$$

pe intervalul $[0, 1]$ folosind:

a. metoda colocaţiei

b. metoda celor mai mici pătrate

folosind ca bază funcţiile $\Phi_0(x) = 0$, $\Phi_1(x) = x(1-x)$, $\Phi_2(x) = x^2(1-x)$.

BIBLIOGRAFIE

[1] Şt. Balint, L. Brăescu, N. Bonchiş, *Metode numerice* Timişoara, 2007

[2] C. Berbente, S. Mitran, S. Zancu, *Metode Numerice*, Ed. Tehnică, Bucureşti, 1998.

[3] T.A. Beu, *Calcul numeric în C*, Ediţia a 2-a, Ed. Alabastra, Cluj-Napoca, 2000.

[4] G. Coman, *Analiză Numerică*, Ed. Libris, Cluj-Napoca, 1995.

[5] M. Dinu, G. Lincă, *Algoritmi si teme speciale de analiză numerică* Ed. Matrix Rom, Bucureşti, 1999.

[6] O. Dogaru, Gh. Bocsan, I. Despi, A. Ionica, V. Iordan, L. Luca, D. Petcu, P. Popovici *Informatică pentru definitivare si grad*, Ed. de Vest, Timisoara, 1998.

[7] W. Kelley, A. Peterson, *Difference equation, An Introduction with Applications*, Academic Press, Elsevier, 2000.

[8] Şt. Maruşter, *Metode numerice în rezolvarea ecuaţiilor neliniare*, Ed. Tehnică, Bucureşti, 1981.

[9] P. Naslau, R. Negrea, L. Cadariu, B. Caruntu, D. Popescu, M. Balmez, C. Dumitrascu, *Matematici asistate pe calculator*, Ed. Politehnica, Timisoara, 2005.

[10] D. Petcu, *Maple, un standard pentru matematica computerizată*, Tipografia UVT, Timişoara, 1997

[11] V. A. Patel, *Numerical Analysis*, Humboldt State University, USA, 1994.

[12] MapleV4 - pagina de help